Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini , Simone Licciardi

Report for Project 1

Due date: 09 March 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Euler warm-up [10 points]

Question 1: What is the module system, and how is it used?
The module system (Lmod, a Lua-based environment modules implementation) is used on HPC systems to dynamically manage software environments. It allows users to load, unload, and switch between different software versions without manually modifying system paths.
Loading all software globally would lead to conflicts between different versions of compilers and libraries. The module system avoids this problem by dynamically modifying environment variables.
In practice, before compiling or running programs on Euler, users load the required modules (e.g., compilers or libraries) inside their batch scripts using commands such as `module load`.

Question 2: What is Slurm, and what is its purpose?
Slurm is an open-source workload manager and job scheduler for Linux and Unix-like systems, widely used by supercomputers and clusters. It schedules and executes jobs on compute nodes, handling resource allocation, queuing, and execution control.
In particular (from the Slurm documentation), it provides three main functions:

- it allocates exclusive and/or non-exclusive access to resources (compute nodes, CPUs, memory, GPUs) to users for a specified duration of time so they can perform work;
- it provides a framework for starting, executing, and monitoring work (normally a parallel job) on the set of allocated nodes.
- it arbitrates contention for resources by managing a queue of pending jobs.

We now illustrate how the remaning questions were adressed.
The solution to question 3 and 4 can be found by first running the Makefile provided in the directory `1_Euler_Warmup` and then running the file `slurm_job_one.sh`. We already provide in the directory `1_Euler_Warmup/build_job_one` the ouputs of the batch script as requested. The output is reported also here for completeness:

```
Currently Loaded Modules:
1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 9654 96-Core Processor
eu-g9-101-4
Hello World from eu-g9-101-4
```

As we can see we have specified to use a node with AMD EPYC 9654 96-Core Processor, and the job was executed on the node `eu-g9-101-4`.
The solution to question 5 can be found by running the file `slurm_job_two.sh`. The output of the batch script is reported here for completeness:

```
Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 7763 64-Core Processor
Model name: AMD EPYC 7763 64-Core Processor
eu-a2p-487
eu-a2p-482
Hello World from eu-a2p-487
Hello World from eu-a2p-482
```

We have thus successfully executed the same job on two nodes, both with AMD EPYC 7763 64-Core Processor, and jobs were executed on the nodes `eu-a2p-487` and `eu-a2p-482` respectively.

2. Performance characteristics [50 points]

2.1. Peak performance [16 Points]

Proceeding as in the example given in the project description, the hardware specifications can be obtained from the Euler webpage.
For Euler VII (phase 1), the system consists of $n_{\text{nodes}} = 292$ nodes, each equipped with $n_{\text{sockets}} = 2$ sockets and $n_{\text{cores}} = 64$ cores per socket, operating at a frequency $f = 2.6\,\text{GHz}$.
For Euler VII (phase 2), the system consists of $n_{\text{nodes}} = 248$ nodes, each with $n_{\text{sockets}} = 2$ sockets and $n_{\text{cores}} = 64$ cores per socket, operating at a frequency $f = 2.45\,\text{GHz}$.

Using the uops.info database provided, we determine both the superscalability factor and the FMA factor for the corresponding AMD architectures.
For both Euler VII phases, which are based on AMD Zen2 and AMD Zen3 respectively, we obtain $n_{\text{super}} = 2$ and $n_{\text{FMA}} = 2$.

The FMA factor is determined by observing that all instructions marked as FMA in the table have a corresponding number of micro-operations equal to one. This indicates that the fused multiply-add is executed as a single micro-operation by dedicated hardware. Since an FMA instruction performs the operation $a \cdot b + c$, it consists of one floating-point multiplication and one floating-point addition. Therefore, each FMA instruction corresponds to two floating-point operations, and thus $n_{\text{FMA}} = 2$.

The superscalability is defined as the number of floating-point operations that the FPU of a core can execute in parallel within a single clock cycle. To determine this value, we analyze the instruction `VFMADD132PD (YMM, YMM, YMM)` reported in the uops.info table. The table specifies the throughput 0.50 cycles/instruction implies 2 vector FMA instructions can be issued per cycle, we can conclude the core can sustain two FMA pipelines, hence $n_{\text{super}} = 2$.
Therefore, for both AMD Zen2 and AMD Zen3, the superscalar throughput is 2 vector FP instructions per cycle per core.

To determine n_{SIMD}, we inspect the floating-point instructions reported in the uops.info table, where instructions such as `VFMADD132PD (YMM, YMM, YMM)` appear. The presence of YMM operands indicates the use of 256-bit AVX registers, since YMM registers are defined as 256-bit wide in the AVX/AVX2 instruction set. As both AMD Zen 2 and Zen 3 execute these YMM-based instructions, they necessarily support AVX2 with 256-bit SIMD registers. A double-precision floating-point number occupies 64 bits; therefore, the number of doubles that can be processed concurrently by one SIMD instruction is

$$n_{\text{SIMD}} = \frac{256}{64} = 4.$$

Hence, each SIMD instruction can operate on four double-precision values in parallel.

Parameter	**Euler VII (Phase 1)**	**Euler VII (Phase 2)**
n_{nodes}	292	248
n_{sockets}	2	2
n_{cores} (per socket)	64	64
Clock frequency f	2.6 GHz	2.45 GHz
n_{FMA}	2	2
n_{super} (FMA instructions per cycle per core)	2	2
n_{SIMD} (doubles per SIMD instr.)	4	4

Table 1: Summary of hardware parameters and architectural factors used for peak performance estimation.

We can now compute all the peak performance parameters for both Euler VII phases :

$$P_{\text{core},1} = n_{\text{super}} \times n_{\text{FMA}} \times n_{\text{SIMD}} \times f = 2 \times 2 \times 4 \times 2.6\,\text{GHz} = 41.6\,\text{GFLOPS/core}.$$
$$P_{\text{CPU},1} = n_{\text{cores}} \times P_{\text{core},1} = 64 \times 41.6\,\text{GFLOPS/core} = 2.6624\,\text{TFLOPS/CPU}.$$
$$P_{\text{node},1} = n_{\text{sockets}} \times P_{\text{CPU},1} = 2 \times 2.6624\,\text{TFLOPS/node} = 5.3248\,\text{TFLOPS/node}.$$
$$P_{\text{cluster},1} = n_{\text{nodes}} \times P_{\text{node},1} = 292 \times 5.3248\,\text{TFLOPS/node} = 1555.04\,\text{TFLOPS}.$$

$$P_{\text{core},2} = n_{\text{super}} \times n_{\text{FMA}} \times n_{\text{SIMD}} \times f = 2 \times 2 \times 4 \times 2.45\,\text{GHz} = 39.2\,\text{GFLOPS/core}.$$
$$P_{\text{CPU},2} = n_{\text{cores}} \times P_{\text{core},2} = 64 \times 39.2\,\text{GFLOPS/core} = 2.5088\,\text{TFLOPS/CPU}.$$
$$P_{\text{node},2} = n_{\text{sockets}} \times P_{\text{CPU},2} = 2 \times 2.5088\,\text{TFLOPS/node} = 5.0176\,\text{TFLOPS/node}.$$
$$P_{\text{cluster},2} = n_{\text{nodes}} \times P_{\text{node},2} = 248 \times 5.0176\,\text{TFLOPS/node} = 1244.3648\,\text{TFLOPS}.$$

2.2. Memory Hierarchies [8 Points]

2.2.1. Cache and main memory size

The solution to this exercise can be found by first running the Makefile in `2_Performance_Characteristics/Memory_Hierarchies` and then executing `memory_hierarchies_01.sh` for Euler VII (phase 1). The batch-script output is available in `2_Performance_Characteristics/Memory_Hierarchies/build_1`.
We report in table 2 the results for Euler VII (phase 1) and in figure 1 the memory topology.

Main memory (total)	249 GB
L3 cache (total)	512 MB (32 × 16 MB)
L3 cache (per socket)	256 MB (16 × 16 MB)
L2 cache (per core)	512 KB
L1 data cache (per core)	32 KB
L1 instruction cache (per core)	32 KB

Table 2: Memory hierarchy of an Euler compute node with AMD EPYC 7H12 (2 sockets, 64 cores per socket). Each core has private L1 and L2 caches, while L3 cache is organized in 32 blocks of 16 MB shared among groups of 4 cores.

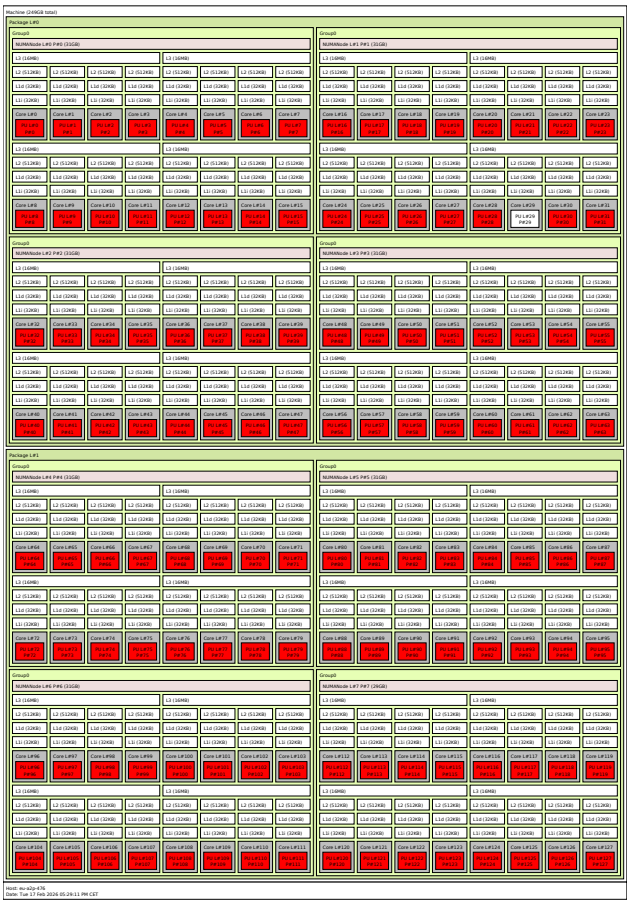


Figure 1: Topology of Euler VII (phase 1) compute node with AMD EPYC 7H12 (2 sockets, 64 cores per socket).

We proceed in the same way for Euler VII (phase 2) by executing `memory_hierarchies_02.sh` and we report in table 3 the results for Euler VII (phase 2) and in figure 2 the memory topology.

Main memory (total)	249 GB
L3 cache (total)	512 MB (16 × 32 MB)
L3 cache (per socket)	256 MB (8 × 32 MB)
L2 cache (per core)	512 KB
L1 data cache (per core)	32 KB
L1 instruction cache (per core)	32 KB

Table 3: Memory hierarchy of an Euler compute node with AMD EPYC 7763 (2 sockets, 64 cores per socket). Each core has private L1 and L2 caches, while the L3 cache is organized in 16 blocks of 32 MB shared among groups of cores.

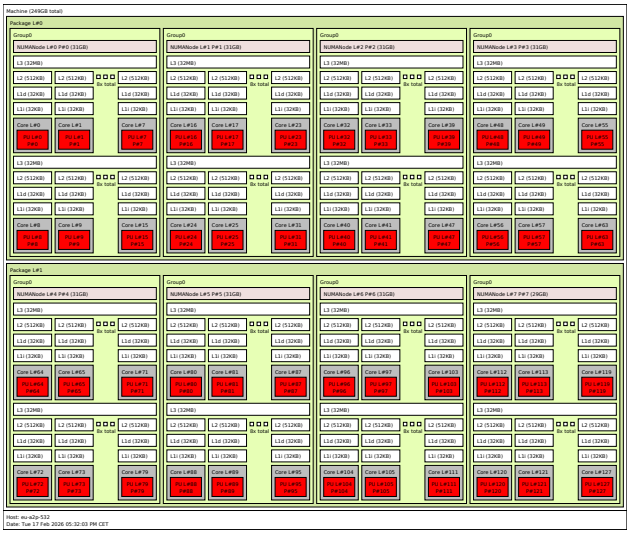


Figure 2: Topology of an Euler compute node with AMD EPYC 7763 (2 sockets, 64 cores per socket).

2.3. Bandwidth: STREAM benchmark [10 Points]

For this exercise one can run the Makefile in `2_Performance_Characteristics/STREAM_Benchmark` and then execute `stream_one.sh` for Euler VII (phase 1) and `stream_two.sh` for Euler VII (phase 2). The batch-script outputs are available in `2_Performance_Characteristics/STREAM_Benchmark/build_1` and `2_Performance_Characteristics/STREAM_Benchmark/build_2` respectively.

For Euler VII (phase 1), we run the file stream.c with the parameters specified in the following code snippet, which corresponds to the command used in the batch script `stream_one.sh`:

```
gcc -O3 -march=native -mcmodel=medium -DSTREAM_TYPE=double -DSTREAM_ARRAY_SIZE=268435456
    -DNTIMES=20 stream.c -o build_1/stream_one_c.exe
```

We used as array size the value of 2^{28}, which corresponds to 2 GB per array, and we set the number of iterations to 20. The corresponding output of the batch script is reported here for completeness:

```
  Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0




Model name:  AMD EPYC 7H12 64-Core Processor
eu-a2p-476
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
```

```
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
 Array size = 268435456 (elements), Offset = 0 (elements)
Memory per array = 2048.0 MiB (= 2.0 GiB).
Total memory required = 6144.0 MiB (= 6.0 GiB).
Each kernel will be executed 20 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Your clock granularity/precision appears to be 1 microseconds.
Each test below will take on the order of 151041 microseconds.
   (= 151041 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function Best Rate MB/s Avg time Min time Max time
Copy: 33537.1 0.133024 0.128066 0.146805
Scale: 18170.2 0.246928 0.236374 0.258205
Add: 20230.7 0.332126 0.318450 0.354032
Triad: 19668.1 0.341913 0.327558 0.364689
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

We choose `STREAM_ARRAY_SIZE` $= 2^{28}$ (268,435,456 doubles) so that each array occupies 2 GiB and the total working set (3 arrays $\approx$ 6 GiB) is well beyond the last-level cache. This reduces cache effects and makes the reported rates representative of sustained main-memory bandwidth. STREAM reports the *Best Rate* excluding the first iteration; for the roofline model we use the Triad kernel as a proxy for sustainable bandwidth, since it stresses both loads and stores and is commonly the most representative. The Copy kernel can appear faster because it benefits from efficient store handling (e.g., write combining), whereas Add/Triad are more load-intensive. Here, Triad achieves 19.7 GB/s (19,668 MB/s), which we use to define B_1 (after converting MB/s to GB/s).

For Euler VII (phase 2), we run the file stream.c with the parameters specified in the following code snippet, which corresponds to the command used in the batch script `stream_two.sh`:

```
gcc -O3 -march=native -mcmodel=medium -DSTREAM_TYPE=double -DSTREAM_ARRAY_SIZE=268435456
    -DNTIMES=20 stream.c -o build_2/stream_two_c.exe
```

The corresponding output of the batch script is reported here for completeness:

```
Currently Loaded Modules:
  1) stack/2025-06 2) gcc/12.2.0



Model name: AMD EPYC 7763 64-Core Processor
eu-a2p-509
-------------------------------------------------------------
STREAM version $Revision: 5.10 $
-------------------------------------------------------------
This system uses 8 bytes per array element.
-------------------------------------------------------------
 Array size = 268435456 (elements), Offset = 0 (elements)
Memory per array = 2048.0 MiB (= 2.0 GiB).
```

```
Total memory required =  6144.0 MiB (= 6.0 GiB) .
Each kernel will be executed  20 times.
 The *best* time for each kernel (excluding the first iteration)
 will be used to compute the reported bandwidth.
-------------------------------------------------------------
Your clock granularity/precision appears to be  1 microseconds.
Each test below will take on the order of  206516 microseconds.
   (= 206516 clock ticks)
Increase the size of the arrays if this shows that
you are not getting at least 20 clock ticks per test.
-------------------------------------------------------------
WARNING -- The above is only a rough guideline.
For best results, please be sure you know the
precision of your system timer.
-------------------------------------------------------------
Function Best Rate MB/s Avg time Min time Max time
Copy:  15183.6  0.291832 0.282868 0.298069
Scale:  13359.4  0.326391 0.321494 0.331896
Add:  12655.6  0.515276 0.509060 0.519799
Triad:  14198.6  0.468732 0.453738 0.498993
-------------------------------------------------------------
Solution Validates: avg error less than 1.000000e-13 on all three arrays
-------------------------------------------------------------
```

Note that `STREAM_ARRAY_SIZE`= 268435456 with `STREAM_TYPE=double` corresponds to $268435456 \cdot 8 = 2^{28} \cdot 8 = 2^{31}$ bytes, i.e. 2.0,GiB per array (as confirmed by STREAM: 2048.0,MiB). Therefore the total working set is $3 \times 2.0, \text{GiB} \approx 6.0, \text{GiB}$, which is well beyond the last-level cache capacity and strongly reduces cache effects. Hence, the reported bandwidth numbers are representative of sustained main-memory (DRAM) bandwidth. As for the roofline model, we use the `Triad` kernel as a proxy for sustainable bandwidth; in this run, `Triad` achieves 14198.6,MB/s ($\approx$ 14.20,GB/s), which we take as B_2.

2.4. Performance model: A simple roofline model [16 Points]

A (naive) roofline model provides an upper bound on the attainable performance of a kernel as a function of its operational intensity I (in Flops/Byte). For a given hardware, the attainable performance is bounded by:

- a **compute ceiling** given by the peak floating-point performance P_{peak} (in GFlops/s),
- a **bandwidth ceiling** given by $B \cdot I$, where B is the sustainable memory bandwidth (in GBytes/s).

Hence the roofline bound is

$$P(I) = \min\left(P_{\text{peak}},\ B \cdot I\right).$$

The transition between memory-bound and compute-bound regimes occurs at the **ridge point**:

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B}. \tag{1}$$

A kernel with $I < I_{\text{ridge}}$ is memory-bound, while a kernel with $I > I_{\text{ridge}}$ is compute-bound.

From the previous exercises, we have already computed the peak performance P_{peak} and the sustainable memory bandwidth B for both Euler VII phases. The values of the single core peak performances where

$$P_{\text{peak},1} = 41.6 \text{ GFlops/s} \quad \text{(Euler VII phase 1, AMD EPYC 7H12 @ 2.6 GHz)},$$
$$P_{\text{peak},2} = 39.2 \text{ GFlops/s} \quad \text{(Euler VII phase 2, AMD EPYC 7763 @ 2.45 GHz)}.$$

We use the STREAM `Triad` bandwidth as sustainable memory bandwidth, since it represents a typical streaming workload (three arrays, two loads + one store per iteration) and is commonly used in roofline constructions. From our STREAM outputs, we have the following sustainable memory bandwidths:

$$B_1 = 19668.1 \text{ MB/s} = 19.6681 \text{ GB/s},$$
$$B_2 = 14198.6 \text{ MB/s} = 14.2 \text{ GB/s},$$

where we convert MB/s to GB/s by dividing by 1000.
Using Eq.1:

$$I_{\text{ridge},1} = \frac{P_{\text{peak},1}}{B_1} = \frac{41.6}{19.6681} \approx 2.12 \text{ Flops/Byte},$$
$$I_{\text{ridge},2} = \frac{P_{\text{peak},2}}{B_2} = \frac{39.2}{14.2} \approx 2.76 \text{ Flops/Byte}.$$

Quantity	**Euler VII (Phase 1)**	**Euler VII (Phase 2)**
P_{peak} [GFlops/s]	41.6	39.2
B (STREAM Triad) [GB/s]	19.6681	14.20
$I_{\text{ridge}} = P_{\text{peak}}/B$ [Flops/Byte]	2.12	2.76

Table 4: Inputs and ridge point of the single-core roofline models.

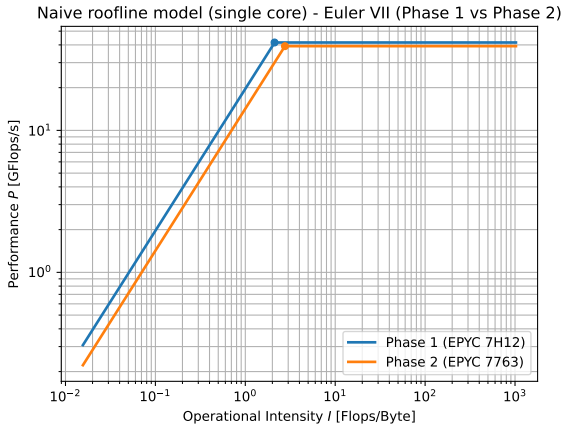


Figure 3: Naive roofline model for a single core on Euler VII (both phases). The ridge points are at $I_{\text{ridge},1} \approx 2.12$ Flops/Byte and $I_{\text{ridge},2} \approx 2.76$ Flops/Byte for phase 1 and phase 2 respectively.

A kernel is memory-bound for $I < I_{\text{ridge}}$ and compute-bound for $I > I_{\text{ridge}}$; thus the transition occurs at 2.12 Flop/Byte (phase 1) and 2.76 Flop/Byte (phase 2).

2.5. Use of LLMs

For this exercise, we used CHAT GPT 5.2 to help us understand and interpret properly the infos available on the website uops.info. In particular, it was used to determine the superscalarity factor for the AMD Zen2 and AMD Zen3 architectures.

3. Auto-vectorization [10 points]

Question 1: what is data alignment and is it important for data structures to be aligned?
Data alignment refers to placing an object in memory at an address that is a multiple of some power-of-two boundary (for example, 16 bytes). In the document (Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference), alignment is discussed as a key performance property for SIMD loads/stores: Intel SSE (and similarly other SIMD ISAs) prefers aligned memory references, and data movement instructions can be significantly more efficient when the data is aligned. The text also warns that using aligned data-movement instructions with incorrectly aligned addresses can even raise an exception (so alignment assertions must be correct).
Because of this, it is generally important that frequently accessed arrays/structures used in vectorized loops are aligned (e.g., 16-byte boundaries for SSE), especially when you want the compiler to generate efficient aligned vector loads/stores and avoid extra fix-up code.

Question 2: what are some obstacles that can prevent automatic vectorization by the compiler?
Several recurring patterns can make vectorization unsafe or not worthwhile, so the compiler may decide to keep scalar code. A first frequent issue is **non-contiguous memory access**: when a loop uses non-unit strides or indirect addressing (for instance, indices coming from another array), the hardware cannot easily use a single packed load, and the compiler anticipates extra gathers and inefficient memory traffic.
Another common blocker is the presence of **loop-carried dependencies**. Since SIMD executes multiple iterations "together", it effectively changes the per-iteration execution order; if the compiler cannot prove that different iterations are independent, it must conservatively assume a dependence and will often avoid vectorization.
Vectorization is also discouraged when the loop is not in a **canonical vector-friendly form**. Compilers prefer countable loops whose trip count is known at entry, with a single entry and a single exit, and with straight-line bodies; complex or unsupported control flow (e.g. `switch` statements, early exits, or other non-structured patterns) makes it harder to build a correct vector loop. Moreover, **function calls inside the loop body** frequently stop vectorization: even something as simple as printing introduces side effects and prevents reordering, unless the call is a recognized intrinsic, can be proven safe, or can be inlined.
Finally, **pointer aliasing** and complicated pointer expressions can force the compiler into worst-case assumptions: if two pointers might refer to overlapping memory, then loads and stores may depend on each other across iterations, and the compiler may refuse vectorization or add costly runtime checks.

Question 3: is there a way to help the compiler to vectorize and how?
Yes. One can increase the chances of vectorization by writing vector-friendly code and by supplying additional information that the compiler cannot safely infer. A direct way is to use **pragmas** that communicate dependence and vectorization intent.
Finally, **data layout choices** matter: adopting a structure-of-arrays (SoA) layout often turns accesses into unit-stride streams, which are typically much easier and more profitable to vectorize than the scattered accesses typical of an array-of-structures (AoS) layout.

Question 4: which loop optimizations are performed by the compiler to vectorize and pipeline loops?
Successful vectorization is often enabled only after stronger optimization settings activate additional loop transformations that reshape the loop into a more regular form. In this context, the compiler commonly applies **loop unrolling** together with packed SIMD instruction generation: vectorization produces SIMD instructions that cover multiple scalar iterations, which is conceptually similar to unrolling, and manual unrolling is generally unnecessary if its only purpose is to "force" vectorization.

Question 5: what can be done if automatic vectorization by the compiler fails or is suboptimal?
A robust approach is to start from diagnosis, then remove the cause through code restructuring, and only afterwards apply stronger directives or explicit SIMD methods. One can often **rewrite the loop into a more vectorizable shape**: remove or move function calls out of the loop, eliminate loop-carried dependencies, and prefer unit-stride array accesses over indirect indexing or complex pointer arithmetic.
When correctness allows it, **adding accurate annotations** can resolve conservative assumptions: `restrict` can remove aliasing ambiguity, alignment declarations or assumptions can reduce misalignment penalties, and pragmas such as `#pragma ivdep` or `#pragma vector always` can steer the compiler past overly cautious dependence or profitability decisions. These mechanisms must be used carefully, since forcing aligned loads

on misaligned addresses or providing incorrect dependence information can change semantics or even lead to crashes. If hints remain insufficient, **user-mandated SIMD directives** such as `#pragma omp simd` can be used to impose vectorization semantics and typically trigger warnings if the compiler cannot generate a vector loop, while also allowing clauses to specify the intended SIMD behavior.

3.1. Use of LLMs

For this exercise, we used CHAT GPT 5.2 to help us write in a more clear and english-correct way the answers to the questions.

4. Matrix multiplication optimization [30 points]

Task 1: Improve the loop ordering in `dgemm_jki.c`.
The original implementation uses the loop order i–j–k:

```
void square_dgemm(int n, double* A, double* B, double* C) {
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      for(int k = 0; k < n; ++k) {
        C[i+j*n] += A[i+k*n] * B[k+j*n];
      }
    }
  }
}
```

This loop ordering is correct, but it performs poorly for column-major storage (indexing $i + jn$). In the innermost loop over k, the access to $A[i + kn]$ moves across different columns of A with a stride of n, i.e., it repeatedly jumps to memory locations that are far apart. This limits spatial locality and makes it difficult for the cache hierarchy and hardware prefetchers to be effective. By contrast, $B[k+jn]$ is read contiguously in the k-loop (a whole column of B), so the main bottleneck of the naive ordering is the strided access pattern to A.

To improve locality, we reorder the loops to j–k–i and hoist invariant quantities out of the innermost loop:

```
void square_dgemm(int n, double* A, double* B, double* C) {
  for (int j = 0; j < n; ++j) {
    double* Ccol = C + j*n; // C(:,j)
    for (int k = 0; k < n; ++k) {
      const double bkj = B[k + j*n]; // B(k,j) reused for all i
      const double* Acol = A + k*n; // A(:,k)
      for (int i = 0; i < n; ++i) {
        Ccol[i] += Acol[i] * bkj; // stride-one on C and A
      }
    }
  }
}
```

With column-major layout, full columns are contiguous in memory. For fixed (j, k), the innermost loop over i streams through $A(:, k)$ and updates $C(:, j)$ with *stride 1* (contiguous memory) in both arrays. This improves cache utilization and makes prefetching more effective, since consecutive iterations access consecutive memory locations. Moreover, loading $B(k, j)$ once into a scalar (`bkj`) and using column pointers (`Acol`, `Ccol`) reduces redundant address computations and avoids repeatedly reloading the same element of B. The computation is unchanged, since we still perform

$$C_{i,j} \mathrel{+}= \sum_{k=0}^{n-1} A_{i,k} B_{k,j},$$

but with a memory-access pattern that is more favorable for the cache hierarchy and that typically facilitates SIMD vectorization in the innermost loop.

Task 2: Implement blocking for square matrix multiplication in `dgemm_blocked.c`.
To further increase performance, we implement *blocking* (tiling) in all three loop dimensions. Instead of working on the full iteration space at once, we partition the indices into blocks of size BS along i, j, and k, and compute the product block-by-block:

```
void square_dgemm(int n, double* A, double* B, double* C)
{
  for (int jj = 0; jj < n; jj += BS) {
    int jmax = imin(jj + BS, n);

    for (int kk = 0; kk < n; kk += BS) {
      int kmax = imin(kk + BS, n);
```

```
      for (int ii = 0; ii < n; ii += BS) {
        int imax = imin(ii + BS, n);

        for (int j = jj; j < jmax; ++j) {
          for (int k = kk; k < kmax; ++k) {
            double bkj = B[k + j * n];
            for (int i = ii; i < imax; ++i) {
              C[i + j * n] += A[i + k * n] * bkj;
            }
          }
        }
      }
    }
  }
}
```

The outer loops `jj`, `kk`, and `ii` select the current block ranges

$$j \in [jj, j_{\max}), \qquad k \in [kk, k_{\max}), \qquad i \in [ii, i_{\max}),$$

and the helper `imin` ensures correct handling of boundary tiles when n is not a multiple of BS. Within each block triple, the inner kernel performs the same j–k–i update as in Task 1, but restricted to the current submatrices.

Blocking improves performance by increasing *temporal locality*. For a fixed (jj, kk, ii) triple, the algorithm repeatedly reuses the same sub-blocks of A and B while accumulating into the corresponding sub-block of C, reducing the frequency at which data must be fetched from main memory. Inside each block, the innermost loop still runs over i, so the access to $A(:, k)$ and $C(:, j)$ remains stride-1 in column-major format. Finally, loading `bkj = B(k,j)` once per (j, k) pair provides scalar reuse and avoids redundant loads in the innermost loop.

We set the default block size to $BS = 64$, a common baseline choice for a first cache-blocked DGEMM implementation. With double-precision elements, a 64×64 tile contains 64^2 entries and occupies $64^2 \cdot 8 \approx 32\,\text{KiB}$. The combined footprint of the three tiles (A, B, C) involved in the update is therefore about $3 \cdot 32 \approx 96\,\text{KiB}$. This working set does not fit in L1 data cache, but it fits comfortably in the per-core L2 cache, allowing substantial reuse from L2 rather than repeatedly streaming the same data from main memory. The block size is kept configurable via `#ifndef BS` so that alternative values can be tested empirically on the target architecture if further tuning is desired.

In figure 4, we show the performance of the naive, reordered (stride-one), blocked and BLAS implementations of DGEMM on a single core of an AMD EPYC 7763 processor. The naive i–j–k implementation stays around a few GFLOP/s and shows large variability (occasional deep drops), consistent with poor cache behavior due to strided accesses in column-major layout. Reordering the loops to achieve stride-one access improves locality substantially, reaching $\sim$10–15 GFLOP/s for moderate sizes; for larger n its performance gradually decreases as the working set no longer fits in the upper cache levels and execution becomes increasingly limited by memory traffic and reduced reuse. Cache blocking stabilizes performance around $\sim$10 GFLOP/s over the full range by increasing temporal locality and reusing sub-tiles of A and B while updating a tile of C. The reference BLAS implementation remains the fastest (tens of GFLOP/s), highlighting the impact of architecture-tuned optimizations such as packing, register blocking, vectorized micro-kernels, and prefetching that go beyond simple loop transformations.

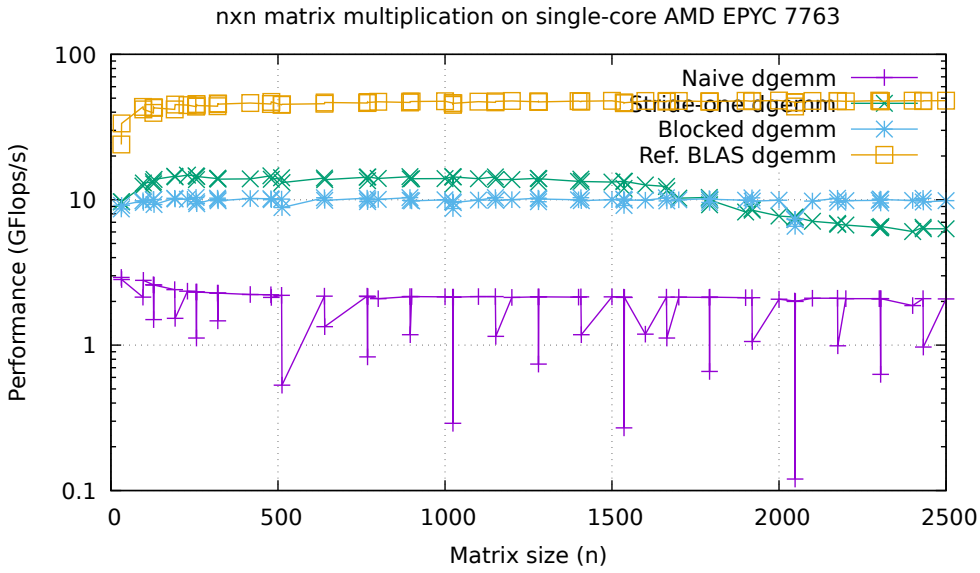


Figure 4: Performance results for the naive, reordered (stride-one), blocked and BLAS implementations of DGEMM (single core)

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Report for Project 2

Due date: 23 March 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with OpenMP [20 points]

1.1. Exercise 1

In this exercise we are given a code that computes an approximation of π using numerical integration. The code uses a simple loop to compute the sum of terms that approximate the value of π. The goal of this exercise is to parallelize the code using OpenMP and to optimize it for better performance.

In the first part of the exercise it was asked to first parallelize the given code using a naive version that protects every loop update with `critical`. In listing 1 we show the code snippet of the naive version that we implemented.

Listing 1: Naive parallel version of the π computation using OpenMP

```
#pragma omp parallel for
for (uint64_t i = 0; i < N; ++i) {
  double x = (i + 0.5) * h;
  double term = 4.0 / (1.0 + x*x);

  #pragma omp critical
  pi += term;
}
pi *= h;
```

The naive version is very slow. This is because every update to the shared variable `pi` is protected by a critical section, which causes a lot of contention and serialization among the threads. Each thread has to wait for the others to finish updating `pi`, which leads to a significant slowdown in performance.

In the second part it was asked to create an optimized version that accumulates locally and enters critical only once per thread. In listing 2 we show the optimized version of the π computation using OpenMP:

Listing 2: Optimized parallel version of the π computation using OpenMP

```
#pragma omp parallel
{
  double sum = 0.0;
  #pragma omp for
  for (uint64_t i = 0; i < N; ++i) {
    double x = (i + 0.5) * h;
    sum += 4.0 / (1.0 + x*x);
  }
  #pragma omp critical
  pi += sum; // once per thread
}
pi *= h;
```

In this version we have a local variable `sum` that is private to each thread, and we only enter the critical section once per thread to update the shared variable `pi`. This reduces the contention on the critical section and allows for better performance.

We are then asked to implement a version that uses the reduction clause. In listing 3 we show the code snippet of the version that uses the reduction clause:

Listing 3: Reduction parallel version of the π computation using OpenMP

```
#pragma omp parallel for reduction(+:sum)
for (uint64_t i = 0; i < N; ++i) {
  double x = (i + 0.5)*h;
  sum += 4.0 / (1.0 + x*x);
}
pi = sum*h;
```

In this version we use the `reduction` clause to automatically handle the accumulation of the variable `pi` across threads. The reduction clause takes care of creating a private copy of the variable for each thread and then combining the results at the end of the parallel region. This allows for a more concise and efficient implementation.
In table 1 we summarize the results of the different versions of the code.

Version	π	num_threads	time [secs]
Serial	3.141592653589971	1	1.88825762
Naive	3.141592653590757	64	640.706
Optimized	3.141592653589771	64	0.19858712
Reduction	3.141592653589771	64	0.11437800

Table 1: Results of the different versions of the code for computing π

1.2. Exercise 2

In this part of the exercise we perform a **strong** and **weak scaling study** of the three implementations of the π computation.

1.2.1. Strong scaling study

In a **strong scaling study**, the total problem size is kept constant while the number of threads p is increased. In our case, the number of integration steps is fixed to $N = 10^9$.
In [1, p. 123] it is shown that the parallel performance is defined as 1

$$P(N) = \frac{W}{T(N)} \tag{1}$$

where W is some fixed amount of work (in our case, the number of iterations N) and $T(N)$ is the execution time for that amount of work. In [1, p. 124] they define the speedup as the ratio of the execution time of the serial version to the execution time of the parallel version:

$$S_p(p) = \frac{P(p)}{P(1)} = \frac{T(1)}{T(p)} \tag{2}$$

For each number of threads we measure the execution time T_p and from these measurements we compute the **speedup**, defined as in equation 2. In [1, p. 124] they state that the ideal speedup is $P_{ideal} = p$ (where p is the number of threads). In practice, we expect to see a speedup that is less than ideal due to overheads such as thread management and synchronization.
The results of the strong scaling experiment are reported in Table 2.

Table 2: Strong scaling results for the different implementations ($N = 10^9$).

Threads	Naive Critical		Optimized Critical		Reduction	
	Time [s]	Speedup	Time [s]	Speedup	Time [s]	Speedup
1	6.4987	1.00	1.4189	1.00	1.3933	1.00
2	61.7839	0.11	0.7696	1.84	0.7573	1.84
4	105.7615	0.06	0.4260	3.33	0.4268	3.26
8	131.8628	0.05	0.2179	6.51	0.2179	6.39
16	391.7679	0.02	0.1575	9.01	0.1574	8.86
32	427.1595	0.02	0.0789	17.98	0.0789	17.66
64	600.1213	0.01	0.0449	31.60	0.0448	31.06

From the results we observe a very different behavior among the three implementations.
The **naive implementation with a critical section** performs extremely poorly. As the number of threads increases, the execution time increases dramatically instead of decreasing. This is due to the fact that every iteration of the loop enters a `critical` section to update the shared variable. As a consequence, the threads serialize their execution and spend most of the time waiting to acquire the lock. This creates severe contention and eliminates the benefits of parallelization.

The **optimized critical version** performs significantly better. In this implementation each thread accumulates a local partial sum and only enters the `critical` section once at the end of its work. This drastically

reduces the synchronization overhead. As a result, the speedup increases almost linearly up to several threads.

The **reduction implementation** shows a very similar behavior to the optimized version. This is expected since OpenMP reductions internally implement an efficient private accumulation followed by a final combination step. The reduction version reaches a speedup of about 31 with 64 threads, showing good parallel scalability.

1.2.2. Weak scaling study

In a **weak scaling study**, the amount of work per thread is kept constant while the total problem size grows proportionally with the number of threads. If p is the number of threads, the total problem size becomes $N = N_0 \times p$ where N_0 is the number of iterations assigned to each thread. In our experiments we use $N_0 = 5 \times 10^7$.
The main metric used to evaluate weak scaling is the **parallel efficiency**. In [1, p. 125] it is defined as in eq. 3.

$$E_p = \frac{\text{speedup}}{p} = \frac{T(1)}{T(p)} \cdot \frac{1}{p} = \frac{T_1}{T_p} \tag{3}$$

where T_p is the execution time with p threads and problem size $N = N_0 \cdot p$. Ideally, the execution time should remain constant as the number of threads increases, resulting in $E_p \approx 1$. The results of the weak scaling experiment are shown in Table 3.

Table 3: Weak scaling results for the different implementations (work per thread = 5×10^7).

Threads	N	Naive Critical		Optimized Critical		Reduction	
		Time [s]	Efficiency	Time [s]	Efficiency	Time [s]	Efficiency
1	5×10^7	0.4024	1.00	0.1260	1.00	0.0758	1.00
2	1×10^8	5.4202	0.07	0.1203	1.05	0.0956	0.79
4	2×10^8	23.1030	0.02	0.1166	1.08	0.0924	0.82
8	4×10^8	88.4441	0.00	0.0943	1.34	0.0909	0.83
16	8×10^8	225.6566	0.00	0.1261	1.00	0.1262	0.60
32	1.6×10^9	616.5533	0.00	0.1265	1.00	0.1275	0.59
64	3.2×10^9	2015.6843	0.00	0.1325	0.95	0.1265	0.60

The **naive implementation** scales extremely poorly. The execution time increases dramatically with the number of threads due to the heavy contention on the `critical` section. Since every iteration requires synchronization, the amount of waiting grows rapidly as more threads are added.

The **optimized critical version** shows very good weak scaling behavior. The execution time remains approximately constant as the number of threads increases, with efficiency values close to one. This indicates that the implementation can handle larger problem sizes effectively while maintaining good parallel performance.

The **reduction implementation** also exhibits good weak scaling, although the efficiency slightly decreases for larger thread counts. This behavior is expected due to additional overheads such as thread management and reduction synchronization.

The data reported are obtained by taking the average of 5 different runs. In this way we can mitigate the effects of variability in execution time and obtain more reliable measurements. By looking at the outputs in the `.out` files one can also see the minimum and maximum times recorded across the runs, which gives an indication of the variability in performance.

1.3. Use of LLMs

Chat gpt was used to create the tables in Latex from the data collected from the experiments and to confirm the correcteness of the equations used for speedrun and efficiency calculations.
It was also used to properly modify the .c files to modify correctly the original codes in order to perform a strong scaling and a weak scaling study.

2. The Mandelbrot set using OpenMP [20 points]

2.1. Exercise 1 and 2

Here we are asked to complete the serial version of the Mandelbrot set code in the missing code part and to count the total number of iterations in order to correctly compute the benchmark statistics. In figure 1 one can see the results of the serial implementation.
In listing 4 we show the code snippet of the serial version of the Mandelbrot set code with the missing part filled in and with the counting of the total number of iterations.

Listing 4: Serial version of the Mandelbrot set code

```
double time_start = walltime();
  // do the calculation
  cy = MIN_Y;
  for (j = 0; j < IMAGE_HEIGHT; j++) {
    cx = MIN_X;
    for (i = 0; i < IMAGE_WIDTH; i++) {
      x = cx;
      y = cy;
      x2 = x * x;
      y2 = y * y;
      int n = 0;

      while (x2 + y2 <= 4.0 \&\& n < MAX\_ITERS) {
        y = 2.0 * x * y + cy;
        x = x2 - y2 + cx;
        x2 = x * x;
        y2 = y * y;
        n++;
      }
       nTotalIterationsCount += n;
      // plot the number of iterations at point (i, j)
      int c = ((long)n * 255) / MAX_ITERS;
      ppm_plot(&pPpm, i, j, c, c, c);
      cx += fDeltaX;
  }
    cy += fDeltaY;
  }
  double time_end = walltime();
```

In the while loop, we are iterating through the orbit of the point c under the function $f(z) = z^2 + c$. The variables x and y represent the real and imaginary parts of the complex number z, respectively. The function $f(z)$ can be expressed in terms of its real and imaginary components as follows:

$$\begin{aligned} f(z) &= z^2 + c \\ &= (x + yi)^2 + (c_x + c_y i) \\ &= (x^2 - y^2 + c_x) + (2xy + c_y)i \end{aligned}$$

From this, we can see that the real part of $f(z)$ is given by $x^2 - y^2 + c_x$, and the imaginary part is given by $2xy + c_y$. The loop continues until the magnitude of z (which is $\sqrt{x^2 + y^2}$) exceeds 2, indicating that the orbit is unbounded, or until we reach the maximum number of iterations, which indicates that the point c is likely to belong to the Mandelbrot set.
Finally, we are asked to count the total number of iterations in order to correctly compute the benchmark statistics. This is done by introducing a variable `nTotalIterationsCount` that accumulates the number of iterations for each point in the complex plane. This variable is incremented by n, which represents the number of iterations for the current point (i, j).

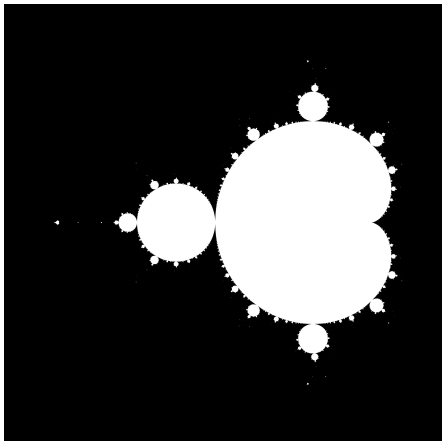


Figure 1: Serial Mandelbrot Set

2.2. Exercise 3

In the third part of the exercise we are asked to parallelize the Mandelbrot set code using `OpenMP`. In listing 5 we show the code snippet of the parallel version of the Mandelbrot set code using OpenMP.

Listing 5: Parallel version of the Mandelbrot set code using OpenMP

```
double time_start = walltime();
#pragma omp parallel for reduction(+:nTotalIterationsCount) schedule(dynamic)
for (long j = 0; j < IMAGE_HEIGHT; j++) {
    double cy = MIN_Y + j * fDeltaY;

  for (long i = 0; i < IMAGE_WIDTH; i++) {
      double cx = MIN_X + i * fDeltaX;

    double x = cx;
    double y = cy;
    double x2 = x * x;
    double y2 = y * y;

    int n = 0;
    while (x2 + y2 <= 4.0 && n < MAX_ITERS) {
      y = 2.0 * x * y + cy;
      x = x2 - y2 + cx;
      x2 = x * x;
      y2 = y * y;
      n++;
    }

    nTotalIterationsCount += n;

    int c = ((long)n * 255) / MAX_ITERS;
    ppm_plot(&pPpm, i, j, c, c, c);
  }
}
double time_end = walltime();
```

To parallelize we used the `parallel for` directive to distribute the iterations of the outer loop (over j) among the threads. We also used a reduction clause to safely accumulate the total number of iterations across all threads. The schedule is set to dynamic to allow for better load balancing, as some points in the complex plane may require more iterations than others.
We have also chanced the way with initialize the variables `cx` and `cy` to make it work in the parallel version. We compute their values inside the loops based on the current indices i and j. This allows each thread to compute the correct values of `cx` and `cy` for its assigned iterations without any race conditions.

Metric	Serial Version	Parallel Version
Total time [seconds]	330.726	6.70717
Total number of iterations	113610974266	113638786651
Average time per pixel [seconds]	1.97128e-05	3.99779e-07
Average time per iteration [seconds]	2.91104e-09	5.90219e-11
Iterations per second	3.4352e+08	1.69429e+10
MFlop/s	2748.16	135543

Table 4: Comparison of the serial and parallel versions of the Mandelbrot set code

The results (see table 4) show a significant performance improvement achieved by parallelizing the code using `OpenMP`. The total time taken to compute the Mandelbrot set is reduced from 330.726 seconds in the serial version to 6.70717 seconds in the parallel version, which is a substantial decrease in execution time. The total number of iterations is slightly higher in the parallel version, which could be due to differences in the order of execution and how the iterations are distributed among the threads. The average time per pixel and average time per iteration are also significantly reduced in the parallel version, indicating that the computation is much faster when utilizing multiple threads. The iterations per second and MFlop/s metrics show a dramatic increase in performance, with the parallel version achieving 1.69429e+10 iterations per second and 135543 MFlop/s compared to 3.4352e+08 iterations per second and 2748.16 MFlop/s in the serial version.

2.2.1. Strong scaling analysis

To analyze the strong scaling of the parallelized Mandelbrot set code, we can measure the execution time for different numbers of threads while keeping the problem size fixed. This allows us to see how well the code scales with increasing thread count.
We also calculate the speedup as the ratio of the execution time of the serial version to the execution time of the parallel version for each thread count. In figure 2 we show the plots of the strong scalying analysis. As we can see the execution times halves every time we double the number of threads, which indicates that the code scales well with increasing thread count. In figure 3 instead we plot the theoretical speedup with the one computed from the simulations. The actual speedup is computed as shown in equation 2.

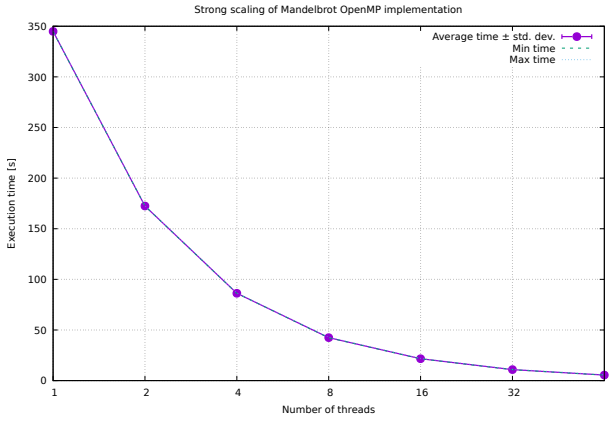


Figure 2: Strong scaling analysis of the parallelized Mandelbrot set code

Figure 2 shows the execution time of the Mandelbrot OpenMP implementation as a function of the number of threads. As expected in a strong scaling study, the problem size is fixed while the number of threads increases. The execution time always halves when moving from 1 to 64 threads, demonstrating that the parallelization is effective in exploiting multiple cores. However, beyond 16 threads the improvement becomes less pronounced and the curve starts to flatten.

The error bars representing the standard deviation are relatively small across all thread counts, indicating that the measurements are stable and reproducible. Indeed by looking at the .data file one can see that the minimum and maximum execution times remain close to the average values.
Figure 3 reports the corresponding speedup with respect to the single-thread execution. The speedup stays consistently close to the the ideal speedup curve as the number of threads increases.

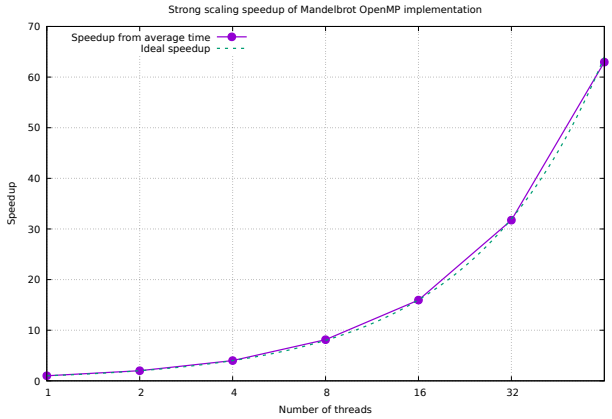


Figure 3: Speedup of the parallelized Mandelbrot set code compared to the ideal speedup

Overall, the results show that the OpenMP parallelization of the Mandelbrot computation scales well. As we did for the first exercise, the data shown in figures 2 and 3 are obtained by taking the average of 5 different runs.

2.3. Use of LLMs

Chat GPT was used to create the gnuplot files and the bash file to generate the strong scaling plots.

3. Bug hunt [10 points]

3.1. File 1: `omp_bug1.c`

The program attempts to parallelize the loop that computes $c[i] = a[i] + b[i]$ using the OpenMP `parallel for` construct.
The first problem is related to the use of the OpenMP directive `#pragma omp parallel for`. This combined directive creates a parallel region and distributes the iterations of the following loop among the threads. For this reason, the directive must be followed *immediately* by a `for` loop. In the program, however, the directive is followed by a compound block (see listing 6).

Listing 6: Incorrect use of the OpenMP `parallel for` directive

```
#pragma omp parallel for shared(a, b, c, chunk) private(i, tid) \
          schedule(static, chunk)
{
  tid = omp_get_thread_num();
  for (i = 0; i < N; i++)
```

This structure is invalid because the `parallel for` directive must be attached directly to the loop it parallelizes.

The second issue concerns the variable `tid`. The programmer wants to print which thread executes each iteration, but the statement `tid = omp_get_thread_num()` is placed outside the loop that the directive is supposed to parallelize. Because the directive requires the loop to appear immediately after it, this statement breaks the required structure of the construct.

To fix the problem, the loop must appear directly after the OpenMP directive. The thread identifier should instead be retrieved inside the loop body (or inside a parallel region if the `parallel` and `for` constructs are separated). In this way, each thread can correctly obtain its ID and print the information for the iterations it executes.

3.2. File 2: `omp_bug2.c`

The program contains errors related to the incorrect handling of `shared` and `private` variables inside the OpenMP parallel region.

First, the variable `tid` is shared, although each thread assigns its own thread ID through `tid = omp_get_thread_num()`. Since all threads write to the same memory location, the value of `tid` can be overwritten by other threads. As a result, the value stored in `tid` is not guaranteed to correspond to the correct thread.

A more serious problem concerns the variable `total`. Inside the parallel region the code initializes `total = 0.0;` and then performs a parallel loop (see listing 7).

Listing 7: Incorrect handling of shared and private variables in an OpenMP parallel region

```
#pragma omp for schedule(dynamic, 10)
for (i = 0; i < 1000000; i++)
    total = total + i * 1.0;
```

Because `total` is shared, all threads update the same variable concurrently. This produces a race condition and the resulting sum is incorrect and non-deterministic. Moreover, each thread resets the shared variable to zero before the loop, which introduces additional data races.
The variable `tid` should be declared *private* so that each thread keeps its own thread identifier. The summation should be implemented using an OpenMP `reduction` on `total`, which safely combines the partial sums computed by different threads.

3.3. File 3: `omp_bug3.c`

The main problem in this program is the incorrect use of the `barrier` directive inside the function `print_results`. In OpenMP, a barrier must be encountered by *all* threads of the team, or by none of them. Here, however, `print_results` is called only by the threads that actually execute one of the two `sections`. Since there are only two sections, at most two threads will enter the function, while the other threads in the parallel region will skip it entirely. As a consequence, only a subset of threads reaches the barrier placed at the end of `print_results`, which leads to an illegal synchronization pattern and can cause the program to hang at run time.

A second issue is that the intent of having "clean output" is only partially achieved. The `critical` region correctly prevents the printed array contents from being interleaved, so the output of one thread is kept together. However, the statement `printf("Thread %d done and synchronized.", tid);` is placed *after* the barrier and outside the critical region, so even if the barrier were legal, these messages could still appear in overlapping or non-deterministic order with respect to other threads' output.

There is also a conceptual inconsistency in combining `sections nowait` with the later synchronization logic. The `nowait` clause removes the implicit barrier at the end of the `sections` construct, but then the code tries to enforce synchronization manually inside `print_results` using a barrier, which is not valid for the reason explained above. If synchronization after the sections is desired, it should be done at a point that is reached by every thread in the parallel region.

To fix the code, the barrier inside `print_results` should be removed. Any barrier should instead be placed in the main parallel region, at a location where all threads are guaranteed to encounter it. Moreover, if the programmer wants all output messages to appear cleanly and without overlap, every printing operation that must remain grouped should be enclosed in the same `critical` region, or alternatively all printing should be reorganized so that only one thread prints at a time in a controlled way. Finally, if synchronization at the end of the `sections` construct is required, the simplest correction is to remove the `nowait` clause and rely on the implicit barrier provided by `sections`.

3.4. File 4: `omp_bug4.c`

The problem is caused by *stack overflow* in the OpenMP region. The key issue is that the array `real(8) :: e_in(nmax)` is declared inside the main program and the parallel region uses `!$omp parallel default(firstprivate)`. Because of `firstprivate`, each OpenMP thread receives its own private copy of `e_in`. Since $n_{\max} = 202000$, this array alone occupies about $202000 \times 8 \approx 1.6$ MB per thread. These private copies are typically placed on the thread stack. If the available stack size is too small, entering the parallel region causes a segmentation fault.

There is a second large object involved: `double complex :: eTDSE(ntmax)` inside the module, declared as `!$omp threadprivate(eTDSE)`.
This means that each thread also has its own copy of `eTDSE`. A `double complex` element usually requires 16 bytes, so this array takes about $202000 \times 16 \approx 3.2$ MB per thread. Therefore, with OpenMP enabled, the memory required per thread becomes quite large. The crash appears only when both arrays are effectively used in the parallel region, which is why removing one of the `print` statements may make the program run.

A first possible fix is to avoid making `e_in` firstprivate if this is not needed. Since the array is only read in the loop, it should be shared instead of private. In practice, one should replace the overly broad `default(firstprivate)` with a more careful data-sharing specification, for example making `e_in` shared. Another possible improvement is to avoid storing such large arrays on the stack.

3.5. File 5: `omp_bug5.c`

The main bug in this program is a **deadlock caused by inconsistent lock ordering**. The two `sections` acquire the two locks in opposite order. In the first section the thread acquires `locka` and then `lockb`, while in the second section the order is reversed: the thread first acquires `lockb` and then `locka`.

This can lead to a deadlock situation. For example, Thread 0 may acquire `locka` while Thread 1 acquires `lockb`. Afterwards, Thread 0 waits for `lockb`, which is already held by Thread 1, and Thread 1 waits for `locka`, which is held by Thread 0. Since each thread is waiting for a lock owned by the other one, neither thread can proceed and the program blocks indefinitely.

There is also a second issue related to the loop variable `i`. The variable `i` is declared outside the parallel region and is therefore **shared** by default. Since both sections execute concurrently and use the same loop variable, this creates a **data race**. Each thread should instead operate on its own private copy of the loop index.

To fix the program, the locking order must be made consistent. All threads should acquire multiple locks in the same global order (for instance always acquiring `locka` before `lockb`). This removes the circular wait condition and therefore prevents the deadlock. Another possible improvement is to redesign the algorithm so that nested locking is not required; for example, the arrays could be initialized first, followed by a synchronization point, and only afterwards the addition operations could be performed. Finally, the loop variable `i` should be made **private** inside the parallel region or declared locally inside each section so that each thread uses its own independent copy.

4. Parallel histogram calculation using `OpenMP` [15 points]

We place the code snippet of the parallelized version of the histogram calculation using `OpenMP` in listing 8. The code is parallelized by using a private histogram for each thread and then combining the results at the end of the parallel region.

Listing 8: Parallelized version of the histogram calculation using OpenMP

```
#define VEC_SIZE 1000000000
#define BINS 16
...
  time_start = walltime();

  int nthreads = omp_get_max_threads();
  std::vector<std::vector<long>> local_hist(nthreads, std::vector<long>(BINS, 0));

  #pragma omp parallel
  {
    int tid = omp_get_thread_num();

    #pragma omp for
    for (long i = 0; i < VEC_SIZE; ++i) {
      local_hist[tid][vec[i]]++;
    }
  }

  // Combine the histograms
  for (int t = 0; t < nthreads; ++t) {
    for (int b = 0; b < BINS; ++b) {
      dist[b] += local_hist[t][b];
    }
  }

  time_end = walltime();
```

To parallelize the histogram computation, we create a local histogram for each thread using a 2D vector `local_hist`, where each row corresponds to a thread and each column corresponds to a bin. We use an OpenMP parallel region to allow multiple threads to execute concurrently. Inside the parallel region, we retrieve the thread ID using `omp_get_thread_num()` and then use a parallel for loop to iterate over the input vector. Each thread updates its own local histogram based on the values in the input vector. After the parallel region, we combine the local histograms into the final histogram `dist` by summing the counts from each thread for each bin. Finally, we record the end time to measure the execution time of the histogram computation. This approach avoids contention on shared histogram bins by allowing each thread to maintain its own local counts, which are then combined at the end, resulting in improved performance and scalability.

4.1. Strong scaling analysis

To analyze the strong scaling of the parallelized histogram calculation, we can measure the execution time for different numbers of threads while keeping the problem size fixed. This allows us to see how well the code scales with increasing thread count. We can also calculate the speedup as the ratio of the execution time of the serial version to the execution time of the parallel version for each thread count. In figure 4 we show the execution time for different numbers of threads and in figure 5 we show the speedup compared to the ideal speedup.
The data shown in the plots are obtained by taking the average of 5 different runs. This helps to mitigate the effects of variability in execution time and provides more reliable measurements. By looking at the outputs in the `timing_hist_omp_strong.data` file one can also see the minimum and maximum times recorded across the runs, which gives an indication of the variability in performance.

From figure 4 we observe that the execution time decreases rapidly when increasing the number of threads from 1 up to about 8 threads, indicating that the parallelization successfully exploits the available cores in this regime. The minimum execution time is reached around 8-16 threads. However, when increasing the

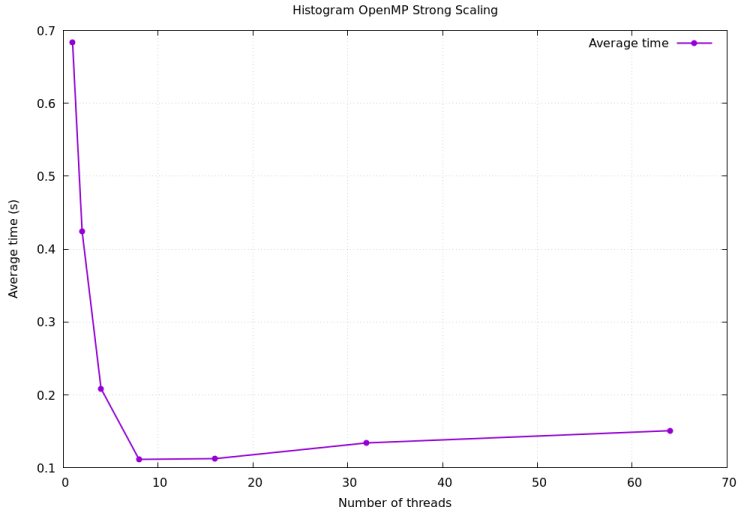


Figure 4: Strong scaling analysis of the parallelized histogram calculation

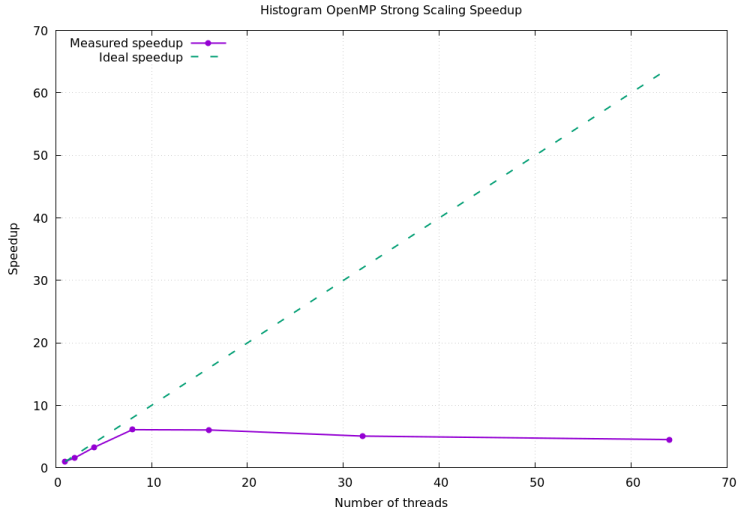


Figure 5: Speedup of the parallelized histogram calculation compared to the ideal speedup

number of threads beyond this point, the execution time no longer decreases and even starts to increase slightly.

This behavior is reflected in the speedup plot shown in figure 5. The speedup initially grows with the number of threads but quickly deviates from the ideal linear scaling. The maximum measured speedup is around 6×, after which the speedup saturates and slightly decreases for larger thread counts.

The loss of scalability at higher thread counts is likely due to memory-system effects. The algorithm performs a very small amount of arithmetic per input element and is therefore largely memory-bound. As the number of threads increases, the available memory bandwidth is gradually saturated, so additional threads cannot provide proportional speedup.

4.2. Use of LLM's

Chat gpt was used to properly create the code for running the strong scaling analysis. In particular, the parallelization was done by ourselves, but the code for running the strong scaling analysis and plotting the results was generated with the help of the LLM.

5. Parallel loop dependencies with `OpenMP` [15 points]

The code snippet of the parallelized version is shown in listing 9.

Listing 9: Parallelized version of the loop with dependencies using OpenMP

```
  #pragma omp parallel
  {
    double log_up = log(up);
    #pragma omp for firstprivate(up)
    for (n = 0; n <= N; ++n) {
      opt[n] = exp(n * log_up);
    }
}
Sn = opt[N]*up;
```

Table 5: Average performance over 10 runs for the sequential and OpenMP implementations (64 threads).

Metric	Sequential	Parallel (64 threads)
Average runtime [s]	4.0896	0.4390
Speedup		9.33
Final S_n	4.851650976×10^8	4.851650928×10^8
$\|\mathbf{opt}\|_2^2$	$5.884629305 \times 10^{15}$	$5.884629187 \times 10^{15}$

The original sequential loop exhibits a loop-carried dependency: the value of S_n at iteration n depends on the value computed in the previous iteration through the recurrence relation $S_{n+1} = S_n \cdot up$. Because of this dependency, each iteration must wait for the result of the previous one, which prevents a direct parallelization of the loop.
To eliminate this dependency, the recurrence can be reformulated analytically. Observing that $S_n = up^n$, each element of the array can be computed independently as

$$\texttt{opt[n]} = up^n = \exp(n \log(up)).$$

This reformulation removes the sequential dependency and allows the loop iterations to be executed independently.

The OpenMP directive `#pragma omp for` distributes the iterations among the available threads, enabling parallel execution. The `firstprivate(up)` clause ensures that each thread receives its own copy of the parameter `up`. After the parallel region, the final value of S_n is reconstructed as `Sn = opt[N] * up`. This approach preserves the correctness of the computation while allowing efficient parallelization.

In an initial implementation, the function `pow` was used to compute `opt[n]`. However, this function introduces unnecessary overhead and is less efficient than computing the value using the equivalent expression based on `exp` and `log`.

The results reported in Table 5 present the average performance over 10 executions of both the sequential implementation and the OpenMP parallel version using 64 threads.

The parallel implementation reduces the execution time from approximately 4.09 seconds in the sequential case to about 0.44 seconds, corresponding to a speedup of 9.33. Although this represents a significant performance improvement, the speedup remains far from the ideal value of 64. This behavior is expected because the computation is largely memory-bound and each loop iteration performs only a small amount of arithmetic work. Consequently, memory bandwidth limitations and cache effects restrict the achievable scalability when a large number of threads is used.

Finally, the numerical results obtained with the two implementations are almost identical. The final values of S_n and of the squared norm $\|\mathbf{opt}\|_2^2$ differ only slightly due to floating-point rounding errors introduced by the different order of operations in the parallel execution. These discrepancies are negligible and confirm that the parallelization preserves the correctness of the algorithm.

5.1. Use of LLMs

Chat GPT was used to generate the bash file to properly run 10 times both the sequential and parallel versions of the code and to compute the average execution times.

6. Quicksort using `OpenMP` tasks [20 points]

6.1. Parallelization

Here we are asked to parallelize the quicksort algorithm using `OpenMP` tasks. The code snippet of the parallelized version of the quicksort algorithm is shown in listing 10.

Listing 10: Parallelized version of the quicksort algorithm using OpenMP tasks

```
void quicksort_omp(double *data, int length, int cutoff) {
  if (length <= 1) return;

  double pivot = data[0];
  double temp;
  int left = 1;
  int right = length - 1;

  do {
    while (left < (length - 1) && data[left] <= pivot) left++;
    while (right > 0 && data[right] >= pivot) right--;

    if (left < right) {
      temp = data[left];
      data[left] = data[right];
      data[right] = temp;
    }
  } while (left < right);

  if (data[right] < pivot) {
    data[0] = data[right];
    data[right] = pivot;
  }

  /* Once in a final task, continue recursively in sequential mode */
  if (omp_in_final()) {
    quicksort_seq(data, right);
    quicksort_seq(&(data[left]), length - left);
    return;
  }

  #pragma omp task shared(data) firstprivate(right, cutoff) final(right <= cutoff)
  quicksort_omp(data, right, cutoff);

  #pragma omp task shared(data) firstprivate(left, length, cutoff) final((length - left)
      <= cutoff)
  quicksort_omp(&(data[left]), length - left, cutoff);

  #pragma omp taskwait
}
```

In this implementation, we use OpenMP tasks to parallelize the recursive calls of the quicksort algorithm. The function `quicksort_omp` takes an additional parameter `cutoff`, which determines when to switch from parallel to sequential execution. If the size of the subarray is less than or equal to the cutoff, the function will execute sequentially using `quicksort_seq`. Otherwise, it will create new tasks for the left and right partitions of the array. The `final` clause is used to indicate that once a task reaches a certain depth (when the size of the partition is small enough), it should execute sequentially to avoid excessive overhead from task creation. The `taskwait` directive ensures that the parent task waits for all child tasks to complete before proceeding.

6.2. Performance analysis

First of all we tried to find an optimal value for the cutoff parameter. We ran the parallelized quicksort algorithm with different cutoff values and measured the execution time for each case.The results are shown in table 6 and obtained by taking the average of 5 different runs for each cutoff value.

N	Cutoff	Threads	Avg Time [s]	Wrong
10000000	100	64	1.3922109940	0
10000000	500	64	0.3112580360	0
10000000	1000	64	0.2273432380	0
10000000	5000	64	0.2251094760	0
10000000	10000	64	0.2258029560	0
10000000	20000	64	0.2444692820	0
10000000	50000	64	0.2295661280	0
10000000	100000	64	0.2476871600	0

Table 6: Performance of the parallelized quicksort algorithm for different cutoff values.

The cutoff value that resulted in the best average runtime over 5 runs for $N = 10^7$ performance is highlighted in yellow in the table. But we notiche that nearby values perform similarly, indicating the optimum is not extremely sharp. This means that when the size of the partition is less than or equal to 5000, it is more efficient to execute the sorting sequentially rather than creating new tasks for smaller partitions. The choice of cutoff can significantly impact the performance of the parallelized algorithm, as it balances the overhead of task creation with the benefits of parallel execution.

We have then performed a strong scaling analysis by running the parallelized quicksort algorithm with the optimal cutoff value and varying the number of threads and for different values of N. The results are shown in figure 6. The data for these plots were obtained by taking the average of 5 different runs for each thread count to ensure reliable measurements.

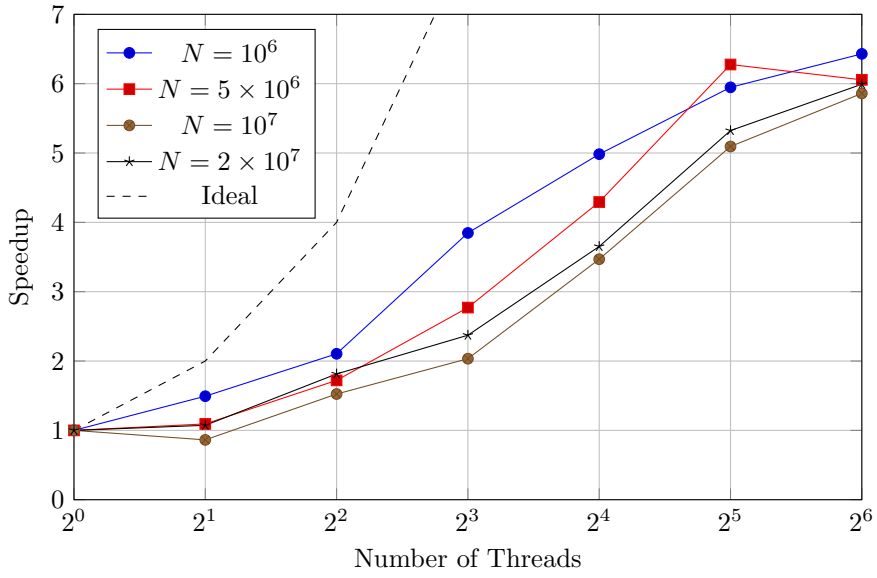


Figure 6: Strong scaling speedup for different problem sizes.

From the results shown in Figure 6, we observe that the parallel implementation of quicksort achieves a clear performance improvement as the number of threads increases, although the scaling remains far from the ideal case. For all tested problem sizes the speedup increases steadily up to 32 threads, after which the gains become more limited. The best observed speedup is around 6× when using 64 threads.

The deviation from the ideal linear scaling can be explained by several factors inherent to the quicksort algorithm and its task-based parallelization. First, quicksort exhibits irregular and data-dependent workloads, since the size of the partitions generated during the recursion depends on the pivot distribution. This can lead to load imbalance among threads, where some tasks finish earlier while others remain active. Second, the creation and scheduling of OpenMP tasks introduce runtime overhead, which becomes more significant

as the recursion produces many small tasks. Although the cutoff parameter mitigates this effect by switching to sequential execution for small partitions, some overhead remains unavoidable.

References

[1] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. 1st ed. CRC Press, 2010.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2025**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 3

Due date: Monday 07 April 2025, 18:00 CEST

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts). If you modified any provided build files, ensure they still work as expected.
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Task: Implementing the linear algebra functions and the stencil operators [50 Points]

1.1. Numerical method

We consider Fisher's reaction–diffusion equation on the square domain $\Omega = [0,1]^2$, where the population concentration $s(x,y,t)$ evolves under the combined effects of diffusion and logistic growth (eq. 1).

$$\frac{\partial s}{\partial t} = D\left(\frac{\partial^2 s}{\partial x^2} + \frac{\partial^2 s}{\partial y^2}\right)s + Rs(1-s) \tag{1}$$

The problem is complemented by constant Dirichlet boundary conditions $s = 0.1$ on $\partial\Omega$ and by an initial condition consisting of a localized perturbation, with $s = 0.2$ inside a small disk centered at $(1/4, 1/4)$ and $s = 0.1$ elsewhere.

The spatial domain is discretized on a uniform Cartesian grid with $(n+2)\times(n+2)$ points, where only the n^2 interior values are treated as unknowns. Spatial derivatives are approximated by a second-order central finite-difference scheme, yielding the standard five-point stencil, while time integration is performed using the implicit Euler method. At each time step, this results in a nonlinear algebraic system for the updated concentration field s^k.

The nonlinear system is solved with Newton's method applied to the residual equation $f(s^k) = 0$. At each Newton iteration, the corresponding linearized correction problem involving the Jacobian of the residual is solved approximately with a matrix-free Conjugate Gradient method, thereby avoiding explicit Jacobian inversion. The converged solution at time step k is then used as the initial guess for the subsequent time step.

We were asked to implement the linear algebra functions and the stencil operators, which are used in the Conjugate Gradient method and in the Newton's method, respectively.
One can look at the linear algebra functions in the file `linalg.cpp` and the stencil operators in the file `operators.cpp`.
In particular for the stencil operators we have to implement the residual function $f^k_{(i,j)}$ for the interior points, and for the points on the boundaries.
The residual function in the interior points is given by eq. 7 of the project handout. For the boundary points, we have to take into account the Dirichlet boundary conditions, obtaining:

$$\begin{aligned}
\text{East:}\quad & f^k_{(n+1,j)} = -(4+\alpha)s^k_{(n+1,j)} + s^k_{(n,j)} + 0.1 + s^k_{(n+1,j-1)} + s^k_{(n+1,j+1)} + \beta s^k_{(n+1,j)}(1 - s^k_{(n+1,j)}) + \alpha s^{k-1}_{(n+1,j)} \\
\text{West:}\quad & f^k_{(0,j)} = -(4+\alpha)s^k_{(0,j)} + 0.1 + s^k_{(1,j)} + s^k_{(0,j-1)} + s^k_{(0,j+1)} + \beta s^k_{(0,j)}(1 - s^k_{(0,j)}) + \alpha s^{k-1}_{(0,j)} \\
\text{North:}\quad & f^k_{(i,n+1)} = -(4+\alpha)s^k_{(i,n+1)} + s^k_{(i-1,n+1)} + s^k_{(i+1,n+1)} + 0.1 + s^k_{(i,n)} + \beta s^k_{(i,n+1)}(1 - s^k_{(i,n+1)}) + \alpha s^{k-1}_{(i,n+1)} \\
\text{South:}\quad & f^k_{(i,0)} = -(4+\alpha)s^k_{(i,0)} + s^k_{(i,1)} + 0.1 + s^k_{(i-1,0)} + s^k_{(i+1,0)} + \beta s^k_{(i,0)}(1 - s^k_{(i,0)}) + \alpha s^{k-1}_{(i,0)} \\
\text{NE corner:}\quad & f^k_{(n+1,n+1)} = -(4+\alpha)s^k_{(n+1,n+1)} + s^k_{(n,n+1)} + 0.1 + s^k_{(n+1,n)} + \beta s^k_{(n+1,n+1)}(1 - s^k_{(n+1,n+1)}) + \alpha s^{k-1}_{(n+1,n+1)} \\
\text{NW corner:}\quad & f^k_{(0,n+1)} = -(4+\alpha)s^k_{(0,n+1)} + 0.1 + s^k_{(1,n+1)} + s^k_{(0,n)} + \beta s^k_{(0,n+1)}(1 - s^k_{(0,n+1)}) + \alpha s^{k-1}_{(0,n+1)} \\
\text{SE corner:}\quad & f^k_{(n+1,0)} = -(4+\alpha)s^k_{(n+1,0)} + s^k_{(n,0)} + 0.1 + s^k_{(n+1,1)} + \beta s^k_{(n+1,0)}(1 - s^k_{(n+1,0)}) + \alpha s^{k-1}_{(n+1,0)} \\
\text{SW corner:}\quad & f^k_{(0,0)} = -(4+\alpha)s^k_{(0,0)} + 0.1 + s^k_{(1,0)} + s^k_{(0,1)} + \beta s^k_{(0,0)}(1 - s^k_{(0,0)}) + \alpha s^{k-1}_{(0,0)}
\end{aligned}$$

The `main.cpp` implements the time integration of the nonlinear system arising from the implicit discretization of Fisher's equation.
For each time step k, Newton's method is applied to solve the nonlinear system $f(s^k) = 0$. At iteration m, the residual is computed as:

$$r^{(m)} = f\big(s^{k,m}\big),$$

which corresponds to the call to the stencil operator `diffusion`. The convergence of the nonlinear iteration is monitored through the Euclidean norm of the residual:

$$\|r^{(m)}\|_2.$$

If the residual falls below a prescribed tolerance, the Newton iteration is terminated.
Otherwise, the Newton correction $\delta s^{(m)}$ is obtained by approximately solving the linearized system

$$J\big(s^{k,m}\big)\,\delta s^{(m)} = r^{(m)},$$

where J denotes the Jacobian of the residual. This system is solved using a matrix-free Conjugate Gradient method, implemented in `hpc_cg`, which avoids the explicit construction of the Jacobian matrix.

If the CG solver converges, the solution is updated according to the Newton step:

$$s^{k,m+1} = s^{k,m} - \delta s^{(m)},$$

implemented via the BLAS-like operation `hpc_axpy`.

The process is repeated until either convergence is reached or a maximum number of Newton iterations is exceeded. If convergence is not achieved, the simulation is terminated and an error message is reported.

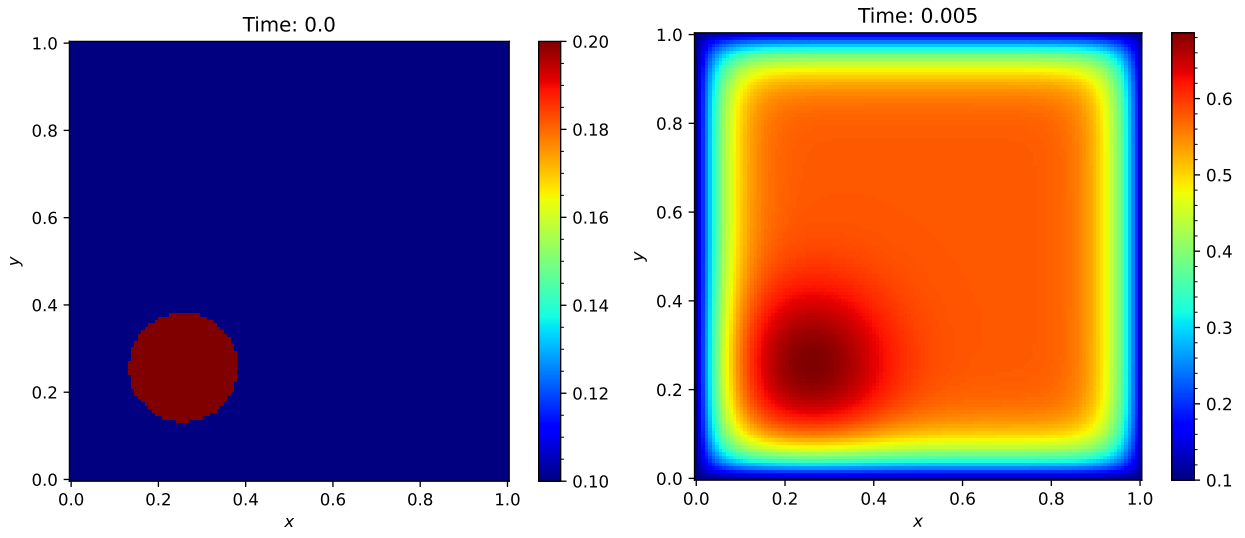
1.2. Results

The implementation of the linear algebra functions and the stencil operators was successful, as the program runs without errors and produces results consistent with the expected behavior of the solution to Fisher's equation.
In particular the output is given in listing 1

Listing 1: Output of serial implementation of Fisher mini-app

```
================================================================================
                     Welcome to mini-stencil!
version :: C++ Serial
mesh :: 128 * 128 dx = 0.00787402
time :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
simulation took 0.157575 seconds
1513 conjugate gradient iterations, at rate of 9601.76 iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 1, 128, 100, 1513, 300, 0.157575
```

Looking at the project handout and comparing with our results, we can see that the number of CG iterations and Newton iterations are consistent with the expected values, which indicates that our implementation is correct.
We can also see in figures 1a and 1b the initial and final solution of the problem, which are consistent with the images provided in the project handout, confirming the correctness of our implementation.



(a) Initial solution (b) Final solution

Figure 1: Comparison between initial and final solution

2. Task: Adding OpenMP to the nonlinear PDE mini-app [50 Points]

2.1. Welcome message in `main.cpp` and serial compilation [5 points]

To solve this task we have modified the `main.cpp` file with the modifications shown in listing 2.

Listing 2: Modifications to the welcome message in `main.cpp` to support OpenMP

```
[...]

#ifdef _OPENMP
#include <omp.h>
#endif

[...]

#ifdef _OPENMP
int threads = omp_get_max_threads();
#endif
#ifndef _OPENMP
int threads = 1;
#endif

[...]

#ifdef _OPENMP
std::cout << "version :: C++ OpenMP" << std::endl;
#endif
#ifndef _OPENMP
std::cout << "version :: C++ serial" << std::endl;
#endif

[...]
```

2.2. Linear algebra kernel [10 Points]

All the functions in the file `linalg.cpp` were parallelized using the `#pragma omp parallel for` directive to parallelize the loops.
For the function `hpc_norm2` and `hpc_dot` we have to take into account the reduction of the sum, which is done using the `reduction(+:sum)` clause.

2.3. The diffusion stencil [15 Points]

The parallelization of the `diffusion()` routine is based on OpenMP loop-level parallelism.
The largest amount of work is associated with the interior points, since they constitute almost the whole domain. For this reason, the interior stencil evaluation is parallelized as one can see in listing 3.

Listing 3: Interior grid points parallelization

```
#pragma omp parallel
{
    #pragma omp for collapse(2) nowait
    for (int j = 1; j < jend; j++) {
        for (int i = 1; i < iend; i++) {
            ...
        }
    }
    ...
}
```

The `#pragma omp parallel` directive creates a team of threads only once, and all subsequent loop worksharing constructs are executed inside the same parallel region. This is more efficient than creating a new parallel region for every loop separately.

For the interior update, the directive `#pragma omp for collapse(2)` is used. The `collapse(2)` clause merges the two nested loops over i and j into a single iteration space.

After the interior points, the four non-corner boundaries are treated in separate loops:

- east boundary,
- west boundary,
- north boundary,
- south boundary.

These loops are also parallelized with `#pragma omp for`. They are kept separate because the stencil on each boundary differs from the interior stencil due to the Dirichlet boundary values `bndE`, `bndW`, `bndN`, and `bndS`.

For the first boundary loops, the `nowait` clause is added obtaining `#pragma omp for nowait`.
This removes the implicit barrier at the end of each loop. Since the different boundary updates are independent, there is no need for all threads to synchronize after every single boundary loop. Avoiding these unnecessary barriers reduces synchronization overhead and improves performance.

The four corner points are computed outside the parallel region and therefore serially. This choice is reasonable because only four grid points are involved, so the computational cost is negligible compared to the interior and edge updates. Parallelizing these few operations would introduce extra overhead without any meaningful speedup.

An important property of this implementation is that it is free of data races. Each iteration writes to a distinct entry of the residual field $f(i, j)$, while the fields `s_old`, `s_new`, and the boundary arrays are only read. Therefore, no two threads update the same memory location simultaneously.

2.4. Strong scaling [10 points]

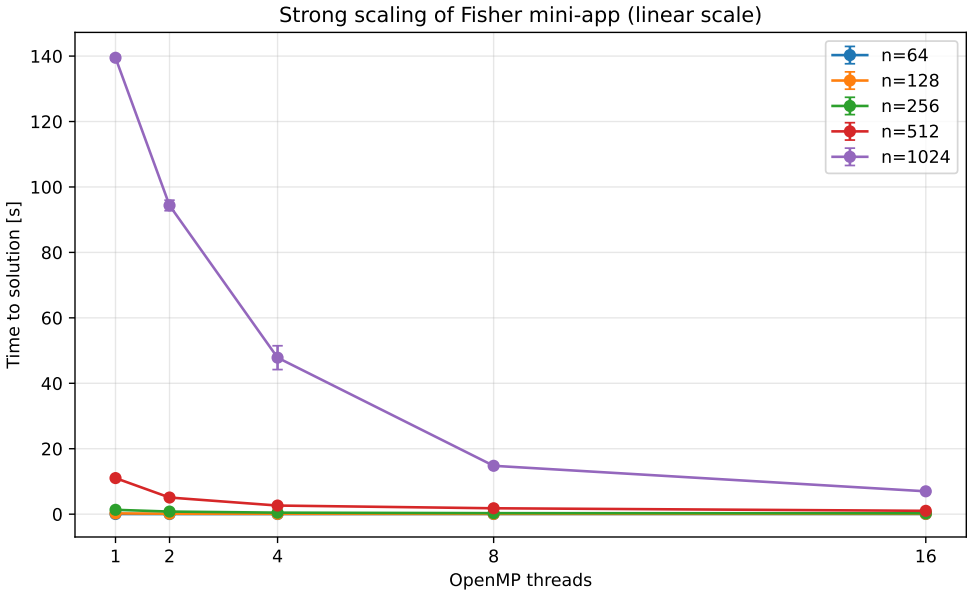
In the strong scaling test we have measured the time to solution for different combinations of number of threads and problem sizes. In particular, we have measured the time to solution for NCPU = 1, 2, 4, 8, 16 threads across resolutions of $n \times n$, where n = 64, 128, 256, 512, 1024.
The time to solution plot and the speedup plot are shown respectively in figures 2a and 2b.

The strong scaling results in figures 2a and 2b show a marked dependence on the problem size. For the smallest meshes, $n = 64$ and, to a lesser extent, $n = 128$, the scaling is poor and even deteriorates as the number of threads increases. In these cases, the time-to-solution is already very small for low thread counts, so the parallel overhead associated with OpenMP, such as thread management, synchronization, and barriers, becomes comparable to or larger than the useful computational work. As a consequence, the measured speedup remains limited and for $n = 64$ even drops below one, meaning that the parallel execution is slower than the serial one.

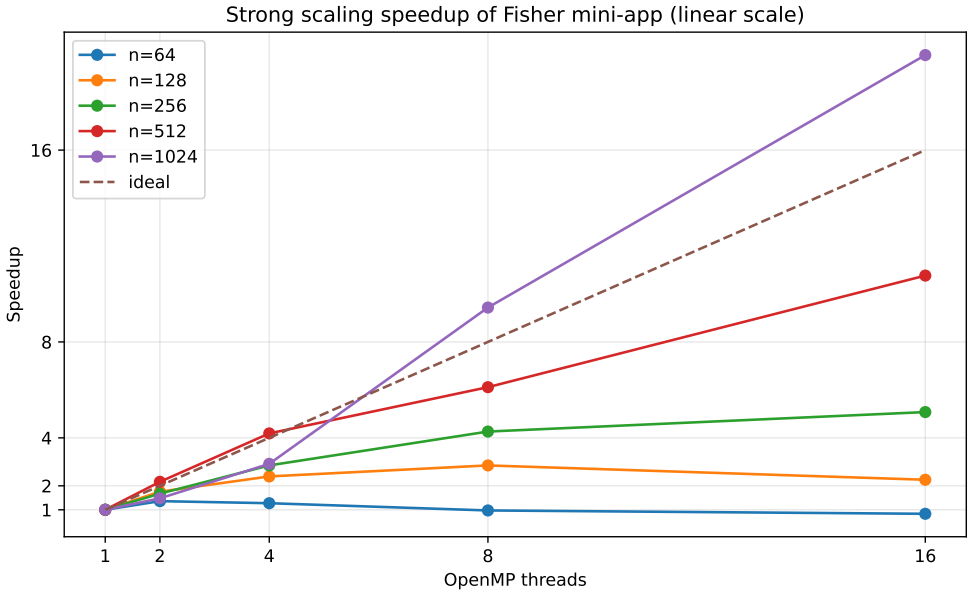
For the intermediate problem sizes, $n = 256$ and $n = 512$, the behavior improves significantly. The time-to-solution decreases as the number of threads increases, and the corresponding speedup is clearly positive. However, the scaling is still sublinear and tends to flatten at larger thread counts. This is a typical strong scaling effect: since the total problem size is fixed, increasing the number of threads reduces the amount of work assigned to each thread, while synchronization costs and memory-access overhead become more relevant.

The largest problem, $n = 1024$, exhibits the best scaling performance. In this case, the computational workload is sufficiently large to amortize the parallel overhead, and the speedup continues to increase up to 16 threads. Moreover, the measured speedup slightly exceeds the ideal trend at the highest thread count. This superlinear behavior can be explained by improved cache efficiency: when the domain is divided among multiple threads, the working set handled by each thread becomes smaller and may fit better into cache, thereby reducing memory traffic compared with the single-thread execution.

Overall, small grids do not provide enough work to exploit many threads efficiently, whereas larger grids benefit much more from parallelization.



(a) Time-to-solution.



(b) Speedup.

Figure 2: Strong scaling results on a linear scale.

To perfrom the strong scaling test every data reported in the plots of figures 2a and 2b is the average of 3 runs, and the error bars represent the standard deviation across these runs.

2.4.1. Use of LLMs

Chat GPT 5.2 was used to create code for plotting and the bash file to properly run the strong scaling analysis. It was also used to review my comments and my results.

2.5. Weak scaling [10 points]

To assess the weak scaling behavior of the Fisher mini-app, we measured the time-to-solution while increasing the total problem size proportionally to the number of OpenMP threads, so as to keep the workload per thread approximately constant. In particular, the tests were performed with $N_{\mathrm{CPU}} = 1, 4, 16, 64$ and with three base resolutions $n = 64$, $n = 128$, and $n = 256$. For each base resolution, the grid size was increased according to the weak scaling requirement.

Figure 3 reports the weak scaling results in two complementary ways. Figure 3a shows the time-to-solution as a function of the total problem size, with annotations indicating the corresponding grid and thread count. Figure 3b shows the same runtimes as a function of the number of OpenMP threads for the three base resolutions.

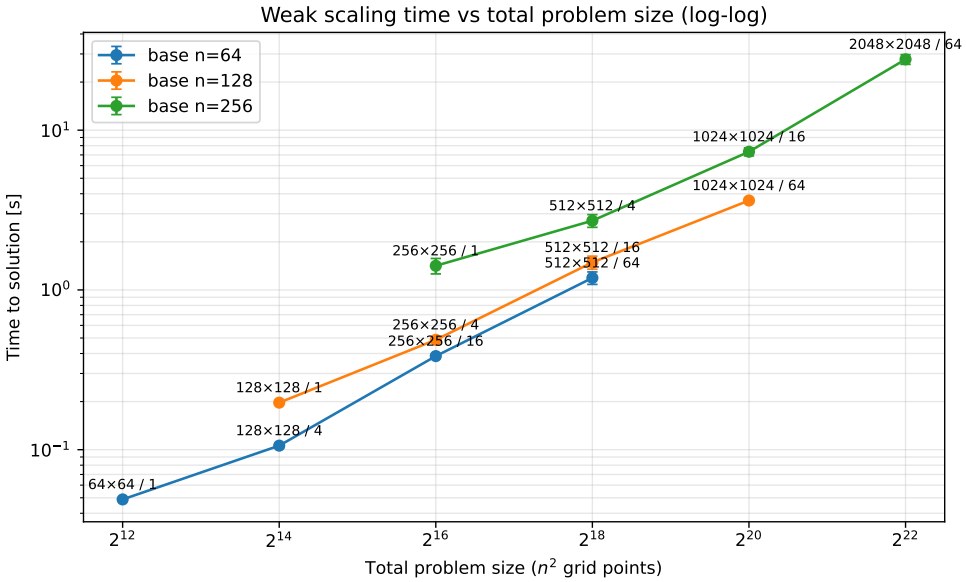
The weak scaling results in Figure 3 show that the runtime does not remain constant as the number of threads and the problem size increase proportionally. For all three base resolutions, the time-to-solution increases with increasing thread count, which indicates a departure from ideal weak scaling. This means that, although the workload per thread is kept constant in terms of grid points, the total computational cost does not scale proportionally.

A first reason for this behavior is the nonlinear nature of the problem. As suggested in the exercise statement, the convergence of the Newton–CG solver depends on the spatial resolution. When the grid is refined, the number of nonlinear and linear iterations may increase, so the cost per thread is no longer strictly constant even if the number of grid points handled by each thread remains approximately the same. Therefore, part of the runtime growth is due to the numerical solver itself and not only to parallel overhead. A second contribution comes from the parallel execution. As the number of threads increases, synchronization, reductions, and memory-access overhead become more relevant. These effects further prevent ideal weak scaling, for which the runtime would be expected to stay nearly flat.
More specifically, the base case $n = 64$ exhibits the best absolute runtimes, but also shows a clear increase from the smallest to the largest configuration. The same qualitative trend is observed for base $n = 128$ and $n = 256$, with larger base resolutions leading to systematically larger runtimes. In particular, the $n = 256$ series is the most expensive and grows most markedly, indicating that the increased resolution significantly affects the total solution cost.

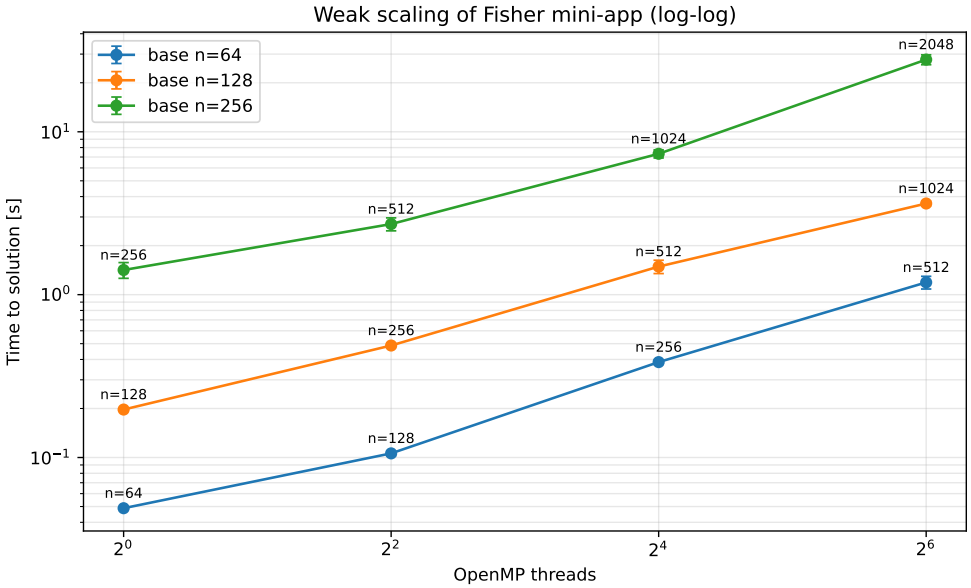
Overall, the results show that the code exhibits only limited weak scaling. The increase in runtime with thread count suggests that keeping the number of grid points per thread constant is not sufficient to maintain a constant time-to-solution for this application. The main reason is that the computational work is influenced not only by the local grid size per thread, but also by the resolution-dependent convergence of the nonlinear solver, together with the usual OpenMP overheads and memory effects.

2.5.1. Use of LLMs

Chat GPT 5.2 was used to create code for plotting and the bash file to properly run the strong scaling analysis. It was also used to review my comments and my results.



(a) Time-to-solution.



(b) Speedup.

Figure 3: Weak scaling results on a linear scale.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Report for Project 4

Due date: 27 April 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with MPI [10 points]

1.1. Parellization using MPI

The parellel implementation of the π computation is based on the idea of dividing the interval $[0, 1]$ into N subintervals. Each process computes the contribution to π from its assigned subinterval and then all contributions are summed together using MPI. In listing 1 we show the implementation of the parallel π computation using MPI.

Listing 1: Parallel π computation using MPI

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

h = 1.0 / (double)N;

MPI_Barrier(MPI_COMM_WORLD);
time_start = walltime();

for (uint64_t i = (uint64_t)rank; i < N; i += (uint64_t)size) {
    double x = (i + 0.5) * h;
    local_sum += 4.0 / (1.0 + x * x);
}

MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
double time = walltime() - time_start;
if (rank == 0) {
    pi = sum * h;
    printf("# pi N procs time [secs]\n");
    printf("%.15f %20" PRIu64 " %3d %.8f\n", pi, N, size, time);
    }

MPI_Finalize();
```

Each process starts from its rank and iterates over the indices of the subintervals assigned to it by incrementing the index by the total number of processes in each iteration. This way, each process computes the contribution to π from its assigned subintervals without overlap. After the local sums are computed, they are reduced to a single sum using `MPI_Reduce`, which is then used to compute the final value of π on the root process (rank 0).

1.2. Strong scaling

To evaluate the strong scaling of the parallel π computation, we run the program with a fixed problem size N and varying number of processes. The results are shown in figure 1, where we plot the speedup achieved by the parallel implementation compared to the serial version.

The strong scaling results obtained with MPI show a near-linear increase in speedup for a small number of processes, closely following the ideal trend. As the number of processes increases, the measured speedup progressively deviates from the ideal curve, highlighting the impact of communication overhead and reduced computational workload per process. Nevertheless, the scalability remains good, reaching a speedup close to 100 with 128 processes.

Comparing these results with the OpenMP reduction implementation in Table 1, we observe a very similar scaling behavior. Both approaches exhibit efficient parallelization at low to moderate parallelism and a loss of efficiency at higher core counts. However, OpenMP achieves slightly better efficiency at comparable core counts, likely due to the shared-memory model which avoids explicit communication overhead.

1.3. Weak scaling

The results of the weak scaling tests are shown in figure 2, where we plot the weak scaling efficiency as a function of the number of processes.

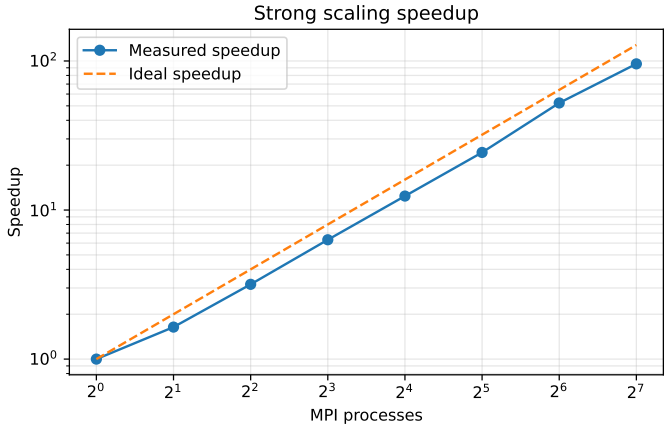


Figure 1: Strong scaling speedup of the parallel π computation. Obtained with $N = 10^9$.

Table 1: Strong scaling results using the OpenMP reduction clause ($N = 10^9$).

Threads	Speedup
1	1.00
2	1.84
4	3.26
8	6.39
16	8.86
32	17.66
64	31.06

The weak scaling efficiency remains high across all tested configurations, staying close to the ideal value of 1. This indicates that the workload is well balanced among MPI processes and that the parallelization strategy is effective.

A slight degradation in efficiency is observed as the number of processes increases, particularly at the highest core counts. This behavior is expected and can be attributed to increasing communication overhead and synchronization costs, which become more significant as more processes are involved.

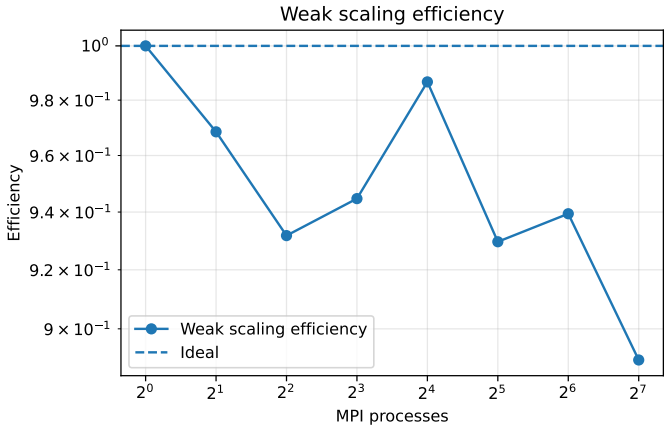


Figure 2: Weak scaling efficiency of the parallel π computation.

2. Ring sum using MPI [10 points]

In listing 2 we show the implementation of the ring sum using MPI.

Listing 2: Ring sum using MPI

```
int main(int argc, char *argv[]) {
int size, rank;
  MPI_Init(&argc, &argv);
  MPI_Comm_size(MPI_COMM_WORLD, &size);
  MPI_Comm_rank(MPI_COMM_WORLD, &rank);

  int send_to   = (rank + 1) % size;
  int recv_from = (rank - 1 + size) % size;

  int sum = rank;
  int value = rank;
  int recv_value;

  for (int i = 0; i < size - 1; i++) {
    MPI_Sendrecv(&value, 1, MPI_INT, send_to, 0,
                 &recv_value, 1, MPI_INT, recv_from, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    sum += recv_value;
    value = recv_value;
  }

  printf("Process %d: Sum = %d\n", rank, sum);

  MPI_Finalize();
  return 0;
}
```

We implemented the ring sum algorithm using a one-dimensional circular topology, where each process communicates with two neighbors: the next process in increasing rank direction and the previous one, defined respectively as $(\text{rank} + 1) \bmod P$ and $(\text{rank} - 1 + P) \bmod P$, where P is the number of processes.

Each process initializes a local accumulator with its own rank and a separate variable containing the value to be propagated through the ring. At every iteration, a process sends its current value to the right neighbor and simultaneously receives a value from the left neighbor. The received value is added to the local sum and then forwarded in the next iteration. After $P - 1$ iterations, each process has received all other ranks exactly once, thus computing the global sum.

To avoid potential deadlock issues, we used the `MPI_Sendrecv` routine, which combines sending and receiving into a single operation. This ensures that all processes can safely exchange messages without the risk of circular waiting that may arise when using separate blocking `MPI_Send` and `MPI_Recv` calls. The communication pattern is therefore fully synchronized and deadlock-free.

Finally, all processes obtain the same result, confirming the correctness of the implementation as one can see from the output shown in listing 3.

Listing 3: Output of ring sum

```
Process 1: Sum = 6
Process 0: Sum = 6
Process 2: Sum = 6
Process 3: Sum = 6
```

3. Cartesian domain decomposition and ghost cells exchange [20 points]

3.1. Cartesian 2D communicator

In listing 4 we show the implementation of the Cartesian 2D communicator using MPI.

Listing 4: Cartesian 2D communicator using MPI

```
1  int rank, size, i, j, dims[2], periods[2], rank_north, rank_south,
       rank_west, rank_east;
2  double data[DOMAINSIZE*DOMAINSIZE];
3  MPI_Request request;
4  MPI_Status status;
5  MPI_Comm comm_cart;
6  MPI_Datatype data_ghost;
7
8  dims[0]= 3;
9  dims[1]= 4;
10 periods[0]= 1;
11 periods[1]= 1;
12
13 MPI_Cart_create(MPI_COMM_WORLD, 2 ,dims , periods , 0 , &comm_cart);
14
15 MPI_Cart_shift(comm_cart, 1, 1, &rank_south, &rank_north);
16 MPI_Cart_shift(comm_cart, 0, 1, &rank_west, &rank_east);
```

The code first defines the Cartesian topology of the process grid by setting `dims[0] = 3` and `dims[1] = 4`, corresponding to a 3×4 decomposition. The array `periods` is set to 1 in both directions, meaning that the topology is periodic both horizontally and vertically: processes at one boundary are connected to processes on the opposite boundary. This implements the cyclic communication pattern required by the exercise.

The call to `MPI_Cart_create` constructs a new communicator, `comm_cart`, in which all processes are arranged according to this two-dimensional Cartesian topology. Using a Cartesian communicator makes neighbor discovery much simpler than manually computing ranks.

The two calls to `MPI_Cart_shift` are then used to determine the ranks of the neighboring processes in the two coordinate directions. In particular, the first call identifies the south and north neighbors, while the second call identifies the west and east neighbors.

3.2. Derived data type for column boundary

In listing 5 we show the implementation of the derived data type used for boundary communication.

Listing 5: Derived data type for column boundary using MPI

```
1 MPI_Type_vector(SUBDOMAIN, 1, DOMAINSIZE, MPI_DOUBLE, &data_ghost);
2 MPI_Type_commit(&data_ghost);
```

The local domain is stored in a one-dimensional array, and the boundary elements that have to be exchanged are not always contiguous in memory. For this reason, a derived MPI datatype is introduced with `MPI_Type_vector`.

The datatype `data_ghost` describes `SUBDOMAIN` elements of type `MPI_DOUBLE`, separated by a stride of `DOMAINSIZE` elements in memory. Hence, it represents a strided boundary segment of the local array. This is exactly the kind of memory layout that appears in the implementation shown in listing 6, where one class of boundaries is accessed through elements that are spaced by one full leading dimension. After its definition, `MPI_Type_commit` makes the datatype available for communication.

3.3. Exchange ghost cells

In listing 6 we show the implementation of the ghost cells exchange using MPI.

Listing 6: Ghost cells exchange using MPI

```
1  /* send top interior row (j = SUBDOMAIN) to north, receive bottom ghost
       row (j = 0) from south */
2      MPI_Sendrecv(&data[SUBDOMAIN + 1 * DOMAINSIZE], 1, data_ghost,
3                   rank_north, 0,
4                   &data[0 + 1 * DOMAINSIZE], 1, data_ghost,
5                   rank_south, 0,
6                   comm_cart, MPI_STATUS_IGNORE);
7
8      /* send bottom interior row (j = 1) to south,
9         receive top ghost row (j = SUBDOMAIN+1) from north */
10     MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], 1, data_ghost,
11                  rank_south, 1,
12                  &data[(SUBDOMAIN + 1) + 1 * DOMAINSIZE], 1, data_ghost,
13                  rank_north, 1,
14                  comm_cart, MPI_STATUS_IGNORE);
15
16
17     /* send right interior column (i = SUBDOMAIN) to east,
18        receive left ghost column (i = 0) from west */
19     MPI_Sendrecv(&data[1 + SUBDOMAIN * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
20                  rank_east, 2,
21                  &data[1 + 0 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
22                  rank_west, 2,
23                  comm_cart, MPI_STATUS_IGNORE);
24
25     /* send left interior column (i = 1) to west,
26        receive right ghost column (i = SUBDOMAIN+1) from east */
27     MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
28                  rank_west, 3,
29                  &data[1 + (SUBDOMAIN + 1) * DOMAINSIZE], SUBDOMAIN,
                        MPI_DOUBLE,
30                  rank_east, 3,
31                  comm_cart, MPI_STATUS_IGNORE);
```

This part of the code performs the exchange of ghost cells with the four neighboring processes of the Cartesian grid. The communication is split into four steps, one for each neighboring direction. In each step, an interior boundary of the local subdomain is sent to the corresponding neighbor, while the matching ghost boundary is received from the opposite side.

An important point is that the implementation follows the indexing convention induced by the linearized storage `data[i + j * DOMAINSIZE]`. As a consequence, the different geometric boundaries of the local domain are not all represented in memory in the same way: some are stored as strided segments and are therefore communicated through the derived datatype `data_ghost`, whereas others are contiguous in memory and can be sent directly as `SUBDOMAIN` entries of type `MPI_DOUBLE`.

For this reason, the first two `MPI_Sendrecv` calls use the derived datatype `data_ghost`. They exchange those boundaries that, under the chosen storage layout, are represented by elements separated by a stride of `DOMAINSIZE`. The last two `MPI_Sendrecv` calls instead exchange boundaries that are contiguous in memory and can therefore be communicated with the basic datatype `MPI_DOUBLE`.

Thus, the distinction in the code is not between "rows" and "columns" in an abstract geometric sense, but between strided and contiguous memory segments in the chosen array layout. With this interpretation, the implementation is fully consistent: the appropriate MPI datatype is used in each case according to how the corresponding boundary is stored in memory.

The use of `MPI_Sendrecv` is particularly important because it avoids possible deadlock issues that could arise if all processes tried to perform blocking send operations before posting matching receives. By combining send and receive into one single communication call, the ghost-cell exchange is both safe and simple to implement.

The output is shown in listing 7.
From it one can see the ranks of the neighboring processes and the data array after the ghost cell exchange, confirming that the communication was successful and that the ghost cells have been updated with the correct values from the neighbors.

Listing 7: Output of ghost cell exchange

```
North rank 6, South rank 4, West rank 1, East rank 9
data of rank 5 after communication
j
^
| -1.0  6.0  6.0  6.0  6.0  6.0  6.0 -1.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
| -1.0  4.0  4.0  4.0  4.0  4.0  4.0 -1.0
----------------------------------------> i
```

4. Parallelizing the Mandelbrot set using MPI [30 points]

4.1. Partitioning of the image

In listing 8 we can see the implementation of the function `createPartition`.

Listing 8: Partitioning of the image

```
Partition createPartition(int mpi_rank, int mpi_size) {
    Partition p;

    // TODO: determine size of the grid of MPI processes (p.nx, p.ny),
    //       see MPI_Dims_create()
    int dims[2];
    MPI_Dims_create(mpi_size , 2 , dims );
    p.ny = dims[0];
    p.nx = dims[1];

    // TODO: Create cartesian communicator (p.comm), we do not allow the
    //       reordering of ranks here, see MPI_Cart_create()
    MPI_Comm cart_comm;
    int periods[2] = {1, 1};
    MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart_comm);
    p.comm = cart_comm;

    // TODO: Determine the coordinates in the Cartesian grid (p.x, p.y),
    //       see MPI_Cart_coords()
    int coords[2];
    MPI_Cart_coords(cart_comm , mpi_rank , 2 , coords);
    p.y = coords[0];
    p.x = coords[1];

    return p;
}
```

The function `createPartition` defines a two-dimensional Cartesian decomposition of the MPI processes. The use of `MPI_Dims_create` allows to automatically determine a balanced process grid $(p.ny, p.nx)$, which tends to minimize load imbalance by distributing processes as evenly as possible across both dimensions.
A Cartesian communicator is constructed through `MPI_Cart_create`. Although periodic boundary conditions are enabled in both directions, this choice is not strictly required for the Mandelbrot computation, since no inter-process communication is needed.
Each process retrieves its coordinates in the Cartesian topology using `MPI_Cart_coords`. These coordinates uniquely identify the position of the process in the logical grid and are later used to map the process to a specific portion of the global image.

In listing 9 we can see the implementation of the function `updatePartition`.

Listing 9: Updating the partitioning of the image

```
Partition updatePartition(Partition p_old, int mpi_rank) {
    Partition p;

    // copy grid dimension and the communicator
    p.ny = p_old.ny;
    p.nx = p_old.nx;
    p.comm = p_old.comm;

    // TODO: Update the coordinates in the cartesian grid (p.x, p.y) for
       given
    //       mpi_rank, see MPI_Cart_coords()
    int coords[2];
    MPI_Cart_coords(p.comm , mpi_rank , 2 , coords);
```

```
13      p.y = coords[0];
14      p.x = coords[1];
15
16      return p;
17  }
```

The function `updatePartition` recomputes the Cartesian coordinates for a given rank while preserving the communicator and the grid dimensions. This ensures consistency of the mapping without reconstructing the full topology.

4.2. Dimensions and start/end of local domain

In listing 10 we can see the implementation of the function `createDomain`.

Listing 10: Dimensions and start/end of local domain

```
1  Domain createDomain(Partition p) {
2      Domain d;
3
4      // TODO: Compute size of the local domain
5      d.nx = IMAGE_WIDTH/p.nx;
6      d.ny = IMAGE_HEIGHT/p.ny;
7
8      // TODO: Compute index of the first pixel in the local domain
9      d.startx = p.x * d.nx;
10     d.starty = p.y * d.ny;
11
12     // TODO: Compute index of the last pixel in the local domain
13     d.endx = d.startx + d.nx - 1;
14     d.endy = d.starty + d.ny - 1;
15
16     return d;
17 }
```

The function `createDomain` assigns to each MPI process a rectangular subdomain of the global image. The local domain sizes are computed as

$$\text{d.nx} = \frac{\text{IMAGE_WIDTH}}{\text{p.nx}}, \qquad \text{d.ny} = \frac{\text{IMAGE_HEIGHT}}{\text{p.ny}},$$

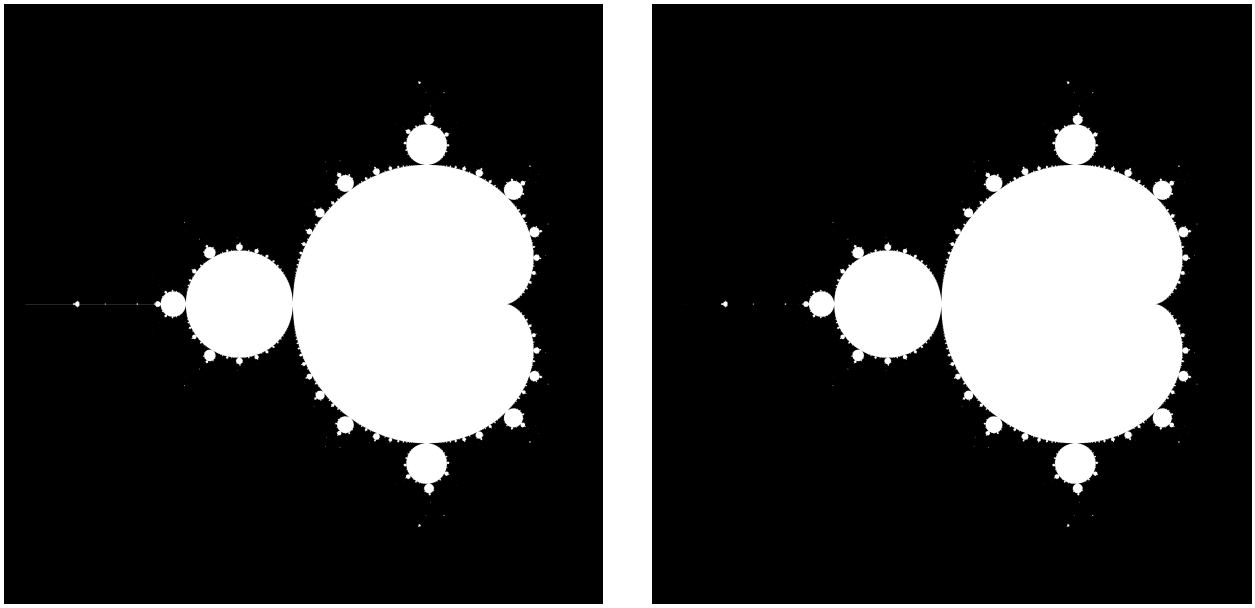
which corresponds to a block decomposition of the image. This approach is simple and efficient, as it avoids overlap between processes and leads to contiguous memory regions.
The starting indices (`d.startx, d.starty`) are determined from the Cartesian coordinates of the process, while the ending indices (`d.endx, d.endy`) define the inclusive bounds of the local subdomain.

This implementation assumes that the global image dimensions are divisible by the number of processes in each direction. While this assumption is acceptable for the current setup, a more general implementation would handle non-divisible dimensions by distributing remainder pixels.

4.3. Output check

Figure 3 compares the Mandelbrot set obtained with the parallel MPI implementation to the one produced by the serial code. The two images are visually identical, showing that the domain decomposition, the mapping from local to global pixel indices, and the final reconstruction of the global image on the master process are all correct.

No visible artifacts such as discontinuities or seams appear at subdomain boundaries in the parallel result. This confirms that each process computes its assigned region correctly and that the local domains are assembled consistently into the final image. Since the Mandelbrot computation itself is purely local, this visual agreement with the serial implementation provides a clear verification of correctness.



(a) Parallel implementation

(b) Serial implementation

Figure 3: Comparison of the Mandelbrot set computed with the parallel MPI implementation and the serial implementation.

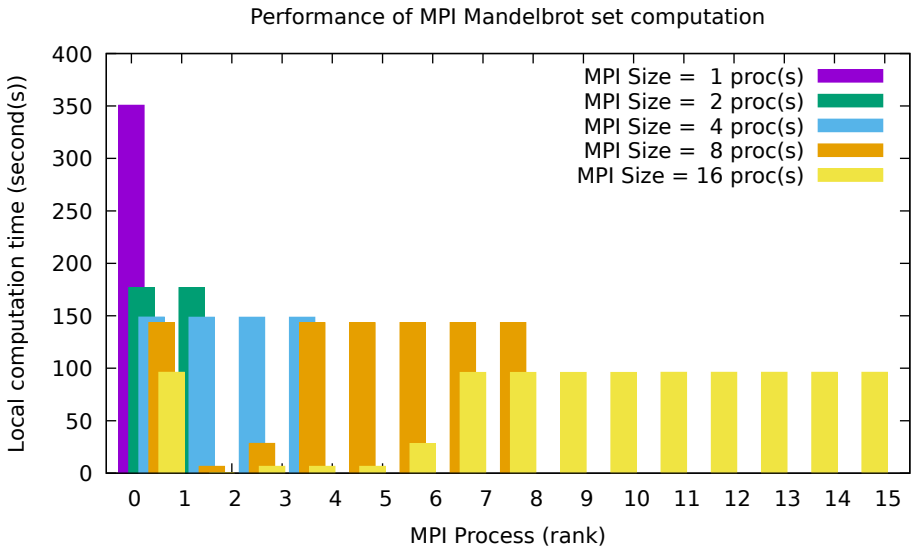


Figure 4: Performance of the parallel Mandelbrot set implementation.

4.4. Performance

Figure 4 presents the local computation time for each MPI process at different levels of parallelism.

A significant variability in execution time across processes can be observed. This is due to the intrinsic nature of the Mandelbrot set, where the number of iterations required for each pixel strongly depends on its position in the complex plane. Regions close to the boundary of the set require substantially more iterations, resulting in higher computational cost for the corresponding processes.

As a consequence, the workload is not uniformly distributed, and some processes complete their computation much earlier than others. This leads to load imbalance, which limits the overall parallel efficiency since the total execution time is determined by the slowest process.

Increasing the number of processes reduces the size of each subdomain but does not eliminate the imbalance. On the contrary, finer partitioning can make the variability more pronounced.

4.5. Possible performance improvements

Several strategies could be adopted to improve the performance of the MPI Mandelbrot implementation.

A first possible improvement concerns load balancing. The current block decomposition assigns each process one contiguous rectangular portion of the image. Since the Mandelbrot set is highly non-uniform, some subdomains contain many points close to the boundary of the set and therefore require significantly more iterations than others. This leads to processes with very different execution times. A better strategy would be to distribute the work in a more balanced way, for example by assigning rows or small tiles cyclically among the MPI processes. In this way, each process would receive a mixture of easy and expensive regions, reducing the imbalance.

Performance could also be improved by optimizing the local computation kernel. Since each process computes many independent pixels, the inner loops are good candidates for shared-memory parallelism through OpenMP, leading to a hybrid MPI+OpenMP implementation.

A further optimization would be to improve data locality and vectorization in the pixel computation loop. Careful restructuring of the computation, compiler optimization flags, and ensuring contiguous memory access patterns could help the compiler generate more efficient code.

5. Parallel matrix-vector multiplication and the power method [30 points]

We now show how all the TODOs in the provided code template have been implemented.
In listing 11 we show how we implement the partitioning of the "n" rows of the matrix evenly among the "size" MPI processes.

Listing 11: Partitioning of the matrix rows among MPI processes

```
if (rank < n % size) {
    nrows_local = n / size + 1;
    row_beg_local = rank * nrows_local;
  } else {
    nrows_local = n / size;
    row_beg_local = (n % size) * (n / size + 1)
                  + (rank - (n % size)) * (n / size);
  }

  row_end_local = row_beg_local + nrows_local - 1;
```

The code in Listing 11 defines the row-wise data distribution of the matrix across the MPI processes. Since the matrix is partitioned by rows, each process is responsible for a contiguous block of rows and therefore computes only the corresponding entries of the matrix-vector product. The distribution is made as balanced as possible: every process receives either $\lfloor n/size \rfloor$ or $\lfloor n/size \rfloor + 1$ rows, so that the load imbalance is at most one row. This is important for parallel efficiency, because the work associated with the matrix-vector multiplication is proportional to the number of rows owned by each process. The variables `row_beg_local` and `row_end_local` identify the global row range assigned to each rank, which is then used consistently throughout the power iteration to map local computations onto the correct global indices.

Then, we were asked to broadcast the random initial guess vector to all MPI processes. This was done using the `MPI_Bcast` function, as shown in listing 12.

Listing 12: Broadcasting the initial guess vector to all MPI processes

```
MPI_Bcast(y, n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
```

The operation shown in Listing 12 ensures that all MPI processes start from the same initial guess vector. In the power method, every process needs access to the full current iterate, because even though each rank stores only a subset of matrix rows, the multiplication of one local row with the vector requires all vector entries. For this reason, the vector generated on rank 0 is broadcast to the entire communicator. This collective communication step establishes a consistent initial state on all processes and allows the subsequent iterations to proceed in parallel without ambiguity about the input vector.

In listing 13 we show the implementation of the power method loop.

Listing 13: Power method loop for parallel matrix-vector multiplication

```
    // Power method
  double theta, error;
  double* y_local = (double*) calloc(nrows_local, sizeof(double));
  double* v       = (double*) calloc(n, sizeof(double));
  int iter;

  MPI_Barrier(MPI_COMM_WORLD);
  double time_start = walltime();

  // Build recvcounts and displacements for MPI_Allgatherv
  int* recvcounts = (int*) calloc(size, sizeof(int));
  int* displs     = (int*) calloc(size, sizeof(int));

  for (int p = 0; p < size; ++p) {
    if (p < n % size) {
      recvcounts[p] = n / size + 1;
      displs[p] = p * recvcounts[p];
```

```
18      } else {
19        recvcounts[p] = n / size;
20        displs[p] = (n % size) * (n / size + 1)
21                  + (p - (n % size)) * (n / size);
22      }
23    }
24      for (iter = 0; iter < niter; ++iter) {
25
26      // Normalize vector: v = y / ||y||_2
27      double norm2_local = 0.0;
28      for (int i_local = 0; i_local < nrows_local; ++i_local) {
29        int i_global = row_beg_local + i_local;
30        norm2_local += y[i_global] * y[i_global];
31      }
32
33      double norm2 = 0.0;
34      MPI_Allreduce(&norm2_local, &norm2, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
35
36      double norm = sqrt(norm2);
37
38      for (int i_global = 0; i_global < n; ++i_global) {
39        v[i_global] = y[i_global] / norm;
40      }
41
42      // Matrix-vector multiply: y_local = A_local * v
43      for (int i_local = 0; i_local < nrows_local; ++i_local) {
44        y_local[i_local] = 0.0;
45        for (int j_global = 0; j_global < n; ++j_global) {
46          y_local[i_local] += A[i_local*n + j_global] * v[j_global];
47        }
48      }
49
50      // Gather full y on all processes
51      MPI_Allgatherv(y_local, nrows_local, MPI_DOUBLE,
52                     y, recvcounts, displs, MPI_DOUBLE,
53                     MPI_COMM_WORLD);
54
55      // Compute eigenvalue: theta = v^T y
56      double theta_local = 0.0;
57      for (int i_local = 0; i_local < nrows_local; ++i_local) {
58        int i_global = row_beg_local + i_local;
59        theta_local += v[i_global] * y[i_global];
60      }
61
62      MPI_Allreduce(&theta_local, &theta, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
63
64      // Check convergence: error = ||y - theta v||_2
65      double error2_local = 0.0;
66      for (int i_local = 0; i_local < nrows_local; ++i_local) {
67        int i_global = row_beg_local + i_local;
68        double r = y[i_global] - theta * v[i_global];
69        error2_local += r * r;
70      }
71
72      double error2 = 0.0;
73      MPI_Allreduce(&error2_local, &error2, 1, MPI_DOUBLE, MPI_SUM,
            MPI_COMM_WORLD);
74
```

```
75      error = sqrt(error2);
76
77      if (verbose && rank == 0) {
78        printf("iteration / theta / error: %4d / %15.5f / %25.15e\n",
79               iter, theta, error);
80      }
81
82      if (error < tol * fabs(theta)) break;
83    }
84
85    MPI_Barrier(MPI_COMM_WORLD);
86    double time_end = walltime();
```

Listing 13 contains the core parallel implementation of the power method. The algorithm combines local computation with collective MPI communication in each iteration. First, every process computes the contribution of its locally owned entries to the Euclidean norm of the current iterate, and `MPI_Allreduce` is used to combine these partial results into the global norm. This allows all ranks to normalize the vector consistently. Each process then performs the matrix-vector multiplication only for its local block of rows, producing the partial result `y_local`. Since the next power iteration again requires the full vector on every process, the local pieces are exchanged with `MPI_Allgatherv`, which is necessary because the row distribution may be nonuniform.

The same parallel pattern is used for the computation of the Rayleigh quotient and the convergence criterion. Each process evaluates its local contribution to the scalar product $v^T y$ and to the residual norm $\|y - \theta v\|_2$, and the global values are reconstructed with `MPI_Allreduce`. In this way, the expensive matrix-vector multiplication is distributed across processes, while the collective operations maintain a synchronized global view of the iterate and of the convergence information. Overall, the implementation follows a replicated-vector, row-distributed-matrix strategy: the matrix is partitioned to distribute the computational work, whereas the vector is kept available on all processes to simplify the multiplication and the iterative update.

5.1. Correctness

We first verified the correctness of our implementation. This was by running the program on all the three test cases provided in the template, which consist of matrices with known dominant eigenvalues. In all cases, the computed eigenvalue converged to the expected value within the specified tolerance, confirming that the power method is correctly implemented and that the parallelization does not introduce errors. One can see the output for the first test case in listing 14.

Listing 14: Output of the power method for the first test case

```
1     Running: p=1, test_case=1, n=16, niter=100, tol=1e-12
2  ### 1, 16, 1, 16.000000, 0.000043 ###
3
4  Running: p=2, test_case=1, n=16, niter=100, tol=1e-12
5  ### 2, 16, 1, 16.000000, 0.000062 ###
6
7  Running: p=4, test_case=1, n=16, niter=100, tol=1e-12
8  ### 4, 16, 1, 16.000000, 0.000256 ###
9
10 Running: p=1, test_case=2, n=16, niter=100, tol=1e-12
11 ### 1, 16, 0, 1.000000, 0.000036 ###
12
13 Running: p=2, test_case=2, n=16, niter=100, tol=1e-12
14 ### 2, 16, 0, 1.000000, 0.000075 ###
15
16 Running: p=4, test_case=2, n=16, niter=100, tol=1e-12
17 ### 4, 16, 0, 1.000000, 0.000159 ###
18
19 Running: p=1, test_case=3, n=16, niter=100, tol=1e-12
20 ### 1, 16, 100, 15.999987, 0.000169 ###
21
22 Running: p=2, test_case=3, n=16, niter=100, tol=1e-12
```

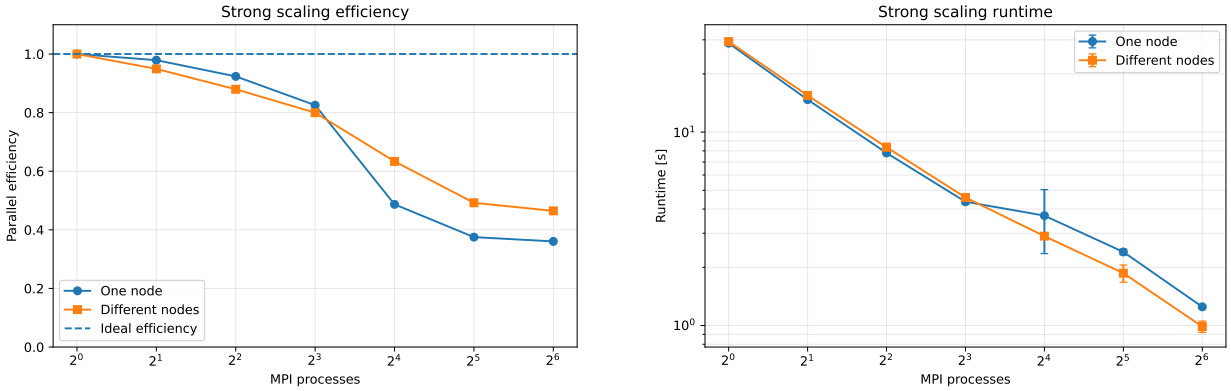
```
### 2, 16, 100, 15.999999, 0.000572 ###

Running: p=4, test_case=3, n=16, niter=100, tol=1e-12
### 4, 16, 100, 15.999972, 0.001331 ###
```

The results reported in Listing 14 confirm that the parallel implementation reproduces the expected dominant eigenvalues for all three verification cases and for all tested numbers of MPI processes. For test case 1, the dominant eigenvalue is correctly recovered as 16, while for test case 2 the method converges to the expected value 1. In test case 3, the approximation is again very close to the exact dominant eigenvalue, with only a small residual discrepancy due to the finite number of iterations and the slower convergence behavior of this case.
An important observation is that the computed eigenvalue remains essentially unchanged when increasing the number of processes from $p = 1$ to $p = 2$ and $p = 4$. This shows that the row-wise matrix distribution, the collective communications, and the global reductions are all functioning correctly.

5.2. Strong scaling

To assess the strong scaling behavior of the MPI implementation, we fixed the problem size to $n = 10\,000$, the maximum number of iterations to $niter = 300$, and the tolerance to a negative value, $\text{tol} = -10^{-6}$, so that the algorithm always performs the full number of iterations. The study was carried out for $p = 1, 2, 4, 8, 16, 32, 64$ MPI processes and for two process distributions: all MPI processes on a single node, and one MPI process per node on different nodes.



(a) Strong scaling efficiency.

(b) Strong scaling runtime.

Figure 5: Strong scaling results for the parallel power method for two MPI process distributions: all processes on one node and one process per node on different nodes.

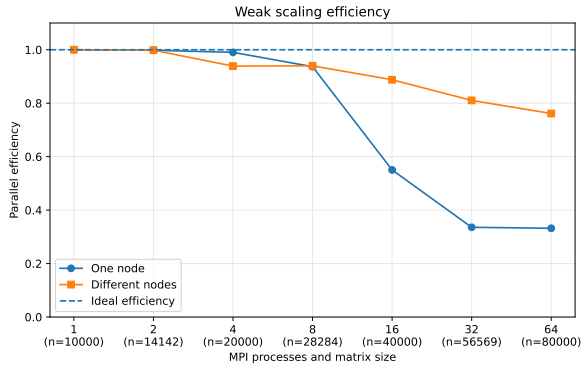
Figure 5 shows that the implementation scales well up to a moderate number of MPI processes, with efficiency close to ideal up to approximately 8 processes in both configurations. For larger process counts, the efficiency decreases significantly, which indicates that communication and synchronization overheads become increasingly important compared with the local computation. This behavior is expected, since each power-method iteration requires several collective communications, including global reductions and a global gather of the distributed vector.

Comparing the two process distributions, the multi-node configuration is slightly slower at small to intermediate process counts, reflecting the higher cost of inter-node communication. However, for the largest process counts, the different-node configuration becomes competitive and even slightly better than the one-node case. A plausible explanation is that, when all ranks are placed on one node, the execution becomes more affected by contention for shared node resources such as memory bandwidth and internal communication paths. In contrast, distributing one MPI rank per node reduces such local contention, although it introduces network communication overhead.

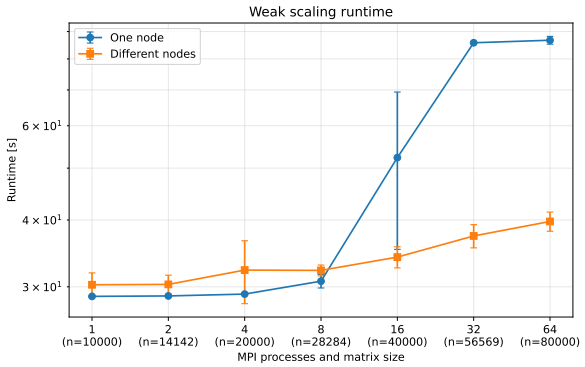
5.3. Weak scaling

For the weak scaling study, we ran the code with $p = 1, 2, 4, 8, 16, 32, 64$ MPI processes and increased the matrix size approximately proportionally to $\sqrt{p}$. Since the matrix has size $n \times n$, both the total memory requirement and the total computational work scale as n^2. Therefore, choosing $n(p) \propto \sqrt{p}$ keeps the memory

required per process and the work performed by each process approximately constant as the number of MPI processes increases. In practice, starting from $n_0 = 10\,000$ for one MPI process, we used $n(p) \approx 10\,000\sqrt{p}$. The maximum number of iterations was fixed to $niter = 300$, the tolerance was set to tol $= -10^{-6}$, and the study was repeated for two MPI process distributions: all processes on one node and one MPI process per node on different nodes.



(a) Weak scaling efficiency of the parallel power method.



(b) Weak scaling runtime of the parallel power method.

Figure 6: Weak scaling results for the parallel power method for two MPI process distributions: all processes on one node and one process per node on different nodes.

Figure 6 shows that the implementation exhibits good weak scaling up to about 8 MPI processes in both configurations, since the runtime remains nearly constant and the efficiency stays close to the ideal value. For larger numbers of processes, the one-node configuration deteriorates significantly: the runtime increases sharply from $p = 16$ onward and the efficiency drops well below one. This indicates that, although the work per process is kept approximately constant, the execution becomes increasingly limited by contention for shared node resources, such as memory bandwidth, as well as by communication and synchronization overheads.

The different-node configuration shows a more favorable trend at large process counts. Its runtime grows more gradually and the weak scaling efficiency remains substantially higher than in the one-node case for $p \geq 16$. This suggests that distributing one MPI process per node alleviates local contention effects, even though inter-node communication is more expensive.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 5

Due date: Monday 11 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Parallel Space Solution of a nonlinear PDE using MPI [in total 60 points]

1.1. Initialize/finalize MPI and welcome message [5 Points]

First we are asked to complete the initialization and finalization of MPI in the main function, and to print a welcome message with the simulation parameters in the file `main.cpp`.
In listings 1 we can see the initialization of MPI. We initialize MPI with `MPI_Init`, and then we query the number of processes and the rank of each process with `MPI_Comm_size` and `MPI_Comm_rank`. We store these values in the `domain` object, which is defined in the file `data.h`.

Listing 1: Initialization of MPI

```
1     MPI_Init(&argc, &argv);
2
3     int size = 1, rank = 0;
4     MPI_Comm_size(MPI_COMM_WORLD, &size);
5     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6
7     domain.rank = rank;
8     domain.size = size;
9
10    // read command line arguments
11    readcmdline(options, argc, argv);
12
13    // initialize sub-domain (data.{h,cpp})
14    domain.init(rank, size, options);
15    int nx = domain.nx;
16    int ny = domain.ny;
17    int N  = domain.N;
18    int nt  = options.nt;
```

In listing 2 we can see the finalization of MPI. We free the Cartesian communicator that we created in the initialization phase, and then we call `MPI_Finalize` to clean up all MPI resources.

Listing 2: Finalization of MPI

```
1     // finalize MPI
2     MPI_Comm_free(&domain.comm_cart);
3     MPI_Finalize();
```

In listing 3 we can see the welcome message that we print at the beginning of the simulation. We only print this message from the rank 0 process, to avoid cluttering the output.

Listing 3: Welcome message

```
1 if(rank == 0)
2     {
3         std::cout << std::string(80, '=') << std::endl;
4         std::cout << "                      Welcome to mini-stencil!" <<
              std::endl;
5         std::cout << "version   :: C++ MPI" << std::endl;
6         std::cout << "processes :: " << size << std::endl;
7         std::cout << "mesh      :: " << options.nx << " * " << options.nx
8                                      << " dx = " << options.dx << std::
                                         endl;
9         std::cout << "time      :: " << nt << " time steps from 0 .. "
10                                        << options.nt*options.dt << std
                                           ::endl;
11        std::cout << "iteration :: " << "CG "          << max_cg_iters
12                                     << ", Newton "    << max_newton_iters
13                                     << ", tolerance " << tolerance << std
                                         ::endl;
14        std::cout << std::string(80, '=') << std::endl;
```

```
15      }
```

1.2. Domain decomposition [10 Points]

Here we are asked to design a domain decomposition strategy that is compatible with any square grid size and any number of MPI processes while considering load balancing, communication overhead and computational efficiency. We were suggested to look at the short discussion in [1, Sec. 10.4.1].
We can first notice that for a subdomain of size $n_x \times n_y$ we have a total computation of $n_x \cdot n_y$ while the halo exchange requires $2 \cdot (n_x + n_y)$ communicated boundary points. For a fixed local area $A = n_x n_y$, the arithmetic-geometric mean inequality gives

$$n_x + n_y \geq 2\sqrt{n_x n_y} = 2\sqrt{A},$$

with equality when $n_x = n_y$. Therefore, for a fixed local area, communication is minimized when the rectangle is as close to a square as possible. Indeed in [1, Sec. 10.4.1] they suggest that we have to avoid slab decomposition. For example, if you split only in x, each process has a long thin strip. It computes fewer points, but the communicated boundary length stays large. A 2D Cartesian decomposition is better because both n_x and n_y decrease as the number of MPI processes increases.

Listing 4: Initialization of the sub-domain

```
1  void SubDomain::init(int mpi_rank, int mpi_size,
2                       Discretization& discretization) {
3      // Determine the number of sub-domains in the x and y dimensions
4      //       using MPI_Dims_create
5      int dims[2] = {0, 0};
6      MPI_Dims_create(mpi_size,2,dims);
7      ndomy = dims[0];
8      ndomx = dims[1];
9
10     // Create a 2D non-periodic Cartesian topology using MPI_Cart_create
11     int periods[2] = {0, 0};
12     MPI_Cart_create(MPI_COMM_WORLD,2,dims,periods,0,&comm_cart);
13
14     // Retrieve coordinates of the rank in the topology using
15     // MPI_Cart_coords
16     int coords[2] = {0, 0};
17     MPI_Cart_coords(comm_cart, mpi_rank , 2 , coords);
18     domy = coords[0] + 1;
19     domx = coords[1] + 1;
20
21     // Set neighbours for all directions using MPI_Cart_shift
22     MPI_Cart_shift(comm_cart, 0 , 1 , &neighbour_south, &neighbour_north);
23     MPI_Cart_shift(comm_cart, 1 , 1 , &neighbour_west, &neighbour_east);
24
25     // get Cartesian coordinates, but use 0-based versions for
           computations
26     int coord_y = coords[0];
27     int coord_x = coords[1];
28
29     // distribute grid points in x direction
30     int base_nx = discretization.nx / ndomx;
31     int rem_x   = discretization.nx % ndomx;
32
33     nx = base_nx + (coord_x < rem_x ? 1 : 0);
34
35     startx = coord_x * base_nx + std::min(coord_x, rem_x);
36     endx   = startx + nx - 1;
37
38     // distribute grid points in y direction
39     int base_ny = discretization.nx / ndomy;
```

```
    int rem_y   = discretization.nx % ndomy;

    ny = base_ny + (coord_y < rem_y ? 1 : 0);

    starty = coord_y * base_ny + std::min(coord_y, rem_y);
    endy   = starty + ny - 1;

    // get total number of grid points in this sub-domain
    N = nx*ny;

    rank = mpi_rank;
    size = mpi_size;
}
```

This code splits the global $N \times N$ grid into a 2D Cartesian process grid. The total number of points in each direction is divided by the number of processes along that axis, giving a base block size (`base_nx` and `base_ny`). The remainders (`rem_x` and `rem_y`) are assigned one extra grid point to the first ranks in each direction, so the subdomains differ in size by at most one cell. The expressions for `startx`, `endx`, `starty`, and `endy` then compute the global index range of each local block using prefix sums. This keeps the workload balanced while still covering the full domain without overlaps or gaps.

1.3. Linear algebra kernels [5 Points]

The linear algebra kernels were adapted to the MPI domain-decomposed setting. Most vector operations, such as `hpc_fill`, `hpc_axpy`, `hpc_scale`, `hpc_lcomb`, `hpc_copy`, `hpc_scaled_diff`, and `hpc_add_scaled_diff`, remain local. Each MPI process stores only its own subdomain, and these routines operate independently on the local entries.

The only kernels requiring MPI communication are the reduction operations `hpc_dot` and `hpc_norm2`. In both cases, each process first computes its local contribution. The local results are then combined across all MPI ranks using `MPI_Allreduce` with `MPI_SUM`. This is necessary because scalar products and norms are global quantities over the full distributed vector. In listing 5 we can see the implementation of the dot product. Each process computes the local dot product of its subdomain, and then we call `MPI_Allreduce` to sum up the local results into a global result. The same pattern is used for the 2-norm in listing 6, where we compute the local sum of squares and then take the square root of the global sum.

Listing 5: MPI parallel implementation of the dot product

```
double hpc_dot(Field const& x, Field const& y) {
    int N = y.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * y[i];
    }

    double global_result = 0.0;
    MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
                  data::domain.comm_cart);

    return global_result;
}
```

Listing 6: MPI parallel implementation of the 2-norm

```
double hpc_norm2(Field const& x) {
    int N = x.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * x[i];
    }
```

```
8
9       double global_result = 0.0;
10      MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
11                    data::domain.comm_cart);
12
13      return sqrt(global_result);
14  }
```

The conjugate-gradient solver `hpc_cg` did not require structural changes, since it calls the linear algebra kernels internally. Once the dot products and norms are made global, the CG coefficients are computed consistently across all ranks.

1.4. The diffusion stencil: Ghost cells exchange [10 Points]

Here we are asked to implement the ghost cell exchange between neighboring processes using non- blocking point-to-point communication with the objective to overlap computation and communication.
In listing 7 we provide a piece of the `diffusion` method where we tried to implement the ghost cell exchange.

Listing 7: Ghost cell exchange using non-blocking point-to-point communication

```
1       int countx = nx ; // number of interior points in x for rows
2       int county = ny ; // number of interior points in y for columns
3
4       // temporary send buffers for columns (non-contiguous in Field)
5       std::vector<double> sendW(county), sendE(county);
6       for (int j = 0; j < county; ++j) {
7           sendW[j] = s_new(0, j);          // left interior column
8           sendE[j] = s_new(nx-1, j);      // right interior column
9       }
10
11      MPI_Request reqs[8];
12      int reqcnt = 0;
13
14      // Post non-blocking receives into the boundary arrays (contiguous)
15      if (domain.neighbour_north != MPI_PROC_NULL) {
16          MPI_Irecv(&bndN[0], countx, MPI_DOUBLE, domain.neighbour_north, 0,
17                    domain.comm_cart, &reqs[reqcnt++]);
18      }
19      if (domain.neighbour_south != MPI_PROC_NULL) {
20          MPI_Irecv(&bndS[0], countx, MPI_DOUBLE, domain.neighbour_south, 0,
21                    domain.comm_cart, &reqs[reqcnt++]);
22      }
23      if (domain.neighbour_west != MPI_PROC_NULL) {
24          MPI_Irecv(&bndW[0], county, MPI_DOUBLE, domain.neighbour_west, 1,
25                    domain.comm_cart, &reqs[reqcnt++]);
26      }
27      if (domain.neighbour_east != MPI_PROC_NULL) {
28          MPI_Irecv(&bndE[0], county, MPI_DOUBLE, domain.neighbour_east, 1,
29                    domain.comm_cart, &reqs[reqcnt++]);
30      }
31
32      // Post non-blocking sends
33      if (domain.neighbour_north != MPI_PROC_NULL) {
34          MPI_Isend(&s_new(0, ny-1), countx, MPI_DOUBLE, domain.
                neighbour_north, 0,
35                    domain.comm_cart, &reqs[reqcnt++]);
36      }
37      if (domain.neighbour_south != MPI_PROC_NULL) {
38          MPI_Isend(&s_new(0, 0), countx, MPI_DOUBLE, domain.neighbour_south
                , 0,
39                    domain.comm_cart, &reqs[reqcnt++]);
40      }
```

```
41      if (domain.neighbour_west != MPI_PROC_NULL) {
42          MPI_Isend(sendW.data(), county, MPI_DOUBLE, domain.neighbour_west,
                1,
43                    domain.comm_cart, &reqs[reqcnt++]);
44      }
45      if (domain.neighbour_east != MPI_PROC_NULL) {
46          MPI_Isend(sendE.data(), county, MPI_DOUBLE, domain.neighbour_east,
                1,
47                    domain.comm_cart, &reqs[reqcnt++]);
48      }
49
50      // Wait for all posted requests
51      if (reqcnt > 0) MPI_Waitall(reqcnt, reqs, MPI_STATUSES_IGNORE);
```

This implementation exchanges the halo cells by separating the communication into two directions. The north and south boundaries can be sent directly, because the corresponding rows are contiguous in memory, while the west and east boundaries require temporary buffers since the columns of the local field are not stored contiguously. For this reason, first we pack the left and right interior columns into `sendW` and `sendE`, and then posts non-blocking receives and sends for all available neighbors. Although the exchange uses non-blocking MPI calls, in the current implementation the code waits immediately after posting the sends and receives. Therefore, the implementation avoids deadlock and prepares the code structure for overlap, but it does not yet realize a true overlap between communication and stencil computation. The tags `0` and `1` distinguish row and column messages, and the checks against `MPI_PROC_NULL` ensure that boundary processes simply skip the missing neighbors. Finally, `MPI_Waitall` guarantees that all halo exchanges are complete before the updated ghost layers are used in the next stencil step.

1.5. Implement parallel I/O [10 Points]

By looking at the file `mpio_io.c` we were asked to implement the output of the computed solution with MPI-I/O.
This was done by completing the function `write_binary` in `main.cpp` as shown in listing 8.

Listing 8: MPI-IO implementation of the output

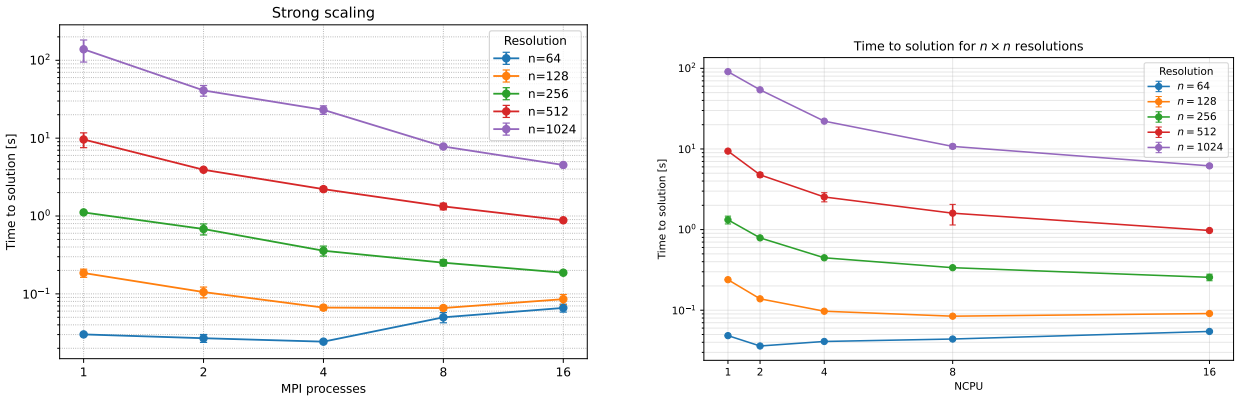
```
1  void write_binary(std::string fname, Field &u, SubDomain &domain,
2                    Discretization &options) {
3      MPI_File file;
4      MPI_File_open(MPI_COMM_WORLD, fname.c_str(),
5                    MPI_MODE_CREATE | MPI_MODE_WRONLY, MPI_INFO_NULL, &file)
                         ;
6
7      // global sizes (full global grid)
8      int sizes[2] = { options.nx, options.nx };
9      // local subarray sizes
10     int subsizes[2] = { domain.ny, domain.nx };
11     // start indices in the global array
12     int starts[2] = { domain.starty, domain.startx };
13
14     MPI_Datatype filetype;
15     MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C,
16                              MPI_DOUBLE, &filetype);
17     MPI_Type_commit(&filetype);
18
19     MPI_Offset disp = 0;
20     MPI_File_set_view(file, disp, MPI_DOUBLE, filetype, "native",
           MPI_INFO_NULL);
21
22     // write the local data collectively
23     MPI_File_write_all(file, u.data(), domain.N, MPI_DOUBLE,
           MPI_STATUS_IGNORE);
24
25     MPI_Type_free(&filetype);
```

```
26      MPI_File_close(&file);
27  }
```

The implementation uses MPI-IO to write the distributed field data into a single binary file. Each process defines a subarray datatype that describes its local portion of the global grid, and then collectively writes its data using `MPI_File_write_all`. The use of `MPI_Type_create_subarray` allows each process to specify its unique offset and size within the global array, enabling a seamless parallel output without the need for gathering data on a single rank.

1.6. Strong scaling [10 Points]

Here we are asked to plot the time to solution for NCPU $= 1, 2, 4, 8, 16$ processes across resolutions of $n \times n$, where $n = 64, 128, 256, 512, 1024$. Figure 1 shows the strong scaling behaviour of the MPI implementation



(a) Strong scaling results for the time to solution in the MPI implementation.

(b) Strong scaling results for the time to solution in the OpenMP implementation.

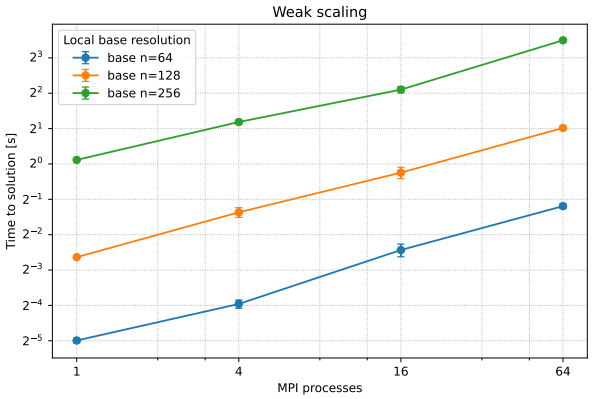
Figure 1: Strong scaling results for the time to solution.

and compares it with the OpenMP implementation from Project 3. For the larger resolutions, especially $n = 512$ and $n = 1024$, the MPI implementation shows a clear reduction of the time to solution when increasing the number of processes. This is the expected behaviour in a strong scaling experiment: the global problem size is fixed, but the work is distributed among more processes. The improvement is particularly visible for the largest grid, where the computational work per process is still large enough to compensate for the cost of MPI communication.

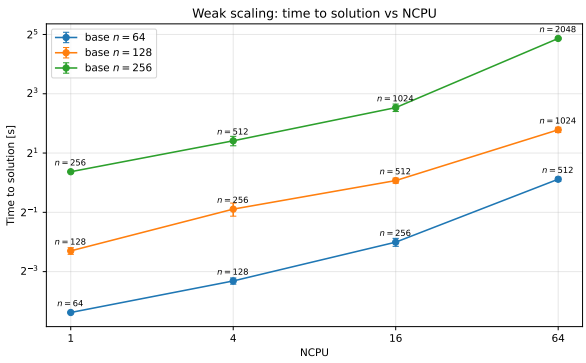
For small resolutions, for example $n = 64$ and partly $n = 128$, the scaling is much weaker and can even become non-monotonic. This is expected because the amount of computation per process is very small. In this regime, the overhead due to MPI initialization, halo exchanges, global reductions in the conjugate-gradient solver, and synchronization becomes comparable to the actual stencil computation. Therefore, adding more processes does not necessarily reduce the runtime. For $n = 64$, the runtime even increases when many processes are used, which indicates that the problem is too small to benefit from parallelization.

Compared with the OpenMP implementation, the qualitative behaviour is similar: large resolutions scale better, while small resolutions are dominated by parallel overhead. However, the reason for the overhead is different. In OpenMP, all threads run in shared memory, so there is no explicit halo exchange between processes, but the code can suffer from thread-management overhead, synchronization, cache effects, and memory-bandwidth contention. In the MPI version, each process owns only a subdomain and communicates explicitly with neighbouring processes, so the main overhead comes from halo exchanges and global reductions. For large enough problems, the MPI implementation scales well because the local computation dominates the communication cost. For small problems, both implementations are limited by overhead rather than by floating-point work.

Overall, the strong scaling results confirm that the MPI implementation is effective for sufficiently large grids, while small grids are not suitable for strong scaling because the local workload becomes too small.



(a) Weak scaling results for the time to solution.



(b) Weak scaling results for the time to solution vs. number of processes.

Figure 2: Weak scaling results for the time to solution.

1.7. Weak scaling [10 Points]

Figure 2 shows the weak scaling behaviour. In a weak scaling experiment, the problem size is increased together with the number of processes in such a way that the amount of work per process should remain approximately constant. Ideally, the time to solution would therefore remain constant as the number of MPI processes increases.

The measured results show that the time to solution increases with the number of processes for all base resolutions. This means that the weak scaling is not ideal. The increase is caused by the fact that, although the local computational work per process is kept approximately constant, the communication and synchronization costs grow when more processes are used. In particular, the halo exchange involves more subdomain boundaries, and the conjugate-gradient method requires global reductions, whose cost increases with the number of MPI ranks. These global reductions are especially important because they introduce synchronization across all processes.

The weak scaling behaviour is better for smaller base resolutions in absolute time, but the relative increase is still visible. For larger base resolutions, such as base $n = 256$, the runtime is higher because each process has a larger local problem. The growth of the runtime with $NCPU$ shows that communication and synchronization become increasingly important as the global problem size and the number of processes increase.

Compared with the OpenMP implementation of Project 3, the MPI implementation again shows the same general trend: weak scaling is not perfectly flat. However, the source of the loss of efficiency is different. In OpenMP, the threads share the same memory space, so there is no explicit message passing, but the performance can be limited by memory bandwidth, cache/NUMA effects, and synchronization inside the iterative solver. In MPI, the memory is distributed, which improves memory locality, but this comes at the price of explicit communication between neighbouring subdomains and collective reductions. Therefore, the MPI weak scaling is mainly limited by communication and global synchronization, while the OpenMP weak scaling is mainly limited by shared-memory contention and thread synchronization.

Overall, the weak scaling results are reasonable for a distributed-memory stencil code. The runtime does not remain perfectly constant, but the increase is expected because the PDE solver is not purely local: every timestep and every conjugate-gradient iteration require communication between subdomains and global reductions.

1.8. Bonus [20 Points]: Overlapping communication/computation details

2. Python for High-Performance Computing [in total 40 points]

2.1. Sum of ranks: MPI collectives [5 Points]

We were asked to implement the sum of ranks using MPI for Python's collective communication method.

2.1.1. Pickle based

In the first case we had to use pickle-based communication of generic Python objects, i.e. the all-lowercase methods.
In listing 9 we can see the implementation of the sum of ranks using the pickle-based collective communication method.

Listing 9: Sum of ranks using pickle-based collective communication

```
from mpi4py import MPI

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

# pickle-based
total = comm.allreduce(rank, op=MPI.SUM)

print(
    f"Sum is {total} on processor {proc}, "
    f"rank {rank} out of {size} processes"
)
```

The pickle-based implementation uses the lower-case collective routine `comm.allreduce`. Each process contributes its local Python integer `rank`, and the operation `MPI.SUM` combines all contributions into the global sum. Since `allreduce` is a collective operation, the result is returned to every MPI process. The lower-case mpi4py interface works with generic Python objects and therefore uses pickle serialization internally. This makes the implementation very compact, although it may introduce more overhead than direct buffer communication for large data objects.

Listing 10: Output of the sum of ranks using pickle-based collective communication

```
ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/01_sum$ srun
   python3 sum_pickle.py
Sum is 28 on processor eu-a2p-301, rank 2 out of 8 processes
Sum is 28 on processor eu-a2p-301, rank 3 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 0 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 6 out of 8 processes
Sum is 28 on processor eu-a2p-335, rank 5 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 1 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 7 out of 8 processes
Sum is 28 on processor eu-a2p-335, rank 4 out of 8 processes
```

The output confirms that all MPI processes compute the same final sum. In this run, `size = 8`, therefore the expected result is

$$0 + 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28.$$

Every rank prints `Sum is 28`, showing that the ring communication has circulated all rank values correctly through the communicator.

2.1.2. Direct array data communication

In the second case we had to use direct array data communication with the all-uppercase methods, i.e. the MPI methods that directly handle array data without pickling.
In listing 11 we can see the implementation of the sum of ranks using direct array data communication.

Listing 11: Sum of ranks using direct array data communication

```
from mpi4py import MPI
import numpy as np

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

# direct-buffer collective communication
sendbuf = np.array(rank, dtype=np.int32)
recvbuf = np.array(0, dtype=np.int32)

comm.Allreduce(sendbuf, recvbuf, op=MPI.SUM)

print(
    f"Sum is {recvbuf.item()} on processor {proc}, "
    f"rank {rank} out of {size} processes"
)
```

The direct-buffer implementation uses the upper-case collective routine `comm.Allreduce`. In contrast to the lower-case `allreduce`, this routine operates directly on buffer-provider objects, here NumPy arrays. Each process stores its local contribution in the array `sendbuf`, while the global sum is written into `recvbuf`. Since the data are communicated as raw memory buffers with an explicit NumPy datatype, mpi4py can avoid Python object serialization and use a communication path closer to the underlying C MPI interface.

Listing 12: Output of the sum of ranks using direct array data communication

```
ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/01_sum$ srun
   python3 sum_fast.py
Sum is 28 on processor eu-a2p-335, rank 4 out of 8 processes
Sum is 28 on processor eu-a2p-335, rank 5 out of 8 processes
Sum is 28 on processor eu-a2p-301, rank 2 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 0 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 6 out of 8 processes
Sum is 28 on processor eu-a2p-301, rank 3 out of 8 processes
Sum is 28 on processor eu-a2p-299, rank 1 out of 8 processes
Sum is 28 on processor eu-a2p-344, rank 7 out of 8 processes
```

The result is again correct: for eight MPI processes the sum of ranks is equal to 28, and all ranks obtain the same value. This shows that the direct-buffer implementation is logically equivalent to the pickle-based version.

2.2. Ghost cell exchange between neighboring processes [5 Points]

Here we were asked to create a two-dimensional, periodic Cartesian topology and implement ghost cell exchange.
In listing 13 we can see the implementation of the ghost cell exchange between neighboring processes in Python using mpi4py.

Listing 13: Ghost cell exchange between neighboring processes in Python using mpi4py

```
from mpi4py import MPI
import numpy as np

# get comm, size, rank & host name
comm = MPI.COMM_WORLD
size = comm.Get_size()
rank = comm.Get_rank()
proc = MPI.Get_processor_name()

SUBDOMAIN_SIZE = 6
```

```
DOMAIN_SIZE = SUBDOMAIN_SIZE + 2

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Initialization
data = np.full((DOMAIN_SIZE, DOMAIN_SIZE), -1, dtype=np.int32)
data[1:-1, 1:-1] = rank

dims = [3, 4]
periods = [True, True]

cart = comm.Create_cart(dims=dims, periods=periods, reorder=False)

rank_south, rank_north = cart.Shift(1, 1)
rank_west, rank_east   = cart.Shift(0, 1)

# North/South exchange
cart.Sendrecv(
    sendbuf=data[DOMAIN_SIZE-2, 1:-1],
    dest=rank_north,
    sendtag=0,
    recvbuf=data[0, 1:-1],
    source=rank_south,
    recvtag=0
)

cart.Sendrecv(
    sendbuf=data[1, 1:-1],
    dest=rank_south,
    sendtag=1,
    recvbuf=data[DOMAIN_SIZE-1, 1:-1],
    source=rank_north,
    recvtag=1
)

# West/East exchange: columns are not contiguous, so use temporary arrays
recv_west = np.empty(SUBDOMAIN_SIZE, dtype=np.int32)
recv_east = np.empty(SUBDOMAIN_SIZE, dtype=np.int32)

send_east = np.ascontiguousarray(data[1:-1, DOMAIN_SIZE-2])
send_west = np.ascontiguousarray(data[1:-1, 1])

cart.Sendrecv(
    sendbuf=send_east,
    dest=rank_east,
    sendtag=2,
    recvbuf=recv_west,
    source=rank_west,
    recvtag=2
)
data[1:-1, 0] = recv_west

cart.Sendrecv(
    sendbuf=send_west,
    dest=rank_west,
    sendtag=3,
    recvbuf=recv_east,
    source=rank_east,
```

```
71      recvtag=3
72  )
73  data[1:-1, DOMAIN_SIZE-1] = recv_east
```

Each MPI process owns a local subdomain of size `SUBDOMAIN_SIZE` by `SUBDOMAIN_SIZE`. The full local array `data` has size `DOMAIN_SIZE` by `DOMAIN_SIZE`, where `DOMAIN_SIZE = SUBDOMAIN_SIZE + 2`. The additional two rows and columns are the ghost layers. The interior entries `data[1:-1, 1:-1]` are initialized with the MPI rank of the process, while the ghost cells are initialized to `-1`. After the exchange, the ghost cells should contain the rank numbers of the neighbouring processes.

The Cartesian topology is created with `comm.Create_cart`. In this implementation, the process grid is fixed to `dims = [3, 4]`, meaning that the 12 MPI processes are arranged in a two-dimensional grid with 3 processes in one direction and 4 processes in the other. The array `periods = [True, True]` makes the topology periodic in both directions. Therefore, processes on the boundary of the process grid still have neighbours: for example, the process at the right boundary has as east neighbour the corresponding process at the left boundary.
The neighbours are obtained using `cart.Shift`. The call `cart.Shift(1, 1)` returns the neighbours in the second Cartesian direction, which are used here as the south and north neighbours. Similarly, `cart.Shift(0, 1)` returns the neighbours in the first Cartesian direction, used here as the west and east neighbours. This avoids manually computing neighbour ranks and makes the code independent of the rank ordering inside the Cartesian communicator.

The north/south exchange communicates rows. Since NumPy arrays are stored in row-major order, slices such as `data[DOMAIN_SIZE-2, 1:-1]` and `data[1, 1:-1]` are contiguous in memory. Therefore, they can be passed directly to `Sendrecv`. The last interior row is sent to the north neighbour and the first ghost row is filled with data received from the south neighbour. Conversely, the first interior row is sent to the south neighbour and the last ghost row is filled with data received from the north neighbour.
The west/east exchange communicates columns. In row-major storage, columns are not contiguous in memory. For this reason, the code first creates contiguous temporary send buffers using `np.ascontiguousarray`. The east interior column is copied into `send_east`, and the west interior column is copied into `send_west`. The received columns are stored in the temporary arrays `recv_west` and `recv_east`, and then copied into the corresponding ghost columns of `data`. This avoids problems associated with sending non-contiguous NumPy views directly.

The use of `Sendrecv` again prevents deadlocks. All processes participate in the same pairwise exchange pattern, and each call simultaneously sends one boundary buffer and receives the corresponding ghost buffer. Different message tags are used for the two directions of each exchange. This makes the matching of messages unambiguous and prevents a message from one exchange direction from being incorrectly matched with another one.

Listing 14: Output of the ghost cell exchange between neighboring processes in Python using mpi4py

```
1  ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/02_ghost$ srun
       python3 ghost.py
2  North rank 6, South rank 4, West rank 1, East rank 9
3  data of rank 5 after communication
4  j
5  ^
6  | -1.0  6.0  6.0  6.0  6.0  6.0  6.0 -1.0
7  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
8  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
9  |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
10 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
11 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
12 |  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
13 | -1.0  4.0  4.0  4.0  4.0  4.0  4.0 -1.0
14 ----------------------------------------> i
```

The printed array corresponds to the local subdomain of rank 5 after the ghost-cell exchange. The interior of the array is filled with the value 5, because those entries belong to rank 5 itself. The ghost layers contain the ranks of the neighbouring processes in the Cartesian topology.

The top ghost row contains the value 6, which corresponds to the north neighbour of rank 5. The bottom ghost row contains the value 4, corresponding to the south neighbour. Similarly, the left ghost column

contains the value 1, and the right ghost column contains the value 9. This confirms that each process has correctly exchanged its boundary data with the four neighbouring processes.

The corner entries remain equal to `-1`. This is expected because the implemented exchange only communicates the north, south, east, and west neighbours required by a standard five-point stencil. Corner values would only be needed for a stencil involving diagonal neighbours, which is not the case here.

2.3. A self-scheduling example: Parallel Mandelbrot [30 Points]

2.3.1. Implementation

Here we were asked to implement the manager-worker pattern to compute the Mandelbrot set in parallel. The manager process is responsible for distributing the work and collecting the results, while the worker processes perform the actual computation of the Mandelbrot set for assigned tasks.
The implementation of the manager-worker pattern for computing the Mandelbrot set is provided in the listing 15.

Listing 15: Manager-worker pattern for computing the Mandelbrot set in parallel

```
# some parameters
MANAGER = 0       # rank of manager
TAG_TASK      = 1 # task       message tag
TAG_TASK_DONE = 2 # tasks done message tag
TAG_DONE      = 3 # done       message tag

def manager(comm, tasks):
    size = comm.Get_size()
    nworkers = size - 1

    next_task = 0
    active_workers = 0
    TasksDoneByWorker = {i: 0 for i in range(size)}

    # send initial tasks
    for worker in range(1, size):
        if next_task < len(tasks):
            comm.send((next_task, tasks[next_task]), dest=worker, tag=
               TAG_TASK)
            next_task += 1
            active_workers += 1
        else:
            comm.send(None, dest=worker, tag=TAG_DONE)

    # receive results and assign new tasks
    status = MPI.Status()

    while active_workers > 0:
        task_id, completed_task = comm.recv(
        source=MPI.ANY_SOURCE,
        tag=TAG_TASK_DONE,
        status=status
        )

        worker = status.Get_source()
        TasksDoneByWorker[worker] += 1

        # replace the completed task in the task list
        tasks[task_id] = completed_task

        if next_task < len(tasks):
            comm.send((next_task, tasks[next_task]), dest=worker, tag=
               TAG_TASK)
            next_task += 1
```

```
43          else:
44              comm.send(None, dest=worker, tag=TAG_DONE)
45              active_workers -= 1
46
47      return TasksDoneByWorker
48
49
50  def worker(comm):
51      while True:
52          status = MPI.Status()
53          message = comm.recv(source=MANAGER, tag=MPI.ANY_TAG, status=status
                )
54
55          if status.Get_tag() == TAG_DONE:
56              break
57
58          task_id, task = message
59          task.do_work()
60
61          comm.send((task_id, task), dest=MANAGER, tag=TAG_TASK_DONE)
62
63  def readcmdline(rank):
64      """
65      Read command line arguments
66
67      Parameters
68      ----------
69      rank : int
70          Rank of calling MPI process
71
72      Returns
73      -------
74      nx : int
75          number of gridpoints in x-direction
76      ny : int
77          number of gridpoints in y-direction
78      ntasks : int
79          number of tasks
80      """
81      # report usage
82      if len(sys.argv) != 4:
83          if rank == MANAGER:
84              print("Usage: manager_worker nx ny ntasks")
85              print("  nx     number of gridpoints in x-direction")
86              print("  ny     number of gridpoints in y-direction")
87              print("  ntasks number of tasks")
88          sys.exit()
89
90      # read nx, ny, ntasks
91      nx = int(sys.argv[1])
92      if nx < 1:
93          sys.exit("nx must be a positive integer")
94      ny = int(sys.argv[2])
95      if ny < 1:
96          sys.exit("ny must be a positive integer")
97      ntasks = int(sys.argv[3])
98      if ntasks < 1:
99          sys.exit("ntasks must be a positive integer")
100
101     return nx, ny, ntasks
```

```
102
103
104 if __name__ == "__main__":
105
106     # get COMMON WORLD communicator, size & rank
107     comm    = MPI.COMM_WORLD
108     size    = comm.Get_size()
109     my_rank = comm.Get_rank()
110
111     # report on MPI environment
112     if my_rank == MANAGER:
113         print(f"MPI initialized with {size:5d} processes")
114
115     # read command line arguments
116     nx, ny, ntasks = readcmdline(my_rank)
117
118     # start timer
119     timespent = - time.perf_counter()
120
121     if my_rank == MANAGER:
122         x_min = -2.
123         x_max = +1.
124         y_min = -1.5
125         y_max = +1.5
126
127         M = mandelbrot(x_min, x_max, nx, y_min, y_max, ny, ntasks)
128         tasks = M.get_tasks()
129
130         if size == 1:
131             for task in tasks:
132                 task.do_work()
133             TasksDoneByWorker = {0: len(tasks)}
134         else:
135             TasksDoneByWorker = manager(comm, tasks)
136
137         m = M.combine_tasks(tasks)
138         plt.imshow(m.T, cmap="gray", extent=[x_min, x_max, y_min, y_max])
139         plt.savefig("mandelbrot.pdf")
140
141         timespent += time.perf_counter()
142
143         print(f"Run took {timespent:f} seconds")
144         for i in range(size):
145             print(f"Process {i:5d} has done {TasksDoneByWorker.get(i, 0)
                    :10d} tasks")
146         print("Done.")
147
148     else:
149         worker(comm)
```

This listing implements a dynamic manager-worker algorithm. The process with rank zero is assigned the role of manager, while all other MPI processes are workers. The manager is responsible for creating the Mandelbrot tasks, distributing them to the workers, collecting the completed tasks, and finally assembling the global image. The workers do not communicate with each other; they only receive tasks from the manager and send completed results back to it.

The constants `TAG_TASK`, `TAG_TASK_DONE`, and `TAG_DONE` define the message types used in the program. Tags are important because the same pair of processes can exchange different kinds of messages. A message with tag `TAG_TASK` contains a new task to compute, a message with tag `TAG_TASK_DONE` contains a completed task returned by a worker, and a message with tag `TAG_DONE` tells a worker that no more tasks are available and that it can terminate.

Inside the `manager` function, the variable `next_task` stores the index of the next unassigned task. The variable `active_workers` counts how many workers are currently working on a task. At the beginning, the manager sends one task to each worker, as long as tasks are available. If there are more workers than tasks, the extra workers immediately receive a `TAG_DONE` message. This initialization phase fills the available workers with work and starts the computation.
After the initial distribution, the manager enters a loop that continues while at least one worker is active. The manager receives completed tasks using `comm.recv(source=MPI.ANY_SOURCE, tag=TAG_TASK_DONE, status=status)`. The use of `MPI.ANY_SOURCE` is essential for dynamic scheduling: the manager does not wait for a particular worker, but accepts the result from whichever worker finishes first. This allows faster workers or workers assigned easier Mandelbrot patches to immediately receive more work, improving load balance.
The received message contains both the `task_id` and the completed task object. The task identifier is needed because tasks may return in a different order from the one in which they were sent. The manager therefore uses `tasks[task_id] = completed_task` to put the result back into the correct position in the task list. This guarantees that the final call to `combine_tasks` can reconstruct the Mandelbrot image correctly.

Once a worker returns a result, the manager checks whether more tasks are available. If `next_task < len(tasks)`, the same worker immediately receives a new task. Otherwise, the worker receives a termination message with tag `TAG_DONE`, and the number of active workers is decreased. When all workers have received `TAG_DONE`, `active_workers` becomes zero and the manager loop terminates.

The `worker` function is a simple infinite loop. Each worker waits for a message from the manager using `comm.recv`. The tag of the received message is inspected through the `MPI.Status` object. If the tag is `TAG_DONE`, the worker exits the loop and terminates. Otherwise, the message contains a Mandelbrot task. The worker calls `task.do_work()`, which computes the Mandelbrot patch locally, and then sends the completed task back to the manager with tag `TAG_TASK_DONE`.

In the main part of the program, all MPI processes first obtain the communicator, the total number of processes, and their rank. The manager prints the number of MPI processes and creates the global Mandelbrot object. The call `M.get_tasks()` decomposes the global image into independent patches. If the program is run with only one MPI process, there are no workers. In this special case, the manager computes all tasks sequentially by directly calling `task.do_work()` for every task. This makes the code valid also for `size = 1` and provides a useful serial reference.
For `size > 1`, the manager calls the `manager` function to perform the dynamic parallel scheduling. After all tasks have been completed, `M.combine_tasks(tasks)` assembles the full Mandelbrot image from the computed patches.

Listing 16: Output of the manager-worker pattern for computing the Mandelbrot set in parallel

```
1 ripolo@eu-login-14:~/hpc_projects/project05/02_hpc_python/03
     _ManagerWorker$ srun python3 manager_worker.py 4001 4001 100
2 MPI initialized with     4 processes
3 Run took 3.746471 seconds
4 Process     0 has done          0 tasks
5 Process     1 has done         33 tasks
6 Process     2 has done         34 tasks
7 Process     3 has done         33 tasks
8 Done.
```

The output shows a run with four MPI processes. Rank zero acts as the manager, while ranks one, two, and three are workers. The Mandelbrot domain has resolution `4001 x 4001` and is split into 100 tasks. Since the manager only schedules work and assembles the final result, process zero performs zero Mandelbrot tasks in this parallel run.
The tasks are distributed almost uniformly among the three workers: process 1 completes 33 tasks, process 2 completes 34 tasks, and process 3 completes 33 tasks. This indicates that the dynamic self-scheduling mechanism is working as intended. Workers request new work implicitly by returning completed tasks, and the manager assigns the next available task to whichever worker finishes first.

The near-equal number of completed tasks does not necessarily mean that all workers performed exactly the same amount of computational work, because different Mandelbrot patches may have different computational costs. Nevertheless, the dynamic scheduling strategy helps reduce idle time compared with a purely static

assignment. The total runtime printed by the manager includes the computation of the Mandelbrot patches, the communication of task objects between manager and workers, the collection of the completed patches, and the final assembly and output of the image.

2.3.2. Scaling study

We were asked to perform a scaling study using 2-32 workers for a 4001x4001 domain and split the workload into 50 and 100 tasks.
The scaling results for the manager-worker implementation of the Mandelbrot set are shown in figure 3.

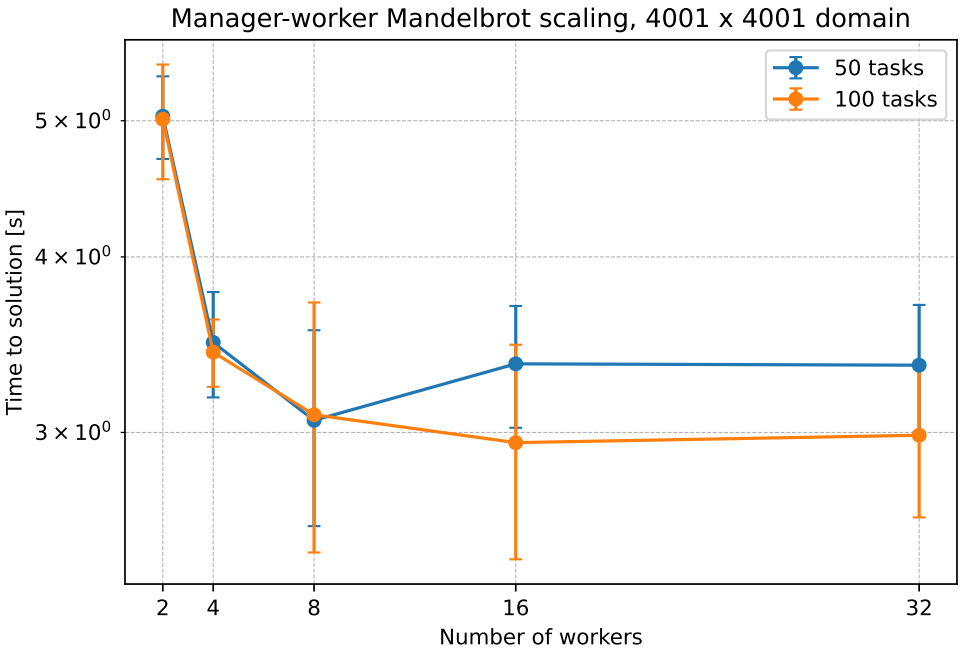


Figure 3: Scaling results for the manager-worker implementation of the Mandelbrot set.

Figure 3 shows the time to solution of the manager-worker Mandelbrot implementation for a fixed 4001×4001 domain, using 2, 4, 8, 16, and 32 workers. The workload is split into either 50 or 100 tasks.

The runtime decreases significantly when going from 2 to 4 workers. This shows that the manager-worker implementation is effective when the number of workers is still small: the Mandelbrot patches can be computed independently, and distributing them among more workers reduces the amount of work per worker. The improvement is also visible when going from 4 to 8 workers, although the gain is already smaller than in the previous step.

For larger worker counts, the scaling saturates. In particular, going from 8 to 16 and 32 workers does not lead to a clear additional reduction in runtime. For 50 tasks, the runtime even increases slightly after 8 workers. This indicates that the computation is no longer dominated only by the Mandelbrot calculation itself. The manager has to distribute tasks, receive completed patches, send termination messages, and finally assemble the global image. As the number of workers increases, the manager becomes a synchronization and communication bottleneck. Moreover, the number of tasks per worker becomes small, so the overhead of sending task objects and collecting results becomes comparable to the useful computation.

The comparison between 50 and 100 tasks shows the effect of task granularity. With 50 tasks, each task is larger, so the communication overhead per unit of computation is smaller. However, there are fewer tasks available for dynamic load balancing. This can limit scalability when many workers are used, because some workers may become idle while others are still working on expensive Mandelbrot patches. With 100 tasks, each task is smaller, so the manager has more flexibility to distribute work dynamically. This generally improves load balance, especially for higher worker counts, but it also increases the number of messages exchanged between the manager and the workers.

The results show that using 100 tasks gives slightly better performance at larger worker counts. In particular, for 16 and 32 workers the 100-task case is faster than the 50-task case. This suggests that, for this

problem size, the improved load balancing obtained with finer tasks compensates for the additional communication overhead. On the other hand, the difference between 50 and 100 tasks is not very large, which means that both decompositions are reasonably effective.

The error bars indicate the variability between repeated measurements. The variability is especially visible for larger worker counts. This is expected on a shared cluster system, where communication latency, node placement, and temporary load on the machine can affect the runtime. Since the measurements were performed with one process per node, the computation avoids intra-node resource contention, but it also makes the communication between manager and workers more expensive than a single-node run.

References

[1] Georg Hager and Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. 1st ed. CRC Press, 2010.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 6

Due date: **Friday** 22 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Graph Partitioning with Julia: Load balancing for HPC [50 points]

1.1. Graph Partitioning Algorithms

1.1.1. Inertial Bisection algorithm

Listing 1 reports the implementation of the inertial bisection algorithm. This method uses the geometrical information associated with the graph vertices, namely their coordinates, to split the graph into two subdomains. The basic idea is to interpret the vertices as a set of points in the plane and to compute the direction along which the point cloud has the largest spatial spread. The graph is then partitioned by projecting the vertices onto this direction and separating them according to their position with respect to a cutting line.

Listing 1: Inertial Bisection algorithm

```
function inertial_part(A, coords)

    p = ones(Int, size(coords, 1))

    xbar = mean(coords[:, 1])
    ybar = mean(coords[:, 2])

    Sxx = sum((coords[:, 1] .- xbar) .^ 2)
    Syy = sum((coords[:, 2] .- ybar) .^ 2)
    Sxy = sum((coords[:, 1] .- xbar) .* (coords[:, 2] .- ybar))

    M = Symmetric([Syy -Sxy;
                   -Sxy Sxx])

    eig = eigen(M)
    eigv = eig.vectors[:, argmin(eig.values)]

    p1, p2 = partition(coords, eigv)

    p[p1] .= 1
    p[p2] .= 2

    return p
end
```

The function first initializes the partition vector `p`, assigning all vertices to the same subdomain. The number of entries of `p` is equal to the number of rows of `coords`, hence to the number of vertices of the graph.
The quantities `xbar` and `ybar` represent the coordinates of the center of mass of the vertices. The coordinates are then shifted with respect to this center of mass in order to compute the second-order moments `Sxx`, `Syy`, and `Sxy`. These quantities measure the spatial distribution of the vertices around the center of mass: `Sxx` and `Syy` quantify the spread along the coordinate directions, while `Sxy` measures the correlation between the two coordinates.

Using these moments, the matrix

$$M = \begin{pmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{pmatrix}$$

is assembled as a symmetric matrix. The eigenvector associated with the smallest eigenvalue of `M` minimizes the sum of squared distances of the vertices from the corresponding line. Equivalently, the resulting line follows the direction of largest spatial spread of the point cloud. Since `M` is only a 2×2 dense symmetric matrix, the standard dense routine `eigen` is sufficient and avoids unnecessary use of ARPACK on such a small problem. The selected eigenvector is stored in `eigv`.
The actual split is then performed by the auxiliary routine `partition(coords, eigv)`. This routine projects the vertex coordinates onto the inertial direction and separates the vertices into two sets, denoted by `p1` and `p2`. These two sets correspond to the two sides of the bisecting line. Finally, the partition vector is updated by assigning label `1` to all vertices in `p1` and label `2` to all vertices in `p2`. The returned vector `p` therefore contains the bisection of the graph induced by the inertial direction.

1.1.2. Spectral Bisection Algorithm

In Listing 2, we report the implementation of the spectral bisection algorithm. Unlike inertial bisection, this method does not use the geometrical coordinates of the vertices. Instead, it exploits the connectivity structure of the graph through the graph Laplacian matrix. The main idea is to compute an eigenvector of the Laplacian, known as the Fiedler vector, and use its components to divide the vertices into two groups.

Listing 2: Spectral Bisection algorithm

```
function spectral_part(A)

    n = size(A, 1)

    if n == 1
        return ones(Int, 1)
    elseif n == 2
        return [1, 2]
    end

    # 1. Construct the sparse graph Laplacian.
    d = vec(sum(A, dims=2))
    L = spdiagm(0 => d) - A

    # 2. Compute the Fiedler vector.
    if n <= 4
        eig = eigen(Symmetric(Matrix(L)))
        perm = sortperm(eig.values)
        fiedler_vec = eig.vectors[:, perm[2]]
    else
        eigvals, eigvecs = eigs(L; nev=2, which=:SM)
        perm = sortperm(real(eigvals))
        fiedler_vec = real.(eigvecs[:, perm[2]])
    end

    # 3. Balanced partition by sorting the Fiedler vector.
    order = sortperm(fiedler_vec)

    p = ones(Int, n)

    n1 = div(n, 2)
    p[order[1:n1]] .= 1
    p[order[n1+1:end]] .= 2

    return p

end
```

The function takes as input the adjacency matrix `A` of the graph and first extracts the number of vertices `n`. A special case is included for graphs with a single vertex: in this situation no meaningful bisection can be performed, and the function simply returns a partition vector containing one label.

The first step of the algorithm consists in constructing the graph Laplacian. The vector `d` contains the degrees of the vertices, computed by summing the rows of the adjacency matrix. The Laplacian is then assembled in sparse form as

$$L = D - A.$$

Keeping `L` sparse is important because the adjacency matrices of mesh graphs are sparse, and forming a dense matrix would destroy this structure unnecessarily.

The spectral information of the graph is then obtained from the eigenvectors of `L`. For the general case, the implementation uses `eigs` from ARPACK to compute only the two eigenpairs of smallest magnitude. This is sufficient because the first eigenvector of the Laplacian corresponds to the constant mode, while the second eigenvector is the Fiedler vector used for the bisection. A small dense fallback is used only for very

small subgraphs, where ARPACK is not appropriate.

In the theoretical version of spectral bisection, the split can be obtained by comparing the components of the Fiedler vector with a threshold, often chosen as zero or as the median value. In this implementation, a balanced partition is enforced explicitly. The entries of the Fiedler vector are sorted, and the first `div(n, 2)` vertices in this ordering are assigned to the first subdomain, while the remaining vertices are assigned to the second subdomain. This guarantees that the two parts have approximately the same number of vertices, which is desirable in recursive graph partitioning because it avoids strongly unbalanced subdomains.

Finally, the partition vector `p` is returned. Its entries contain the labels of the two subdomains: vertices assigned label `1` belong to the first part, while vertices assigned label `2` belong to the second part. In disconnected or nearly disconnected subgraphs, the Fiedler vector may not be unique; therefore, the exact shape of the spectral partition can be sensitive to eigenvalue degeneracies. This is one reason why the partitions are inspected together with their edge-cuts and part sizes.

1.1.3. Results

The script `main_bench.jl` was used to run the four bisection strategies on the toy meshes loaded by `bench_bisection.jl`. For each mesh and each partitioning strategy, we measured the bisection edge-cut, i.e. the number of graph edges whose endpoints belong to different parts. Since the partitions are constrained to be balanced, the edge-cut provides a simple measure of the quality of the interface between the two subdomains: lower values correspond to fewer inter-partition connections and therefore to less communication in a parallel graph-distributed computation.

Mesh	Coordinate	Inertial	Spectral	METIS
`3elt`	172	209	117	90
`airfoil1`	94	94	132	73
`barth4`	206	194	127	100
`crack`	353	377	233	200
`mesh1e1`	18	19	18	17
`mesh2e1`	37	47	35	34
`mesh3e1`	19	24	22	18
`netz4504_dual`	25	30	23	20
`stufe`	16	16	16	17
`ukerbe1`	32	28	28	30

Table 1: Bisection edge-cut obtained on the toy meshes loaded by `bench_bisection.jl`.

From Table 1, METIS gives the lowest edge-cut for most of the meshes. This is particularly clear for the largest and most irregular examples: for `3elt`, the edge-cut is reduced from 172 with coordinate bisection and 209 with inertial bisection to 90 with METIS; for `barth4`, METIS reduces the edge-cut to 100, compared with 206, 194, and 127 for coordinate, inertial, and spectral bisection, respectively; and for `crack`, METIS gives 200, compared with 353, 377, and 233. These results confirm that a graph-based multilevel method is generally more effective at finding separators with small communication interfaces.

However, the table also shows that the best method is not always the same for every mesh. For `stufe`, coordinate, inertial, and spectral bisection all give the same edge-cut 16, slightly better than METIS, which gives 17. For `ukerbe1`, inertial and spectral bisection give the best value 28, while METIS gives 30. Moreover, spectral bisection is not uniformly better than the geometrical methods: on `airfoil1`, it gives an edge-cut of 132, which is worse than both coordinate and inertial bisection. Therefore, although graph-based methods are usually preferable, the final quality still depends on the specific mesh geometry, the refinement pattern, and the connectivity structure.

The corresponding partitions are shown in Figures 1–4 for the first four meshes, in Figures 5–9 for the additional structured meshes, and in Figure 10 for the `ukerbe1` mesh. The coordinate and inertial methods are purely geometrical: they only use the position of the vertices and therefore tend to produce almost straight cuts. This behaviour is clearly visible in the `3elt`, `barth4`, and `crack` examples. Such partitions are simple and often reasonable when the mesh is approximately uniform and the graph connectivity follows the geometry closely. However, they can become sub-optimal when the mesh has non-uniform density or when the geometry contains local features that should not be cut through.

The spectral and METIS partitions are instead based on the graph connectivity. As a consequence, they are generally more flexible: the interface is not restricted to be a straight geometrical line, but can follow weaker connections in the graph. This is particularly important for meshes such as `airfoil1`, where the local refinement around the airfoil and its wake makes a purely geometrical split less adapted to the graph structure. In this case, the coordinate and inertial cuts separate the mesh in a rather direct left-right fashion, while spectral bisection and METIS produce more irregular interfaces that better reflect the connectivity of the mesh.

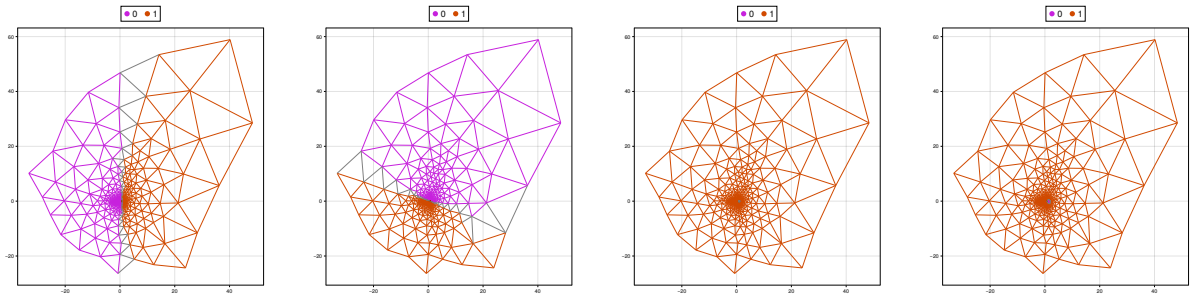
Some partitions may appear visually surprising. In particular, for meshes such as `barth4`, `3elt`, and `ukerbe1`, the spectral and METIS partitions sometimes contain one part that occupies a relatively small physical region of the plotted domain. This should not immediately be interpreted as poor load balance. The balance constraint is imposed on the number of graph vertices, not on the physical area covered by each part. Therefore, in highly refined regions, a geometrically small subdomain may still contain a large number of vertices. For this reason, the visual inspection of the partitions should always be complemented by the edge-cut values and, when needed, by the number of vertices assigned to each part. This distinction is particularly important for non-uniform meshes, where physical area and graph size are not proportional.

The additional structured meshes further illustrate the difference between geometrical and connectivity-based partitioning. For the regular meshes `mesh1e1`, `mesh2e1`, and `mesh3e1`, all methods produce relatively clean partitions, but the shape of the interface depends on the criterion used by the algorithm. Coordinate bisection typically produces a vertical or horizontal cut aligned with the coordinate axes, while inertial bisection may rotate the cut according to the dominant spatial direction of the mesh. Spectral bisection and METIS may instead introduce slightly irregular interfaces when this reduces the number of cut edges.

The `netz4504_dual` and `stufe` meshes are more interesting because their geometry is less uniform. In these cases, a purely geometrical cut does not necessarily correspond to the smallest communication interface. The graph-based methods are able to adapt the separator to the local connectivity of the mesh, especially around refined or stepped regions. This is the expected behaviour: the objective of graph partitioning is not to produce a visually straight cut, but to minimize the number of edges crossing between different parts while keeping the load balanced.

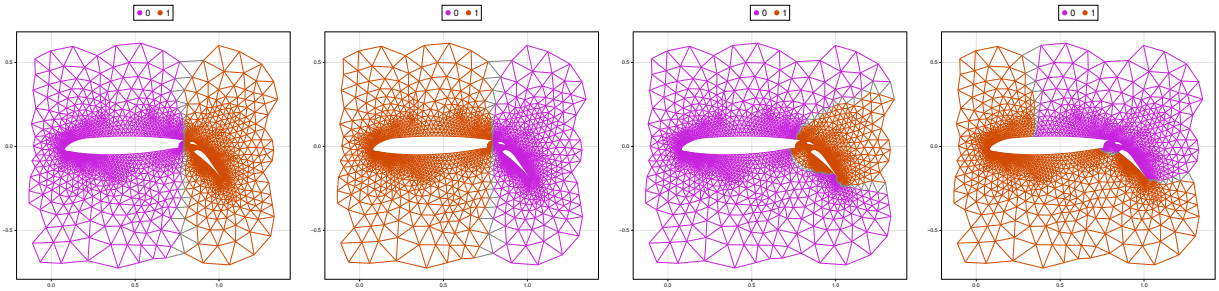
Finally, the `ukerbe1` mesh shows a geometry with a pronounced notch-like feature and several internal circular holes. In this case, the coordinate and inertial methods still produce mostly straight cuts, separating the mesh approximately into upper and lower parts. This is reasonable from the point of view of the global geometry, but it does not explicitly account for the connectivity around the notch and the locally refined region. The spectral and METIS partitions instead place the separator close to this narrow central region. This behaviour is consistent with the graph-based objective, since cutting through a geometrical bottleneck can reduce the number of edges crossing between the two parts. Although some of these graph-based partitions may occupy a small physical area, this is not necessarily a sign of poor balance, because the local mesh density near the notch is much higher than in the coarser parts of the domain.

Overall, the results confirm the expected trade-off between geometrical and graph-based partitioning. Coordinate and inertial bisection are simple and produce regular cuts, but they may ignore important connectivity features and can therefore lead to large edge-cuts on non-uniform meshes. Spectral bisection uses the graph Laplacian and can adapt better to the connectivity, but its performance is not uniformly superior, as shown for example by the `airfoil1` case. METIS gives the lowest edge-cut in most examples, especially for the most irregular meshes, because it is specifically designed to optimize graph separators. Nevertheless, the exceptions `stufe` and `ukerbe1` show that visual inspection and numerical edge-cut values should both be considered when evaluating the quality of a partition.



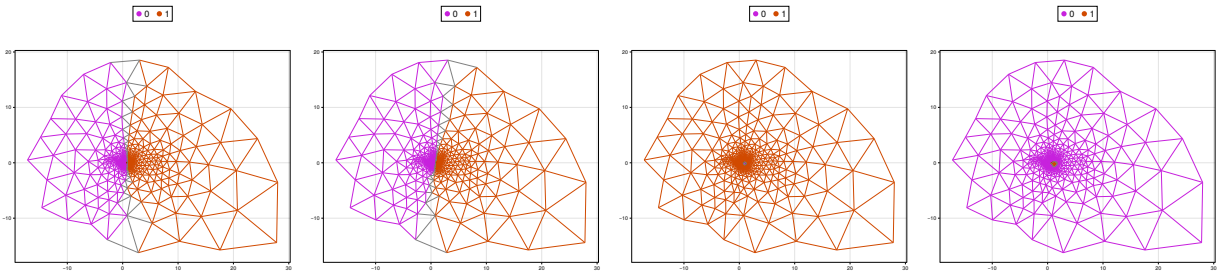
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 1: Bisection partitions obtained for the `3elt` mesh.



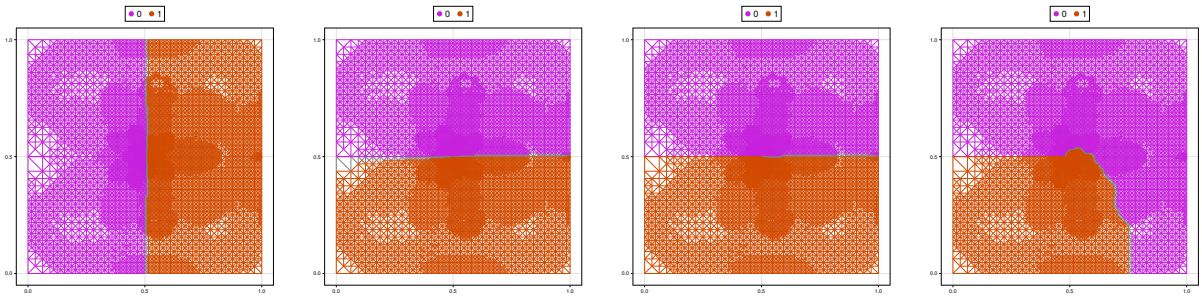
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 2: Bisection partitions obtained for the `airfoil1` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 3: Bisection partitions obtained for the `barth4` mesh.



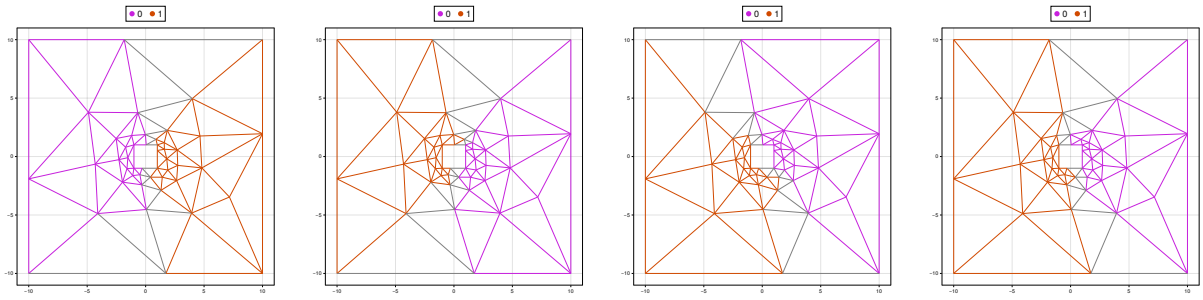
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 4: Bisection partitions obtained for the `crack` mesh.

1.2. Recursive Bisection Algorithm

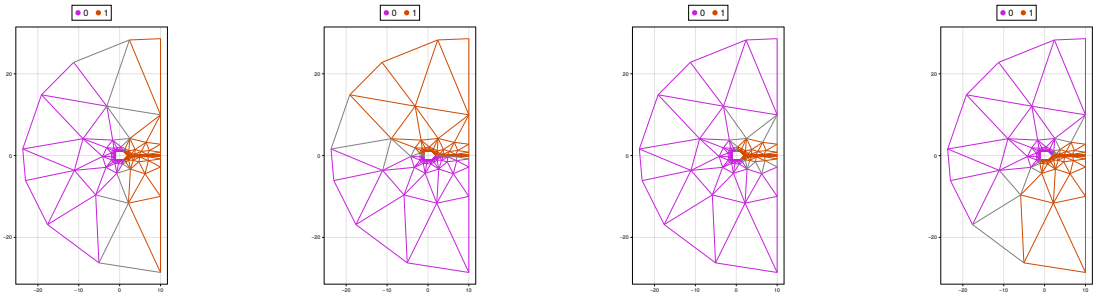
1.2.1. Implementation

Listing 3: Recursive Bisection algorithm



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 5: Bisection partitions obtained for the `mesh1e1` mesh.



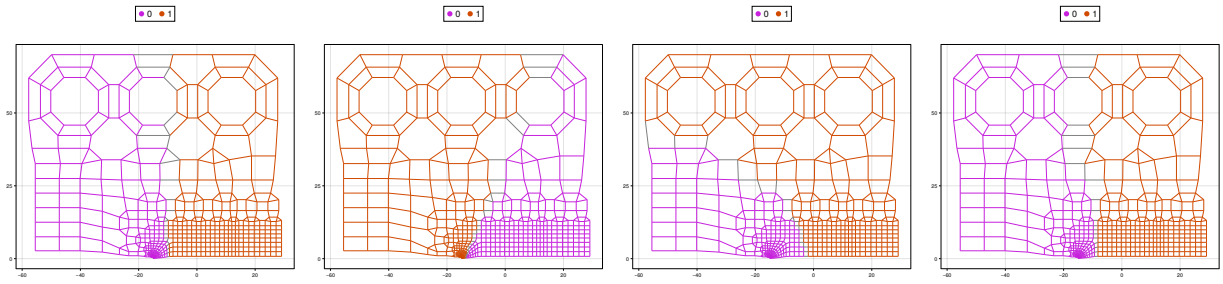
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 6: Bisection partitions obtained for the `mesh2e1` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 7: Bisection partitions obtained for the `mesh3e1` mesh.



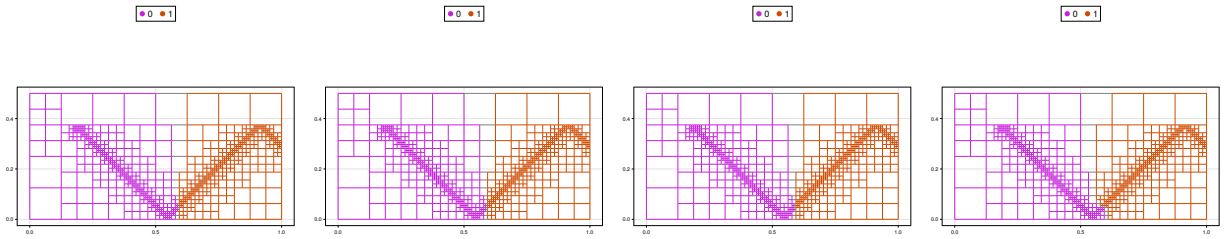
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 8: Bisection partitions obtained for the `netz4504_dual` mesh.

```
function rec_bisection(method, levels, A, coords=zeros(0), vn=zeros(0))

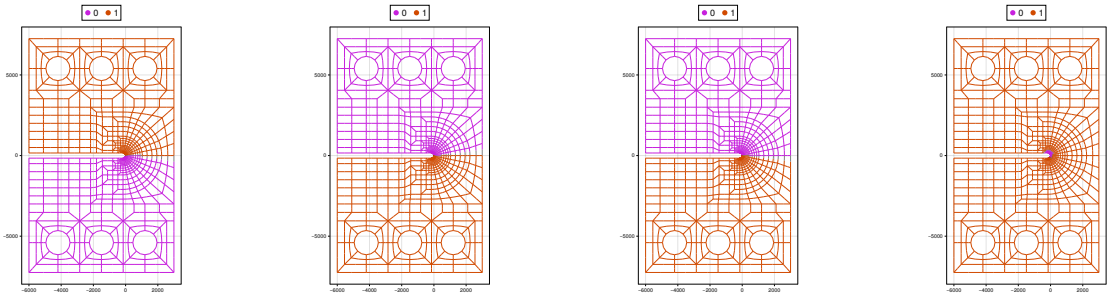
    n = size(A, 1)

    # Final partition vector.
```



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 9: Bisection partitions obtained for the `stufe` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 10: Bisection partitions obtained for the `ukerbe1` mesh.

```
    # p[i] is the final part assigned to vertex i.
    p = zeros(Int, n)

    function recursion(vertices, level, index)

        # Stop if no more bisections are required.
        if level == 0
            p[vertices] .= index
            return
        end

        # A subgraph with fewer vertices than requested final parts
        # cannot be split into 2^level non-empty parts.
        if length(vertices) < 2^level
            error("Recursive bisection failed: not enough vertices for the
                requested number of parts.")
        end

        # Extract induced subgraph.
        A_sub = A[vertices, vertices]

        # Apply chosen bisection method.
        if method == "spectral_part"
            local_p = spectral_part(A_sub)

        elseif method == "coordinate_part"
            if length(coords) == 0
                error("Coordinates are required for coordinate bisection
                    .")
            end
            coords_sub = coords[vertices, :]
            local_p = coordinate_part(A_sub, coords_sub)
```

```
36
37          elseif method == "inertial_part"
38              if length(coords) == 0
39                  error("Coordinates are required for inertial bisection.")
40              end
41              coords_sub = coords[vertices, :]
42              local_p = inertial_part(A_sub, coords_sub)
43
44          else
45              error("Unknown partitioning method: $method")
46          end
47
48          # Extract the two labels returned by the bisection method.
49          labels = sort(unique(local_p))
50
51          if length(labels) != 2
52              error("Bisection failed: expected 2 parts, got labels =
                    $labels")
53          end
54
55          vertices_1 = vertices[findall(local_p .== labels[1])]
56          vertices_2 = vertices[findall(local_p .== labels[2])]
57
58          if isempty(vertices_1) || isempty(vertices_2)
59              error("Bisection failed: one child partition is empty.")
60          end
61
62          # Each child branch contains 2^(level-1) final labels.
63          offset = 2^(level - 1)
64
65          recursion(vertices_1, level - 1, index)
66          recursion(vertices_2, level - 1, index + offset)
67
68      end
69
70      recursion(collect(1:n), levels, 1)
71
72      expected_parts = 2^levels
73      actual_parts = sort(unique(p))
74
75      if length(actual_parts) != expected_parts
76          error("Expected $expected_parts parts, got $(length(actual_parts))
                : $actual_parts")
77      end
78
79      return p
80
81  end
```

The recursive bisection routine in Listing 3 generalizes the previous two-way partitioning methods to a $p = 2^{\ell}$-way partition. Instead of computing all parts at once, the graph is repeatedly split into two subgraphs. After the first bisection, each of the two parts is bisected again, and this process is repeated until the prescribed number of recursion levels has been reached. Therefore, after ℓ levels, the algorithm produces 2^{ℓ} final subdomains.

The function takes as input the name of the bisection method, the number of recursive levels, the adjacency matrix `A`, and, when needed, the vertex coordinates `coords`. The vector `p` stores the final partition labels. Initially all entries are set to zero. The actual recursive work is performed by the inner function `recursion(vertices, level, index)`, where `vertices` contains the indices of the current subgraph, `level` is the number of remaining bisections, and `index` is the first label assigned to the current branch of the recursion tree.

The stopping criterion is reached when `level == 0`. In this case, no further bisection is required, and all vertices of the current subgraph are assigned the current final label `index`. Before performing a split, the routine also checks that the current subgraph contains at least $2^{\texttt{level}}$ vertices. Otherwise, it would be impossible to split it into the requested number of non-empty final parts. In such a case the routine raises an error, rather than silently returning a partition with fewer than 2^ℓ parts.

If further splitting is required, the induced subgraph is extracted as `A_sub = A[vertices, vertices]`. The chosen bisection method is then applied to this subgraph. For `spectral_part`, only the adjacency matrix is needed. For `coordinate_part` and `inertial_part`, the corresponding coordinates of the selected vertices are also extracted as `coords_sub`. The routine therefore uses the same local numbering inside the bisection method, but maps the result back to the original vertex indices through the array `vertices`.

After the local bisection has been computed, the algorithm extracts the two labels returned by the bisection method. The implementation explicitly checks that exactly two non-empty child partitions have been produced. If this is not the case, an error is raised. This behaviour is preferable during benchmarking, because silently assigning the whole subgraph to a single label would produce fewer than the requested 2^ℓ final parts and would make the edge-cut values misleading.

The label offset is computed as

$$\texttt{offset} = 2^{\texttt{level}-1}.$$

This is necessary because each child branch will generate $2^{\texttt{level}-1}$ final labels. The first child therefore keeps the current starting label `index`, while the second child starts at `index + offset`. For example, with $\ell = 2$, the first bisection creates two branches: the first branch eventually receives labels 1 and 2, while the second branch receives labels 3 and 4. This indexing convention guarantees that the final partition labels are consecutive and consistent with the recursion tree.

Finally, the recursion is initialized on the full vertex set `collect(1:n)` with the prescribed number of levels and initial label 1. After the recursion terminates, the implementation verifies that exactly 2^ℓ different labels have been produced. The returned vector `p` contains the final 2^ℓ-way partition of the graph.

1.2.2. Results

The recursive bisection procedure was tested by generating $p = 8$ and $p = 16$ partitions. For the recursive algorithms, this corresponds respectively to three and four levels of recursive bisection. Listing 3 shows the recursive driver used for the coordinate, inertial, and spectral bisection methods implemented above. The recursive METIS partition and the direct METIS k-way partition are instead computed using the METIS routines provided in the skeleton code. The comparison is useful because recursive bisection constructs the final partition through a sequence of two-way splits, while the k-way METIS routine computes the p-way partition directly.

As in the two-way case, the quality of the partition can be assessed through the total edge-cut, i.e. the number of graph edges connecting vertices assigned to different parts. For recursive partitioning, this quantity is especially important because each bisection step creates an interface, and the final edge-cut is the cumulative result of all these recursive splits.

The corresponding partitions are shown in Figures 11–18.

The numerical values in Table 2 show that the edge-cut increases when going from $p = 8$ to $p = 16$. This is expected, because increasing the number of parts creates more interfaces between subdomains. The increase is particularly large for the irregular and highly refined meshes, such as `3elt`, `barth4`, and `crack`. For example, for `crack`, the METIS k-way edge-cut increases from 776 for $p = 8$ to 1186 for $p = 16$, while the coordinate edge-cut increases from 1344 to 1861.

The table also confirms that the graph-based methods generally outperform the purely geometrical recursive methods. In most cases, METIS recursive and METIS k-way give the lowest edge-cuts, followed by spectral bisection. The coordinate and inertial methods are often worse, especially for non-uniform meshes, because they do not directly use the graph connectivity when choosing the separator. This is particularly evident for `3elt`, where for $p = 16$ the inertial method gives an edge-cut of 1342, while METIS k-way gives 636.

However, METIS k-way is not always strictly better than recursive METIS. For `barth4` with $p = 8$, recursive METIS gives 409, slightly better than METIS k-way with 419. Similarly, for `crack` with $p = 8$, recursive METIS gives 736, while METIS k-way gives 776. Nevertheless, for $p = 16$, METIS k-way usually becomes competitive or better, because it can optimize the final p-way partition globally instead of committing to a

Mesh	Parts	Coordinate	Inertial	Spectral	METIS rec.	METIS k-way
3elt	$p = 8$	733	880	469	402	383
	$p = 16$	1168	1342	752	645	636
airfoil1	$p = 8$	516	577	398	324	321
	$p = 16$	819	902	631	566	545
barth4	$p = 8$	875	888	549	409	419
	$p = 16$	1306	1349	835	716	683
crack	$p = 8$	1344	1061	883	736	776
	$p = 16$	1861	1618	1419	1238	1186
mesh3e1	$p = 8$	75	75	76	74	71
	$p = 16$	118	117	123	112	217
netz4504_dual	$p = 8$	127	122	113	101	99
	$p = 16$	198	202	182	159	160
stufe	$p = 8$	123	135	129	110	113
	$p = 16$	228	269	248	196	193
ukerbe1	$p = 8$	225	280	127	140	119
	$p = 16$	374	467	237	237	230

Table 2: Recursive partitioning edge-cuts for the considered meshes.

sequence of two-way decisions.

In the plots, small physical subdomains should not automatically be interpreted as unbalanced partitions. The load-balancing criterion is based on the number of graph vertices assigned to each part, whereas the figures show the physical geometry of the mesh. For highly non-uniform meshes, a small geometrical region can contain many vertices because of local refinement. Therefore, the edge-cut values in Table 2 should be interpreted together with the vertex counts of the parts when assessing the quality of the partition.

The first qualitative difference is visible when comparing the recursive geometrical methods with the graph-based methods. Coordinate and inertial recursive bisection tend to preserve the structure of the first geometrical cuts: the final subdomains are mostly obtained by repeatedly splitting the mesh along approximately straight directions. This gives partitions that are easy to interpret geometrically, but it may also create long interfaces when the mesh density is non-uniform.

For the `3elt` mesh, the recursive coordinate and inertial methods generate partitions that follow the global shape of the point cloud. For $p = 16$, several subdomains occupy small physical regions close to the highly refined central part of the mesh. This is expected for a non-uniform mesh: recursive bisection balances the number of graph vertices, not the physical area of the subdomains. The spectral and METIS recursive partitions are more strongly influenced by the graph connectivity, and therefore also allocate several labels close to the central refinement. These plots illustrate why geometrical area alone is not a reliable indicator of load balance for unstructured meshes; the visual inspection should be interpreted together with the part sizes and edge-cut values.

For the `airfoil1` mesh, the effect of the airfoil geometry and the refined wake region is more pronounced. The recursive coordinate and inertial partitions divide the domain into large geometric sectors. These partitions are regular and visually clear, but they do not necessarily follow the weakest graph connections. The spectral and METIS recursive partitions adapt more to the refined regions around the airfoil and the wake. In particular, several parts are placed around the leading edge, trailing edge, and wake, where the mesh is much denser. This behaviour is expected, since these regions contain many vertices and therefore require more subdomains if the partition is to remain balanced in terms of graph vertices rather than physical area.

For the `barth4` and `crack` meshes, the same qualitative behaviour can be observed, but the effect of the geometry is different. The `barth4` mesh has a strongly non-uniform vertex distribution, with many vertices concentrated around the central region. As a consequence, some recursive partitions appear to occupy a very small physical area near the centre of the mesh. This does not necessarily imply that the partition is unbalanced in terms of graph vertices, since the central region is much more densely discretized than the outer part of the domain. Nevertheless, these plots show a limitation of interpreting graph partitions only from their geometrical area.

The `crack` mesh is instead characterized by a localized internal feature around the centre. The recursive

coordinate and inertial methods generate partitions that remain mostly aligned with successive geometrical cuts. For $p = 8$ and $p = 16$, this creates large block-like regions away from the crack and much smaller regions around the refined centre. The spectral and METIS-based methods are more sensitive to the graph connectivity near the crack and tend to isolate the refined central region more strongly. This is expected, because the high local mesh density around the crack contains many graph vertices and therefore receives several subdomains when the partition is balanced by vertex count.

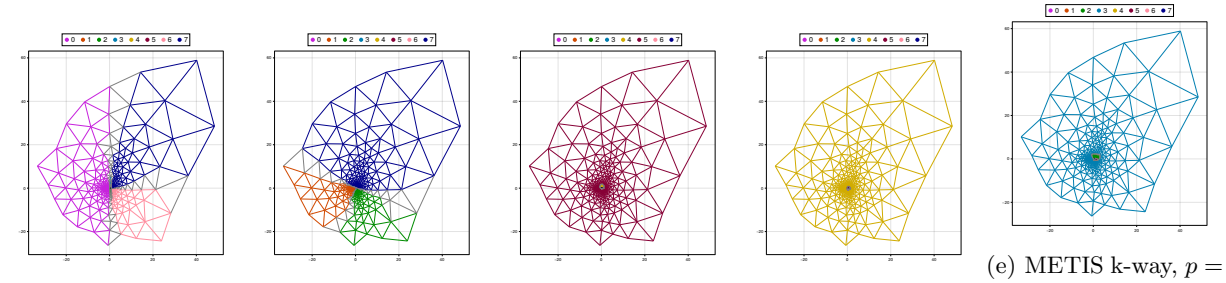
The `mesh3e1` mesh is useful as a reference case because it is much more regular than the previous unstructured examples. In this case, the recursive coordinate and inertial methods produce partitions that are mostly block-like and aligned with the geometry of the square domain. Since the mesh is close to structured, these geometrical cuts are already quite reasonable. The spectral and METIS-based methods also produce compact subdomains, but their interfaces are less constrained by coordinate directions and can adapt to the graph connectivity. For this mesh, the visual difference between the methods is less dramatic than for highly non-uniform meshes such as `airfoil1`, `barth4`, or `crack`.

The `netz4504_dual` mesh has a more complex geometry, with large coarse cells in the upper part and a much finer structured region near the bottom boundary. This makes the recursive partitioning behaviour particularly interesting. The coordinate and inertial methods tend to split the domain according to its global geometric extent, producing long vertical or horizontal regions. By contrast, the spectral and METIS partitions allocate several smaller parts close to the refined bottom region. This is consistent with the fact that graph partitioning balances the number of vertices rather than the physical area of the subdomains. Therefore, regions that are geometrically small but highly discretized may receive several partition labels.

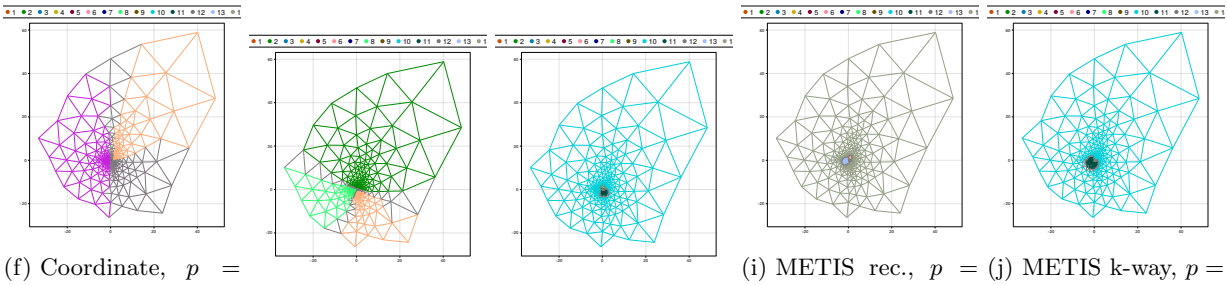
The `stufe` mesh contains a step-like geometry, with two refined diagonal regions and a relatively coarse outer rectangular structure. The recursive coordinate and inertial methods again produce partitions that are strongly driven by the global geometry. As a consequence, the parts tend to form vertical or horizontal blocks, with smaller subdomains appearing around the refined regions. The graph-based methods produce more irregular partitions, especially for $p = 16$, because the separator is allowed to follow the local mesh connectivity rather than a global coordinate direction. This gives a less visually regular decomposition, but it can reduce the number of edges crossing between parts.

The `ukerbe1` mesh is one of the most challenging examples, because it combines a large structured geometry, several circular holes, and a pronounced notch-like feature near the centre. For this mesh, the recursive coordinate and inertial methods split the domain mostly into large upper and lower or left and right regions, with additional refinement around the notch in later levels of the recursion. The spectral and METIS-based methods instead place several separators close to the central notch, which acts as a natural bottleneck of the graph. This is a reasonable behaviour for graph partitioning: cutting through a narrow, highly connected transition region can reduce the final edge-cut. However, as in the previous non-uniform meshes, some partitions appear to occupy a small physical area. This should not immediately be interpreted as poor load balance, since the local mesh density is much higher near the notch.

The comparison with METIS k-way partitioning is useful because it shows the difference between recursively applying a two-way partitioner and computing the final p-way partition directly. The k-way METIS partitions are generally more globally optimized: the algorithm can account for all p parts simultaneously, rather than committing to early bisection decisions that cannot be changed later. Therefore, one often expects METIS k-way partitioning to give lower edge-cuts than recursive partitioning, especially for larger p. The numerical results mostly support this trend, although not in every case. For example, recursive METIS is slightly better than METIS k-way for `barth4` and `crack` when $p = 8$. This shows that the advantage of k-way partitioning is not absolute, but it is generally competitive and becomes particularly relevant as the number of parts increases.
One notable exception is the mesh3e1 case with p=16, where METIS k-way gives a substantially larger edge-cut than recursive METIS and the other recursive methods. Since this mesh is rather regular, this behaviour is somewhat surprising and may be due to the particular objective, random choices, or balance constraints used internally by METIS. It also illustrates that the direct k-way formulation is not guaranteed to dominate recursive bisection for every small test case.

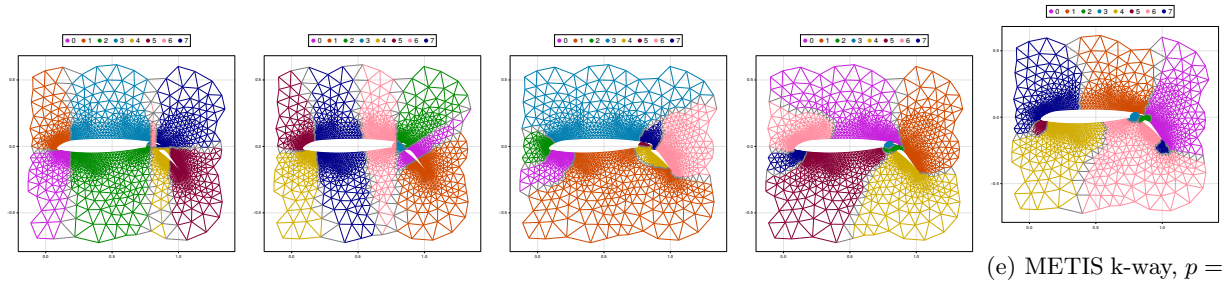


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

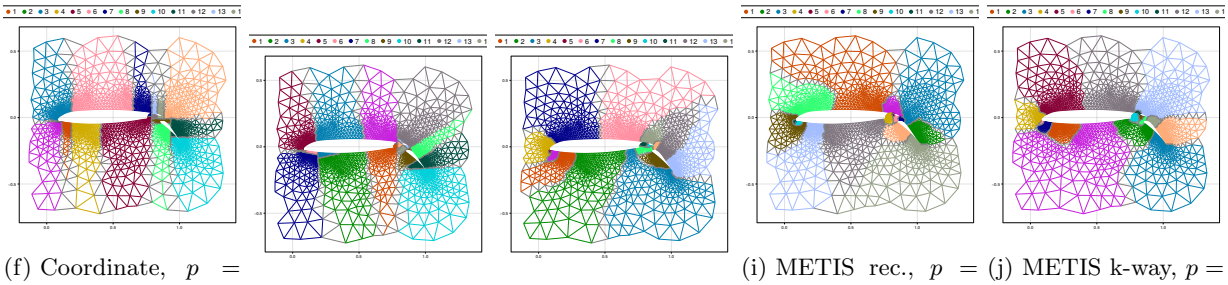


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 11: Recursive and k-way partitions obtained for the `3elt` mesh with $p = 8$ and $p = 16$ parts.

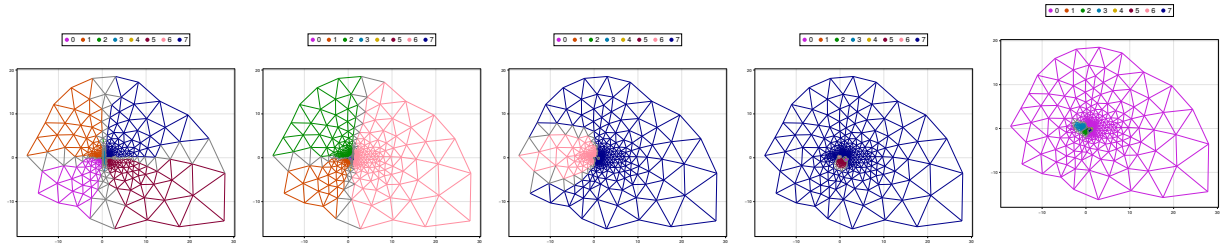


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

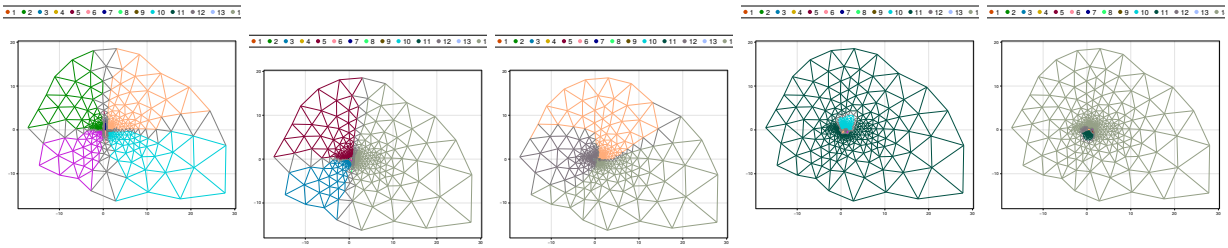


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 12: Recursive and k-way partitions obtained for the `airfoil1` mesh with $p = 8$ and $p = 16$ parts.

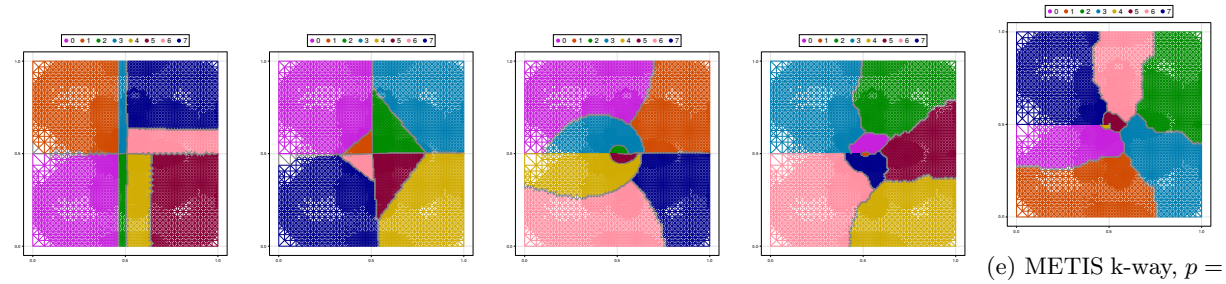


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

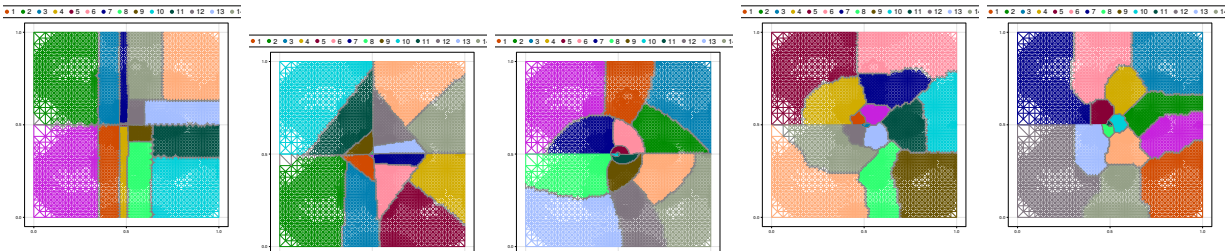


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 13: Recursive and k-way partitions obtained for the `barth4` mesh with $p = 8$ and $p = 16$ parts.

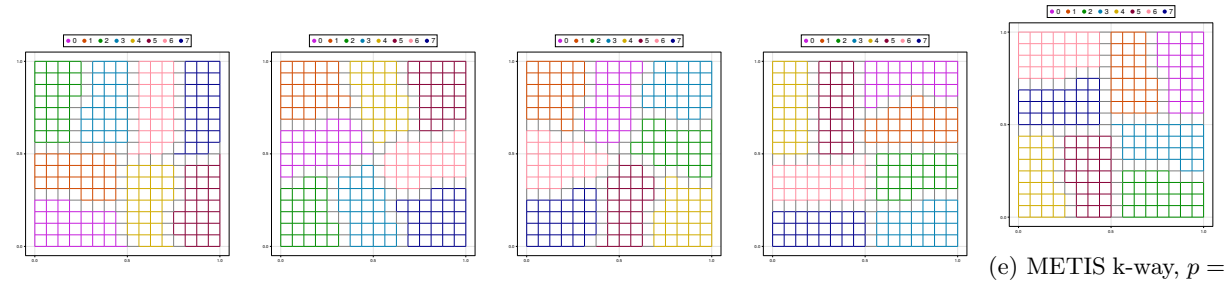


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

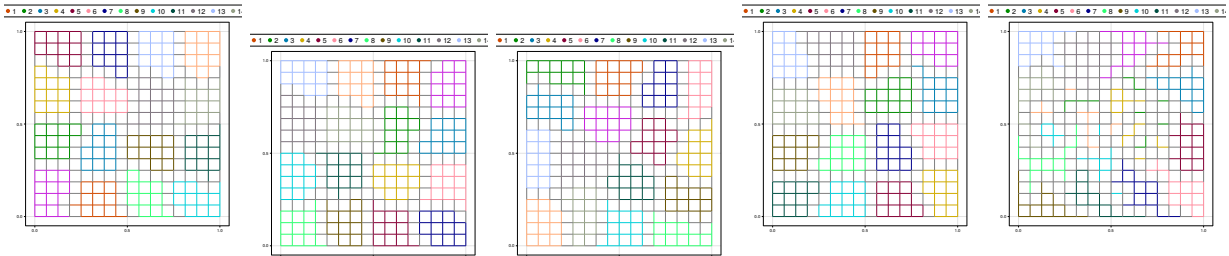


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 14: Recursive and k-way partitions obtained for the `crack` mesh with $p = 8$ and $p = 16$ parts.

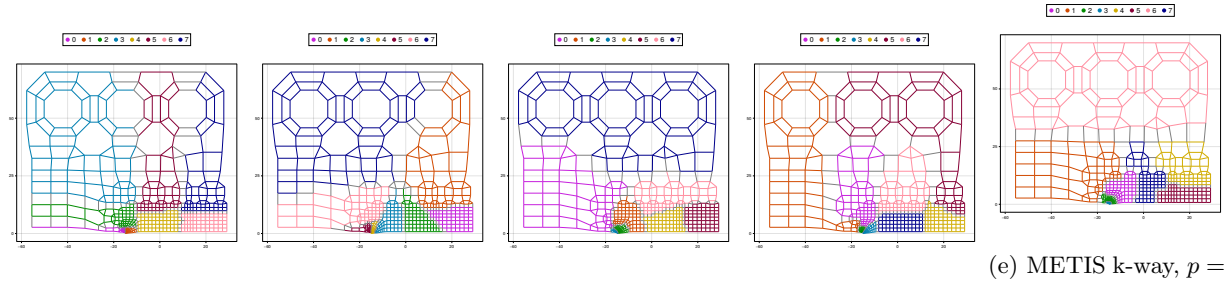


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

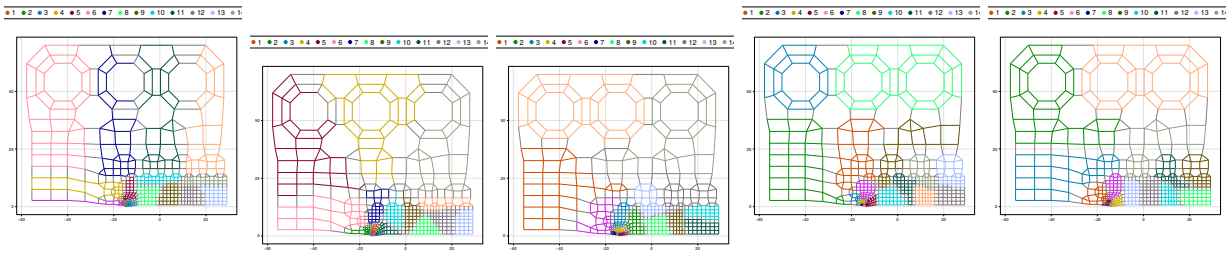


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 15: Recursive and k-way partitions obtained for the `mesh3e1` mesh with $p = 8$ and $p = 16$ parts.

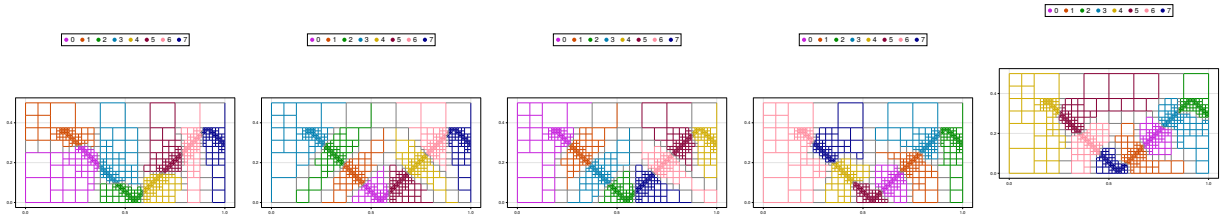


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

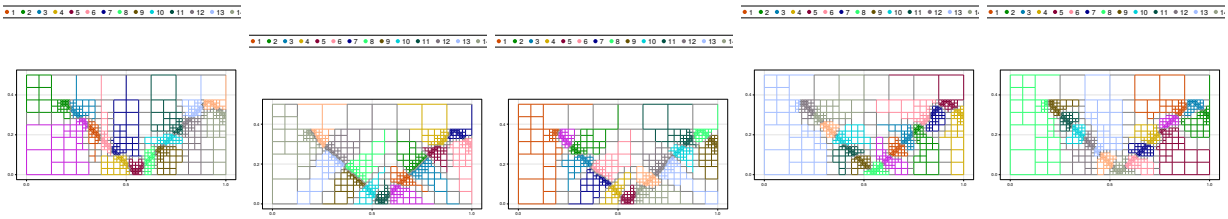


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 16: Recursive and k-way partitions obtained for the `netz4504_dual` mesh with $p = 8$ and $p = 16$ parts.

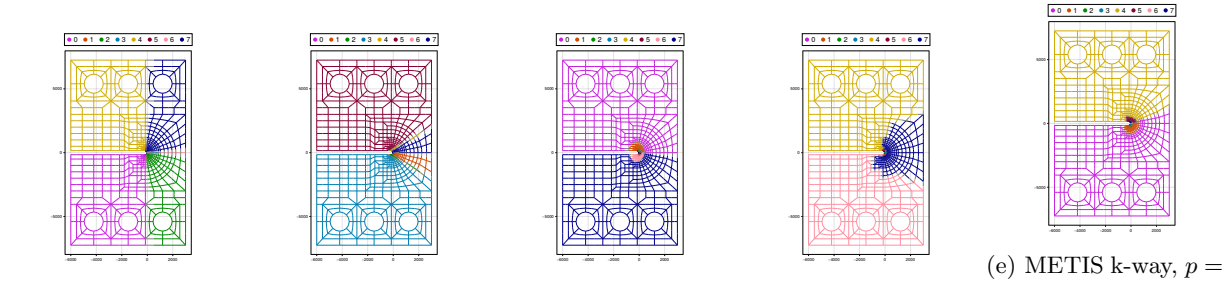


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

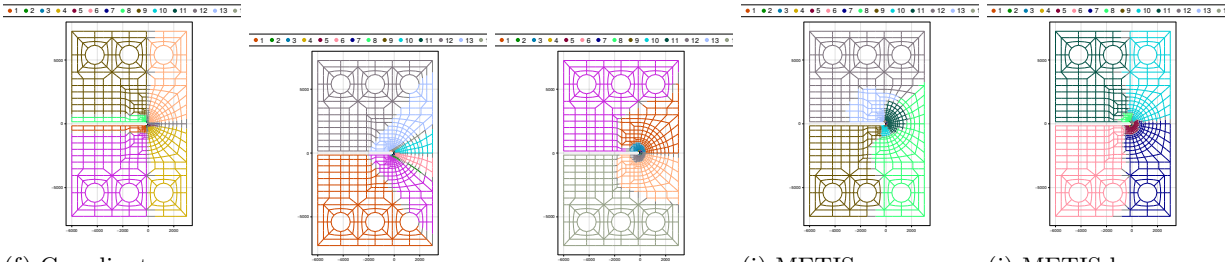


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 17: Recursive and k-way partitions obtained for the `stufe` mesh with $p = 8$ and $p = 16$ parts.



(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$



(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 18: Recursive and k-way partitions obtained for the `ukerbe1` mesh with $p = 8$ and $p = 16$ parts.

2. HPC software frameworks [50 points]

2.1. Mathematical Formulation

In this section we solve the two-dimensional Poisson problem with homogeneous Dirichlet boundary conditions,

$$\begin{aligned} -\Delta u &= f && \text{in } \Omega, \\ u &= 0 && \text{on } \partial\Omega, \end{aligned}$$

where the computational domain is the unit square

$$\Omega = (0,1)^2,$$

and the source term is constant,

$$f(x_1, x_2) = 20.$$

In Cartesian coordinates the Laplace operator is

$$\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2}.$$

The homogeneous Dirichlet condition imposes that the solution is zero on the whole boundary of the square. Therefore, the expected qualitative behaviour is a positive solution in the interior, symmetric with respect to the two midlines of the square, and attaining its maximum close to the centre of the domain.

The domain is discretized by a uniform Cartesian grid with m_x grid points in the x_1-direction and m_y grid points in the x_2-direction, including boundary points. The mesh widths are

$$h_x = \frac{1}{m_x - 1}, \qquad h_y = \frac{1}{m_y - 1}.$$

The grid points are

$$x_i = i h_x, \qquad i = 0, \ldots, m_x - 1,$$

and

$$y_j = j h_y, \qquad j = 0, \ldots, m_y - 1.$$

The numerical approximation of $u(x_i, y_j)$ is denoted by

$$u_{i,j} \approx u(x_i, y_j).$$

For an interior grid point, namely for

$$1 \le i \le m_x - 2, \qquad 1 \le j \le m_y - 2,$$

the second derivatives are approximated with second-order centred finite differences:

$$\frac{\partial^2 u}{\partial x_1^2}(x_i, y_j) = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} + \mathcal{O}(h_x^2),$$

and

$$\frac{\partial^2 u}{\partial x_2^2}(x_i, y_j) = \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} + \mathcal{O}(h_y^2).$$

Substituting these approximations into $-\Delta u = 20$ gives

$$-\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} - \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} = 20.$$

Equivalently,

$$\left(\frac{2}{h_x^2} + \frac{2}{h_y^2}\right) u_{i,j} - \frac{1}{h_x^2}\left(u_{i-1,j} + u_{i+1,j}\right) - \frac{1}{h_y^2}\left(u_{i,j-1} + u_{i,j+1}\right) = 20.$$

This is the standard five-point finite-difference stencil for the negative Laplacian.

In the implementation, the equation is multiplied by $h_x h_y$. This does not change the solution of the linear system, but leads to the coefficients used in the assembled PETSc matrix:

$$2\left(\frac{h_x}{h_y} + \frac{h_y}{h_x}\right) u_{i,j} - \frac{h_y}{h_x}\left(u_{i-1,j} + u_{i+1,j}\right) - \frac{h_x}{h_y}\left(u_{i,j-1} + u_{i,j+1}\right) = 20 h_x h_y. \tag{1}$$

Therefore, the matrix stencil used for an interior point is

$$\begin{matrix} & -h_x/h_y & \\ -h_y/h_x & 2(h_x/h_y + h_y/h_x) & -h_y/h_x \\ & -h_x/h_y & \end{matrix} \tag{2}$$

where the horizontal neighbours correspond to the coefficients $-h_y/h_x$, while the vertical neighbours correspond to the coefficients $-h_x/h_y$.

The boundary condition is imposed strongly. If a node lies on the boundary, that is if

$$i = 0, \quad i = m_x - 1, \quad j = 0, \quad \text{or} \quad j = m_y - 1,$$

then the discrete equation is replaced by

$$u_{i,j} = 0.$$

In matrix form, this corresponds to inserting an identity row for every boundary degree of freedom:

$$A_{kk} = 1, \qquad b_k = 0.$$

For interior degrees of freedom, the finite-difference equations define a sparse linear system

$$A\mathbf{u} = \mathbf{b},$$

where $\mathbf{u}$ contains the nodal values $u_{i,j}$. If the interior unknowns are ordered lexicographically, for example with index

$$k = i + m_x j,$$

then each row of A contains at most five nonzero entries, corresponding to the centre point and its four nearest neighbours. Since the operator is the negative Laplacian with homogeneous Dirichlet boundary conditions, the resulting matrix is symmetric positive definite on the constrained problem. This property motivates the use of Krylov methods for symmetric positive definite systems, such as the conjugate-gradient method.

2.2. PETSc Implementation

The Poisson problem was implemented in C using PETSc. The code uses the PETSc distributed array abstraction `DMDA`, which is particularly convenient for finite-difference discretizations on structured grids. The distributed array stores the two-dimensional Cartesian layout of the grid and distributes grid points among MPI processes. In this way, each process assembles only the rows of the matrix and the entries of the right-hand side vector that belong to its local portion of the grid.

The main PETSc objects used in the code are:

`DM`, `KSP`, `Mat`, `Vec`.

The `DM` object describes the distributed structured grid, while the `KSP` object manages the linear solver. The matrix and the right-hand side are assembled through callback functions registered with the `KSP` object.

Listing 4: Creation of the PETSc solver and distributed grid

```
1  PetscCall(PetscInitialize(&argc, &argv, NULL, help));
2
3  PetscCall(KSPCreate(PETSC_COMM_WORLD, &ksp));
4
5  PetscCall(DMDACreate2d(
6      PETSC_COMM_WORLD,
7      DM_BOUNDARY_NONE,
8      DM_BOUNDARY_NONE,
9      DMDA_STENCIL_STAR,
10     3, 3,
11     PETSC_DECIDE, PETSC_DECIDE,
12     1, 1,
13     NULL, NULL,
14     &da));
15
```

```
PetscCall(DMSetFromOptions(da));
PetscCall(DMSetUp(da));

PetscCall(DMDASetUniformCoordinates(da, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0));
PetscCall(DMDASetFieldName(da, 0, "Solution"));

PetscCall(KSPSetDM(ksp, da));
PetscCall(KSPSetComputeRHS(ksp, ComputeRHS, NULL));
PetscCall(KSPSetComputeOperators(ksp, ComputeMatrix, NULL));

PetscCall(KSPSetFromOptions(ksp));
PetscCall(KSPSetUp(ksp));

PetscCall(KSPSolve(ksp, NULL, NULL));
```

The call to `DMDACreate2d` creates a two-dimensional distributed array. The options

`DM_BOUNDARY_NONE`

are used because the problem does not have periodic boundary conditions. The option

`DMDA_STENCIL_STAR`

corresponds to the five-point stencil of the finite-difference discretization: each interior point couples only to itself and to its north, south, east, and west neighbours. The stencil width is set to one, consistently with the second-order centred discretization.

The grid is initialized with default size 3×3, but this value can be overridden from the command line by PETSc options such as

Listing 5: Example of runtime grid-size selection

```
-da_grid_x 512 -da_grid_y 512
```

This is useful because it allows the same executable to be used for several problem resolutions without recompilation.

The right-hand side vector is assembled in the function `ComputeRHS`. Each MPI process first obtains the portion of the grid that it owns through `DMDAGetCorners`. Then it writes the corresponding local entries of the right-hand side vector. Boundary entries are set to zero, while interior entries are set to $20 h_x h_y$, consistently with the scaled finite-difference equation (1).

Listing 6: Assembly of the right-hand side vector

```
PetscCall(DMDAGetInfo(da, NULL, &mx, &my, NULL,
                      NULL, NULL, NULL, NULL,
                      NULL, NULL, NULL, NULL, NULL));

Hx = 1.0 / (PetscReal)(mx - 1);
Hy = 1.0 / (PetscReal)(my - 1);

PetscCall(DMDAGetCorners(da, &xs, &ys, NULL, &xm, &ym, NULL));
PetscCall(DMDAVecGetArray(da, b, &array));

for (j = ys; j < ys + ym; j++) {
  for (i = xs; i < xs + xm; i++) {

    if (i == 0 || j == 0 || i == mx - 1 || j == my - 1) {
      array[j][i] = 0.0;
    } else {
      array[j][i] = 20.0 * Hx * Hy;
    }

  }
}

```

```
PetscCall(DMDAVecRestoreArray(da, b, &array));
PetscCall(VecAssemblyBegin(b));
PetscCall(VecAssemblyEnd(b));
```

The matrix is assembled in the function `ComputeMatrix`. Boundary nodes are treated by inserting an identity row. For interior nodes, the five-point stencil (2) is inserted using `MatSetValuesStencil`. Using stencil indices avoids manually converting the two-dimensional grid indices (i, j) into global algebraic indices.

Listing 7: Assembly of the finite-difference matrix

```
Hx = 1.0 / (PetscReal)(mx - 1);
Hy = 1.0 / (PetscReal)(my - 1);

HxdHy = Hx / Hy;
HydHx = Hy / Hx;

PetscCall(DMDAGetCorners(da, &xs, &ys, NULL, &xm, &ym, NULL));

for (j = ys; j < ys + ym; j++) {
  for (i = xs; i < xs + xm; i++) {

    row.i = i;
    row.j = j;

    if (i == 0 || j == 0 || i == mx - 1 || j == my - 1) {

      v[0] = 1.0;
      PetscCall(MatSetValuesStencil(jac, 1, &row, 1, &row,
                                    v, INSERT_VALUES));

    } else {

      PetscInt ncols = 0;

      if (j - 1 > 0) {
        v[ncols]     = -HxdHy;
        col[ncols].i = i;
        col[ncols].j = j - 1;
        ncols++;
      }

      if (i - 1 > 0) {
        v[ncols]     = -HydHx;
        col[ncols].i = i - 1;
        col[ncols].j = j;
        ncols++;
      }

      v[ncols]     = 2.0 * (HxdHy + HydHx);
      col[ncols].i = i;
      col[ncols].j = j;
      ncols++;

      if (i + 1 < mx - 1) {
        v[ncols]     = -HydHx;
        col[ncols].i = i + 1;
        col[ncols].j = j;
        ncols++;
      }

      if (j + 1 < my - 1) {
        v[ncols]     = -HxdHy;
```

```
        col[ncols].i = i;
        col[ncols].j = j + 1;
        ncols++;
      }

      PetscCall(MatSetValuesStencil(jac, 1, &row, ncols, col,
                                    v, INSERT_VALUES));
    }
  }
}

PetscCall(MatAssemblyBegin(jac, MAT_FINAL_ASSEMBLY));
PetscCall(MatAssemblyEnd(jac, MAT_FINAL_ASSEMBLY));

PetscCall(MatSetOption(jac, MAT_SYMMETRIC, PETSC_TRUE));
```

Boundary neighbours are not inserted in the matrix for interior rows. This is consistent with the homogeneous Dirichlet condition, because boundary values are known and equal to zero. If the boundary condition were non-homogeneous, the contribution of neighbouring boundary values would instead have to be moved to the right-hand side.
Since the Dirichlet value is homogeneous, the coefficients coupling an interior node to a boundary neighbour are omitted. This is equivalent to moving the known boundary contribution to the right-hand side; here that contribution is zero. Together with identity rows for boundary nodes, this produces the same interior solution while preserving a symmetric constrained operator.

The implementation does not hard-code a specific Krylov method or preconditioner. Instead, it calls

`KSPSetFromOptions(ksp)`,

which allows the solver configuration to be chosen at runtime from the PETSc options database. Since the assembled operator is symmetric positive definite, a natural choice is the conjugate-gradient method. In the runs reported here, the linear system was solved with a Krylov method selected through PETSc options, for example

Listing 8: Example PETSc solver options for the Poisson problem

```
-ksp_type cg -pc_type jacobi -ksp_rtol 1e-10 -ksp_monitor
```

For larger grids, a multigrid preconditioner such as PETSc's algebraic multigrid preconditioner can also be used by changing only the runtime options, for example

Listing 9: Example PETSc options using algebraic multigrid

```
-ksp_type cg -pc_type gamg -ksp_rtol 1e-10 -ksp_monitor
```

This separation between the discretization code and the solver configuration is one of the main advantages of PETSc: the same application code can be tested with different iterative methods, tolerances, and preconditioners without modifying or recompiling the source code.

After solving the system, the numerical solution is written to an ASCII file containing the coordinates and the corresponding solution value. Since the solution vector is distributed across MPI processes, the output is performed using synchronized PETSc ASCII printing.

Listing 10: Writing the distributed solution to file

```
PetscCall(PetscViewerASCIIOpen(PETSC_COMM_WORLD, filename, &viewer));
PetscCall(PetscViewerASCIIPrintf(viewer, "# x y u\n"));

PetscCall(DMDAVecGetArrayRead(da, x, &array));
PetscCall(PetscViewerASCIIPushSynchronized(viewer));

for (j = ys; j < ys + ym; j++) {
  for (i = xs; i < xs + xm; i++) {
    PetscReal xcoord = ((PetscReal)i) * Hx;
    PetscReal ycoord = ((PetscReal)j) * Hy;
    PetscReal value  = PetscRealPart(array[j][i]);
```

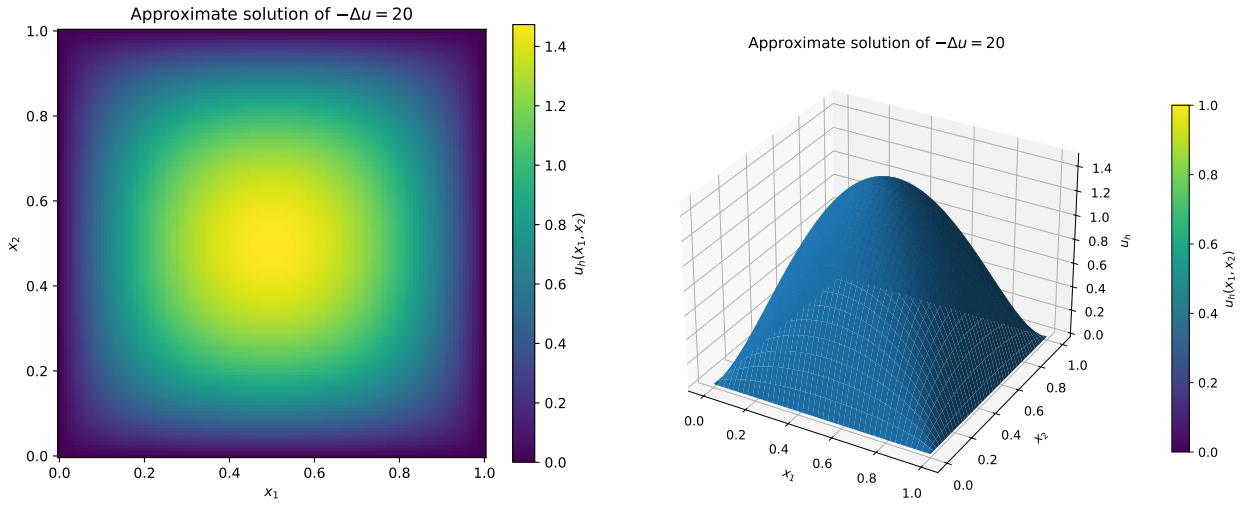
```
12
13      PetscCall(PetscViewerASCIISynchronizedPrintf(
14          viewer,
15          "%.16e %.16e %.16e\n",
16          (double)xcoord,
17          (double)ycoord,
18          (double)value));
19    }
20  }
21
22  PetscCall(PetscViewerFlush(viewer));
23  PetscCall(PetscViewerASCIIPopSynchronized(viewer));
24  PetscCall(DMDAVecRestoreArrayRead(da, x, &array));
25  PetscCall(PetscViewerDestroy(&viewer));
```

2.3. Visualization

The computed solution was exported as a file containing triples

$$(x_i, y_j, u_{i,j}),$$

which were then postprocessed in Python to generate a heatmap and a three- dimensional surface plot. These visualizations are shown in Figure 19.



(a) Heatmap of the numerical solution.

(b) Surface plot of the numerical solution.

Figure 19: Approximate solution of the Poisson problem $-\Delta u = 20$ in $\Omega = (0,1)^2$, with homogeneous Dirichlet boundary conditions on $\partial\Omega$. The solution is zero on the boundary and positive in the interior, with a maximum close to the centre of the domain, as expected from the symmetry of the problem and the positivity of the source term.

The plots confirm the expected qualitative structure of the solution. Because the forcing term is positive and the boundary values are zero, the discrete solution is positive inside the domain and vanishes on the boundary. Moreover, since both the geometry and the source term are symmetric with respect to the lines $x_1 = 1/2$ and $x_2 = 1/2$, the numerical solution is also symmetric. The maximum is attained near the centre of the square, while the solution decreases smoothly towards zero when approaching the boundary. This behaviour is consistent with the maximum principle for the Poisson equation and provides a qualitative validation of the finite-difference discretization and PETSc implementation.

2.4. Strong Scaling Analysis

The final part of the PETSc study consists of a strong scaling experiment for the two-dimensional Poisson solver.
The strong scaling experiment was performed on the Euler cluster using one MPI process per node. This

choice makes the scaling study a genuine multi-node MPI experiment and avoids mixing the effects of intra-node and inter-node parallelism.

The benchmark was run for the two representative grid resolutions

$$n = 512, \qquad n = 1024,$$

and for

$$p = 1, 2, 4, 8, 16, 32$$

MPI processes.
For each pair (n, p), three repetitions were performed and the median value was used in the plots. This reduces the influence of occasional runtime fluctuations caused by system noise, network contention, or temporary load variations on the cluster.

The PETSc linear solver was configured at runtime as

`-ksp_type cg -pc_type gamg -ksp_rtol 1e-8`.

This means that the linear system was solved using the conjugate-gradient method, preconditioned by PETSc's algebraic multigrid preconditioner `GAMG`. This is a suitable choice for the present problem because the finite-difference discretization of the negative Laplacian with homogeneous Dirichlet boundary conditions gives a symmetric positive definite linear system. Moreover, multigrid-type preconditioners are well suited for elliptic operators such as the Poisson operator, since they are designed to reduce errors over several length scales efficiently.

Figure 20 shows the median time to solution as a function of the number of MPI processes. The runtime decreases when increasing the process count, showing that the PETSc implementation is able to exploit parallelism. However, the scaling is not ideal over the full range of process counts. This is expected in a strong scaling setting: as p increases, the local subdomain assigned to each MPI rank becomes smaller, and the relative cost of communication and synchronization increases.

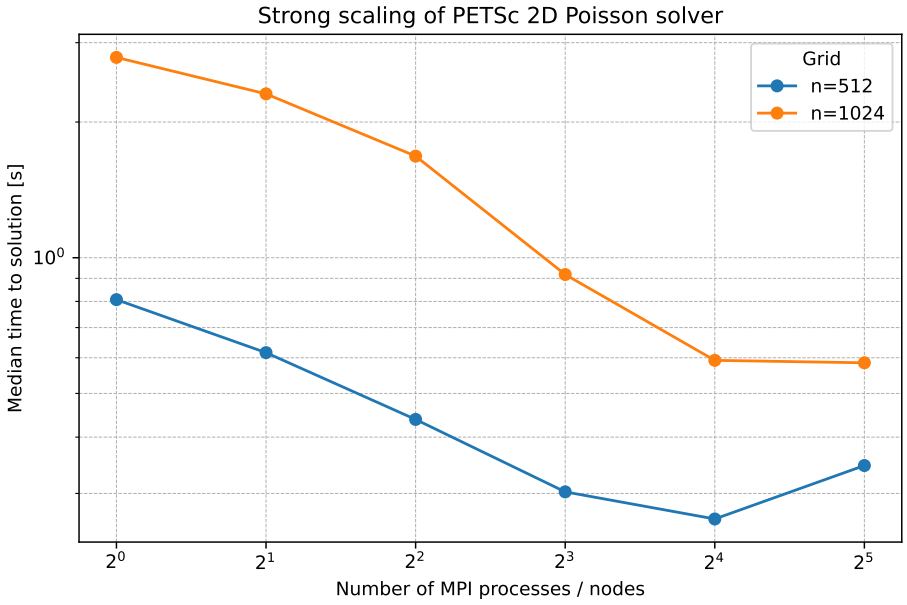


Figure 20: Strong scaling of the PETSc two-dimensional Poisson solver. The plot reports the median total time to solution for the grid resolutions $n = 512$ and $n = 1024$. The global problem size is fixed while the number of MPI processes, and therefore the number of nodes, is increased.

A useful way to interpret the result is to distinguish the total time from the linear-solve time. The total time includes not only the Krylov solve, but also the PETSc setup, creation of the distributed grid, matrix and vector assembly, and other initialization costs. By contrast, the solve time isolates the `KSPSolve` phase, which is the most relevant part for assessing the parallel performance of the iterative linear solver itself.

Figure 21 shows the strong scaling behaviour of the `KSPSolve` phase. The solve time decreases more clearly than the total time, especially for the larger resolution. This confirms that the parallelization of the algebraic solve is effective. The larger grid $n = 1024$ provides more work per MPI process than the smaller grid $n = 512$, and therefore has a better computation-to-communication ratio. As a result, it is expected to scale better to larger process counts.

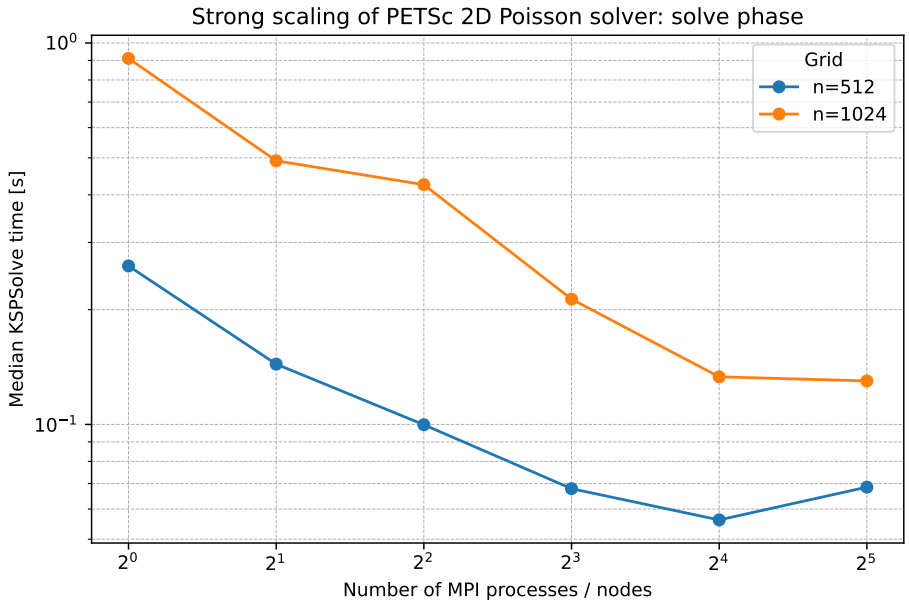


Figure 21: Strong scaling of the PETSc solver phase. The plot reports the median time spent inside `KSPSolve`. Compared with the total runtime, this isolates the performance of the Krylov solver and preconditioner from setup and output costs.

The difference between the two resolutions is also visible in the parallel efficiency, shown in Figure 22. For small numbers of processes, the efficiency remains relatively high, meaning that the additional nodes are still effectively used. As the number of processes grows, the efficiency decreases. This behaviour is typical of strong scaling: eventually, the local amount of work per process becomes too small to amortize the fixed overheads of communication, synchronization, and PETSc object management.

The efficiency degradation is more pronounced for the smaller grid $n = 512$. This is because, for a fixed number of processes, each MPI rank owns fewer grid points than in the $n = 1024$ case. Since the finite-difference operator uses only nearest-neighbour couplings, the computational work scales roughly with the number of local grid points, while communication is associated with the interfaces between neighbouring subdomains. When the local subdomains become small, the ratio between interface size and subdomain area increases. Consequently, communication and synchronization represent a larger fraction of the total runtime.

For the larger grid $n = 1024$, the scaling is better because each process has more local work to perform. This delays the onset of the overhead-dominated regime. In other words, the larger problem size provides enough computation to make the use of additional nodes more effective. This is the standard behaviour expected from a strong scaling study of a domain-decomposed PDE solver.

2.5. Use of LLMs

Chat GPT was heavily used to write the bash file and python file for performing the strong scaling figures. It was also used to understand the PETSc code and to write explanations in the report. The LLM was used to understand the mathematical formulation of the problem. The LLM was also used to generate the LaTeX code for the report, including the figures and tables.

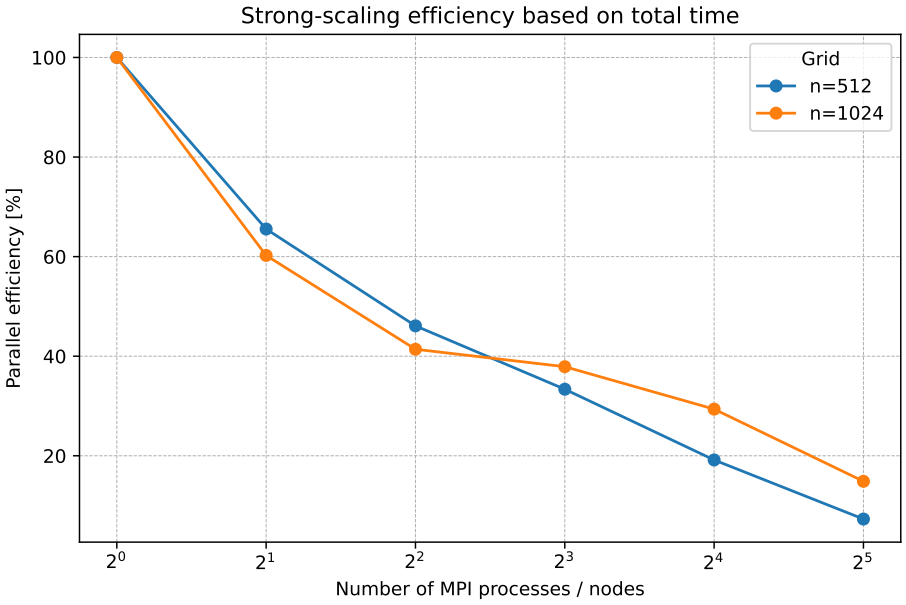


Figure 22: Parallel efficiency based on the median total time to solution. The efficiency is computed as $E_p = T_1/(pT_p)$. The decrease in efficiency at larger process counts reflects the increasing relative importance of communication and parallel overheads in the strong scaling regime.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 7

Due date: **Friday** 29 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. GPU architecture and performance characteristics [15 Points]

1.1. Peak performance [5 Points]

Using the provided utility `deviceQuery` we can retrieve the characteristics of the NVIDIA GeForce RTX 2080 Ti GPU. The output of the utility is reported in listing 1.

Listing 1: Output of the `deviceQuery` CUDA sample.

```
 CUDA Device Query (Runtime API) version (CUDART static linking)

Detected 1 CUDA Capable device(s)

Device 0: "NVIDIA␣GeForce␣RTX␣2080␣Ti"
  CUDA Driver Version / Runtime Version          13.0 / 13.0
  CUDA Capability Major/Minor version number:    7.5
  Total amount of global memory:                 10821 MBytes (11346575360
      bytes)
  (068) Multiprocessors, (064) CUDA Cores/MP:    4352 CUDA Cores
  GPU Max Clock rate:                            1545 MHz (1.54 GHz)
  Memory Clock rate:                             7000 Mhz
  Memory Bus Width:                              352-bit
  L2 Cache Size:                                 5767168 bytes
  Maximum Texture Dimension Size (x,y,z)         1D=(131072), 2D=(131072,
     65536), 3D=(16384, 16384, 16384)
  Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 2048 layers
  Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 32768), 2048
     layers
  Total amount of constant memory:               65536 bytes
  Total amount of shared memory per block:       49152 bytes
  Total shared memory per multiprocessor:        65536 bytes
  Total number of registers available per block: 65536
  Warp size:                                     32
  Maximum number of threads per multiprocessor:  1024
  Maximum number of threads per block:           1024
  Max dimension size of a thread block (x,y,z): (1024, 1024, 64)
  Max dimension size of a grid size    (x,y,z): (2147483647, 65535, 65535)
  Maximum memory pitch:                          2147483647 bytes
  Texture alignment:                             512 bytes
```

From listing 1 we can derive that:

- The number of Streaming Multiprocessors (SMs) is $n_{\text{SMs}} = 68$
- The number of CUDA cores per SM is $n_{\text{cores per SM}} = 64$
- The GPU maximumclock frequency is $f = 1.545$ GHz
- The CUDA Compute Capability is 7.5

1.2. Memory hierarchies and bandwidth [5 Points]

From [1] we can derive the following information about the number of floating point operations per FMA.

- For single precision (FP32) FMAs $TP_{\text{FP32}} = \frac{64}{64} = 1$
- For double precision (FP64) FMAs $TP_{\text{FP64}} = \frac{64}{2} = 32$

Thus we can compute the theoretical peak performance of an NVIDIA GeForce RTX 2080 Ti for single and double precision as:

$$\begin{aligned} P_{\text{FP32}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP32}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{1} \cdot 2\text{FP32} \cdot 1.545\text{GHz} = 13450,56 \text{ GFLOPS} \\ P_{\text{FP64}} &= n_{\text{SMs}} \cdot \frac{n_{\text{cores per SM}}}{TP_{\text{FP64}}} \cdot n_{FMA} \cdot f \\ &= 68 \cdot \frac{64}{32} \cdot 2\text{FP64} \cdot 1.545\text{GHz} = 420,33 \text{ GFLOPS} \end{aligned}$$

From Listing 1 we can get the following informations

- Global Memory : 10821 MiB = 10.57 GiB
- L2 Cache Size: 5.5 MiB
- L1 and Shared Memory per SM 96 KiB
- Read only memory 64 KiB
- Registers 64 Ki registers per block, each holding 4 bytes (total 256 KiB).
- Memory bus width: $W_{\text{mem}} = 352$ bits
- Memory clock frequency: $f_{\text{mem}} = 7$ GHz

So we can compute the (theoretical) peak memory bandwidth of a GPU as:

$$\begin{aligned} B_{\text{GPU}} &= \frac{W_{\text{mem}}}{8} \cdot f_{\text{mem}} \cdot 2 \\ &= \frac{352}{8} \cdot 7\text{GHz} \cdot 2 = 616 \text{ GB/s} \end{aligned}$$

1.3. Performance model: A simple roofline model [5 Points]

Compute ridge points

$$I_{\text{ridge}} = \frac{P_{\text{peak}}}{B_{\text{GPU}}}$$

For single precision:

$$I_{\text{ridge, FP32}} = \frac{13450.56 \text{ GFLOPS}}{616 \text{ GB/s}} = 21.83 \text{ GLOP/byte}$$

For double precision:

$$I_{\text{ridge, FP64}} = \frac{420.33 \text{ GFLOPS}}{616 \text{ GB/s}} = 0.68 \text{ GLOP/byte}$$

We can retrieve a roofline plot for the NVIDIA GeForce RTX 2080 Ti remembering that

$$P_{\text{attainable}}(I) = \min\left(P_{\text{peak}}, B_{\text{GPU}} I\right)$$

.
In figure 1 we can see the roofline plot for the NVIDIA GeForce RTX 2080 Ti for single and double precision.

Thus, when the operational intensity of a kernel is lower than I_{ridge} the kernel is memory bound, otherwise it is compute bound.

2. Introduction to CUDA [50 Points]

2.1. Implement the simple kernels [30 Points]

The CUDA implementation of the functions in the file `simple_kernels.cu` is straightforward and it is based on the standard CUDA programming model. For everty function we first computed the correct index as `int i = blockIdx.x * blockDim.x + threadIdx.x` and then we check if the index is within the bounds of the input vectors by `if ( i < N )`. If the index is valid we perform the corresponding operation on the input

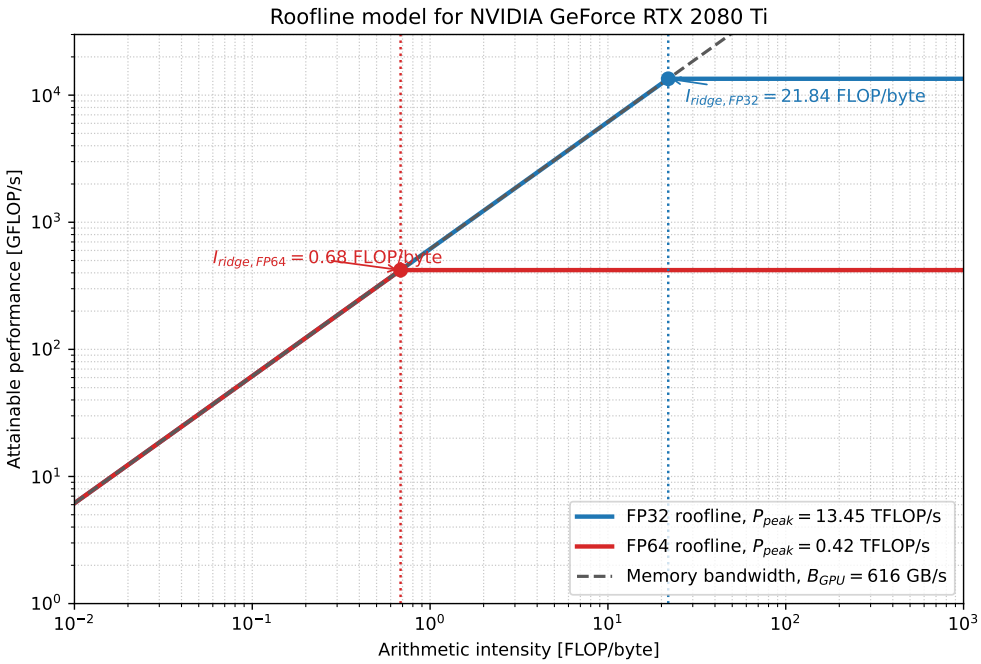


Figure 1: Roofline plot for the NVIDIA GeForce RTX 2080 Ti for single and double precision.

vectors and we store the result in the output vector.
In this case, since in the `benchmark.cu` file the number of blocks is computed as `int blocks = (SIZE + threads - 1) / threads` and the number of threads per block is set to 256, we do not need in principle to worrry about the number of threads and blocks, since the kernel will be launched with enough threads to cover all the elements of the input vectors. However, we need to check if the index is within the bounds of the input vectors to avoid out-of-bounds memory accesses.

The output of the benchmark for the double format is reported in listing 2 and for the float format in listing 3.
From 2 we can retrieve the points in table 1 and from 3 we can retrieve the points in table 2.

Listing 2: Output of the simple kernels for the double format.

```
[benchmark] axpy: warmup=5 runs=25 avg_time=0.363086 ms
Compute utilization = 46.2073 GFLOPS/s
Memory utilization = 554.487 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.477494 ms
Compute utilization = 52.704 GFLOPS/s
Memory utilization = 562.176 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.363736 ms
Compute utilization = 46.1247 GFLOPS/s
Memory utilization = 553.496 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.247412 ms
Compute utilization = 33.9054 GFLOPS/s
Memory utilization = 542.486 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.363924 ms
Compute utilization = 69.1512 GFLOPS/s
Memory utilization = 553.21 GB/s
first = 1.5, last = 1.5, expected = 1.5
```

Listing 3: Output of the simple kernels for the float format.

```
[benchmark] axpy: warmup=5 runs=25 avg_time=0.182811 ms
```

Kernel	Compute Utilization (GFLOPS/s)	Arithmetic Intensity (FLOP/byte)
axpy	46.2073	$2/(3 \cdot 8)$
add scaled diff	52.704	$3/(4 \cdot 8)$
scaled diff	46.1247	$2/(3 \cdot 8)$
scale	33.9054	$1/(2 \cdot 8)$
linear combination	69.1512	$3/(3 \cdot 8)$

Table 1: Compute utilization and arithmetic intensity for the simple kernels in double precision.

Kernel	Compute Utilization (GFLOPS/s)	Arithmetic Intensity (FLOP/byte)
axpy	91.7736	$2/(3 \cdot 4)$
add scaled diff	104.988	$3/(4 \cdot 4)$
scaled diff	91.798	$2/(3 \cdot 4)$
scale	67.0205	$1/(2 \cdot 4)$
linear combination	137.747	$3/(3 \cdot 4)$

Table 2: Compute utilization and arithmetic intensity for the simple kernels in single precision.

```
Compute utilization = 91.7736 GFLOPS/s
Memory utilization = 550.642 GB/s
first = 4, last = 4, expected = 4
[benchmark] add scaled diff: warmup=5 runs=25 avg_time=0.239703 ms
Compute utilization = 104.988 GFLOPS/s
Memory utilization = 559.933 GB/s
first = 1.6, last = 1.6, expected = 1.6
[benchmark] scaled diff: warmup=5 runs=25 avg_time=0.182762 ms
Compute utilization = 91.798 GFLOPS/s
Memory utilization = 550.788 GB/s
first = 0.6, last = 0.6, expected = 0.6
[benchmark] scale: warmup=5 runs=25 avg_time=0.125165 ms
Compute utilization = 67.0205 GFLOPS/s
Memory utilization = 536.164 GB/s
first = 0.1, last = 0.1, expected = 0.1
[benchmark] linear combination: warmup=5 runs=25 avg_time=0.182696 ms
Compute utilization = 137.747 GFLOPS/s
Memory utilization = 550.989 GB/s
first = 1.5, last = 1.5, expected = 1.5
```

Figure **??** shows that all simple linear algebra kernels are located far to the left of the ridge points. Therefore, their performance is limited by global memory bandwidth rather than by the peak floating-point throughput of the GPU. This is also consistent with the measured memory bandwidths, which are close to the theoretical peak bandwidth of 616 GB/s. The single-precision kernels reach higher GFLOP/s values than the corresponding double-precision kernels because each FP32 value occupies only 4 bytes instead of 8 bytes. Hence, for the same memory bandwidth, the operational intensity and the attainable floating-point rate are approximately twice as large.

2.2. Implement the scalar product kernel [20 Points]

The first CUDA implementation of the scalar product is very simple: each thread computes some products and directly adds them to the final result using `atomicAdd`. This version is correct, but it is slow because many threads try to update the same memory location.

Listing 4: Simple CUDA kernel for the scalar product.

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
int stride = blockDim.x * gridDim.x;

// Each thread works on more than one element, jumping by the total
// number of launched threads.
for (; i < N; i += stride) {
    // Simple but expensive: all threads update the same global result.
```

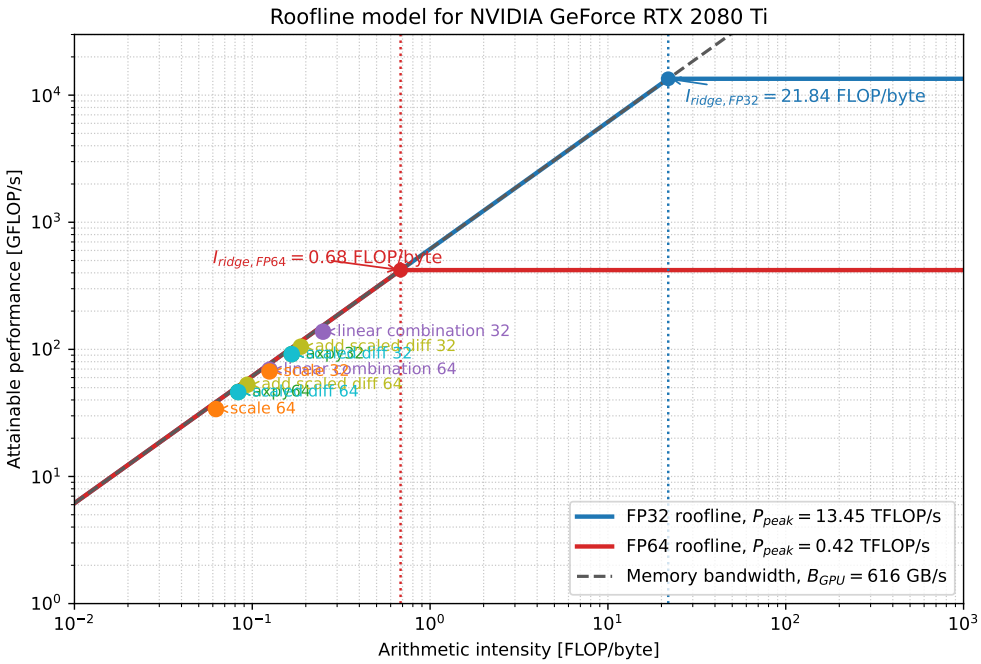


Figure 2: Roofline plot for the simple kernels in single and double precision. The points corresponding to the kernels are highlighted in red.

```
    atomicAdd(result, x[i] * y[i]);
}
```

The optimized version first accumulates a local sum inside each thread. Then the partial sums of the threads in the same block are reduced using shared memory. Only one `atomicAdd` per block is performed, which strongly reduces the contention on the global result.
An optimized version is the one in listing 5.

Listing 5: Optimized CUDA kernel for the scalar product.

```
__global__ void hpc_dot(real_t* const x, real_t* const y, real_t* result,
                        size_t const N) {

    // One partial sum per thread in the block.
    __shared__ real_t smem[256];

    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    // Each thread accumulates its own part of the dot product.
    real_t local_sum = 0.0;
    for (int idx = i; idx < N; idx += stride) {
        local_sum += x[idx] * y[idx];
    }

    // Store the thread-local result in shared memory.
    smem[tid] = local_sum;
    __syncthreads();

    // Parallel reduction inside the block.
    for (int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            smem[tid] += smem[tid + s];
        }
        __syncthreads();
```

```
27      }
28
29      // Only one thread per block updates the global result.
30      if (tid == 0) {
31          atomicAdd(result, smem[0]);
32      }
33  }
```

In table 3 we can see the compute and memory utilization for the scalar product kernel in double and single precision for both the implementations.

Version	Precision	Compute Utilization (GFLOP/s)	Arithmetic Intensity (FLOP/byte)
Simple atomic	Double	1.22579	$2/(4 \cdot 8)$
Simple atomic	Float	1.22839	$2/(4 \cdot 4)$
Block reduction	Double	71.948	$2/(2 \cdot 8)$
Block reduction	Float	140.226	$2/(2 \cdot 4)$

Table 3: Roofline points for the scalar product kernel. The block-reduction version has a higher arithmetic intensity because the main global memory traffic is due to the two input vector reads, while the global atomic update is performed only once per block.

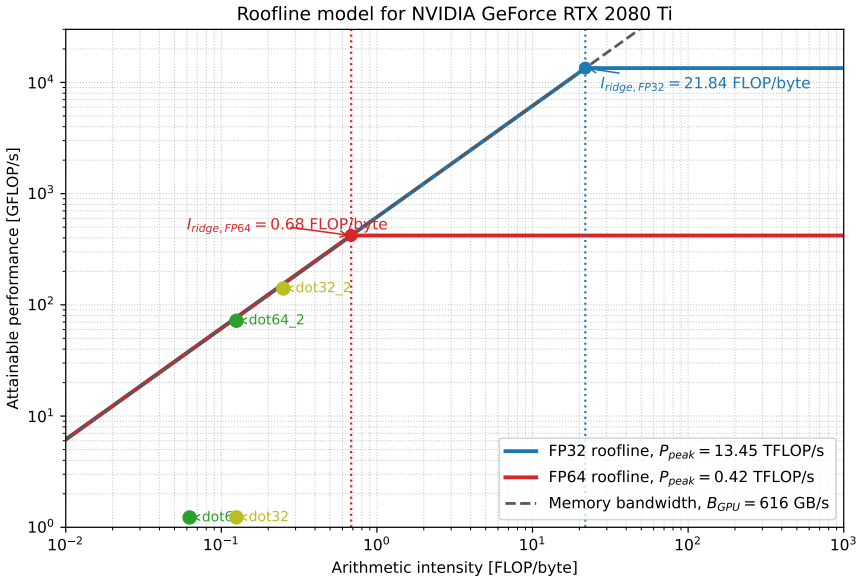


Figure 3: Roofline plot for the scalar product kernel in single and double precision. The simple atomic version is very slow because many threads contend for the same global result. The block-reduction version moves upward in the roofline plot because it performs most of the reduction in shared memory and uses only one global atomic update per block.

3. Non-linear PDE mini-app with CUDA: The Fisher equation [35 Points]

The linear algebra kernels were implemented using the same approach as for the simple kernels, thus we do not report the implementation of the kernels in this report.

The implementation of the stencil operator is reported in listing 6. Each CUDA thread computes one grid point. The kernel first ignores the external layer of points that is not updated. Then it selects the four stencil values: for an internal neighbor it reads from `s_new`, while for a boundary neighbor it uses the corresponding boundary array. This keeps the formula compact and avoids writing separate cases for all edges and corners.

Listing 6: CUDA kernel for the stencil operator.

```
__global__ void diffusion(real_t* const s_old,
                real_t* const s_new,
                real_t* f,
                real_t alpha,
                real_t beta,
                real_t* const bndE,
                real_t* const bndW,
                real_t* const bndN,
                real_t* const bndS,
                int nx) {

    int iend = nx - 1;
    int jend = nx - 1;

    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Ignore threads outside the active computational domain.
    // The CUDA grid may contain more threads than needed.
    if (i < 1 || i > iend || j < 1 || j > jend) {
        return;
    }

    int idx = i + j * nx;
    real_t center = s_new[idx];

    // Use boundary values when the stencil reaches the physical boundary.
    // Otherwise, use the neighboring value from the grid.
    real_t west  = (i == 1)    ? bndW[j] : s_new[idx - 1];
    real_t east  = (i == iend) ? bndE[j] : s_new[idx + 1];
    real_t south = (j == 1)    ? bndS[i] : s_new[idx - nx];
    real_t north = (j == jend) ? bndN[i] : s_new[idx + nx];

    // Same stencil formula for internal points, edges, and corners.
    f[idx] = -(4.0 + alpha) * center
          + west + east + south + north
          + beta * center * (1.0 - center)
          + alpha * s_old[idx];
}
```

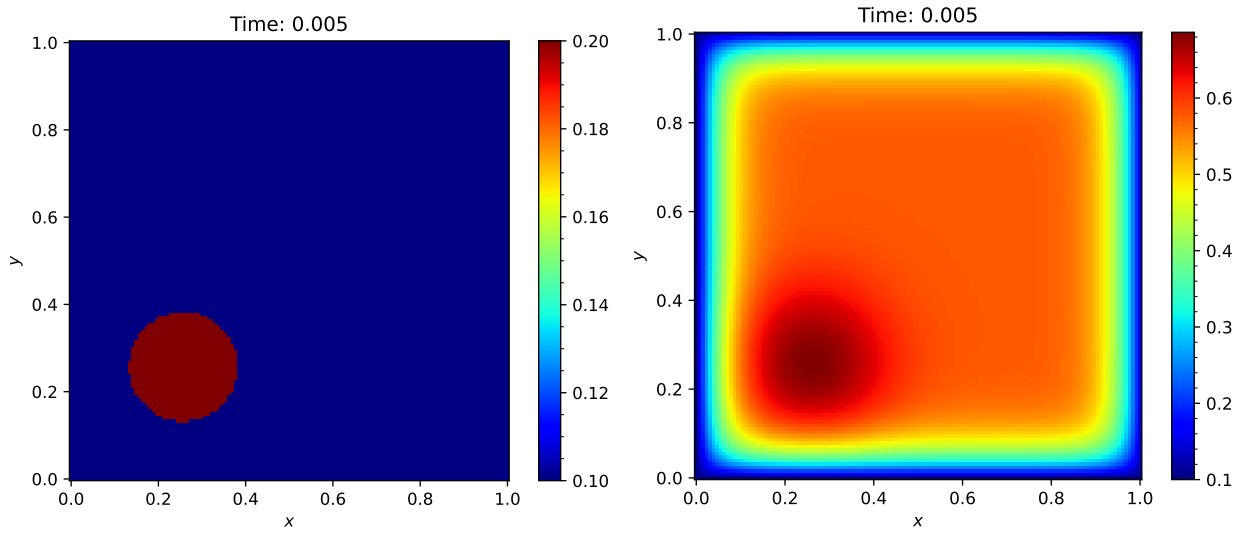
In listing 7 we can see the output of the benchmark for the Fisher equation mini-app.

The code works correctly as we can see from the plots in figure 4 where we can see the initial and final solution of the simulation. This is consistent with the expected behavior of the Fisher equation, which describes the diffusion and growth of a population over time.

Listing 7: Output of the Fisher equation mini-app.

```
================================================================
```

```
                        Welcome to mini-stencil!
version   :: CUDA
mesh      :: 128 * 128 dx = 0.00787402
time      :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
--------------------------------------------------------------------------------
simulation took 0.114354 seconds
1516 conjugate gradient iterations, at rate of 13239.6  iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 128, 100, 1514, 300, 0.114354 ###
Goodbye!
```



(a) Initial solution

(b) Final solution

Figure 4: Comparison between initial and final solution

References

[1] NVIDIA Corporation. *CUDA C++ Best Practices Guide*. Version 13.2. 2026. URL: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/ (visited on 05/21/2026).